DEVELOPMENT OF A SENSITIVITY-AWARE REINFORCEMENT LEARNING FRAMEWORK FOR LAYER-WISE CNN PRUNING

RABAB KHAN RONGON

M.Sc. ENGINEERING THESIS

[MIST MONOGRAM
28 mm × 30 mm]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY
DHAKA, BANGLADESH

[MONTH YEAR]

DEVELOPMENT OF A SENSITIVITY-AWARE REINFORCEMENT LEARNING FRAMEWORK FOR LAYER-WISE CNN PRUNING

RABAB KHAN RONGON (SN. [STUDENT NUMBER])

A Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of Master of Science in Engineering in Computer Science and Engineering

[MIST MONOGRAM
28 mm × 30 mm]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY
DHAKA, BANGLADESH

[MONTH YEAR]

DEVELOPMENT OF A SENSITIVITY-AWARE REINFORCEMENT LEARNING FRAMEWORK FOR LAYER-WISE CNN PRUNING

M.Sc. Engineering Thesis
By
RABAB KHAN RONGON (SN. [STUDENT NUMBER])

Approved as to style and content by the Board of Examination on [DATE OF EXAMINATION]:

_____
[NAME OF SUPERVISOR]
[Designation], [Department]
[Institution], Dhaka

Chairman (Supervisor)
Board of Examination

_____
[NAME OF MEMBER]
[Designation], [Department]
[Institution], Dhaka

Member
Board of Examination

_____
[NAME OF EXTERNAL MEMBER]
[Designation], [Department]
[Institution]

Member (External)
Board of Examination

_____
[NAME OF HEAD OF THE DEPARTMENT]
[Designation], Department of Computer Science and Engineering
MIST, Dhaka

Member (Ex-officio)
Head of the Department

Department of Computer Science and Engineering, MIST, Dhaka

DEVELOPMENT OF A SENSITIVITY-AWARE REINFORCEMENT LEARNING FRAMEWORK FOR LAYER-WISE CNN PRUNING

DECLARATION

I hereby declare that the study reported in this thesis entitled as above is my own original work and has not been submitted before anywhere for any degree or other purposes. Further I certify that the intellectual content of this thesis is the product of my own work and that all the assistance received in preparing this thesis and sources have been acknowledged and/or cited in the reference Section.

_____

Rabab Khan Rongon

Department of Computer Science and Engineering, MIST, Dhaka

ABSTRACT

Development of a Sensitivity-Aware Reinforcement Learning Framework for Layer-Wise CNN Pruning

Neural-network pruning reduces model size and computation, but deciding how much each layer should lose is difficult, and weight sparsity alone does not establish a deployment benefit. This thesis develops and evaluates a sensitivity-aware reinforcement-learning framework for the layer-wise pruning of convolutional neural networks. The study covers the CIFAR-10 and CIFAR-100 datasets, three small convolutional networks and one laptop CPU.

A Proximal Policy Optimisation (PPO) agent assigns a pruning ratio to each layer or layer group, guided by layer sensitivity. Successive experiments with pre-specified decision criteria added a centred sensitivity prior, archive-based selection of the policies visited during search, an accuracy-constrained reward and an equal post-pruning recovery protocol. A later deployment extension realised the pruning structurally, measured CPU latency and package energy directly, and trained a deployment-oriented structured agent.

The learned allocations substantially outperformed uniform and random allocation, by up to approximately 31.4 percentage points of test accuracy over uniform pruning. The deterministic layer-adaptive magnitude-based pruning (LAMP) method nevertheless remained more accurate before equal recovery. Sensitivity guidance improved search and exploration measures, not final accuracy. The agent found high-ranking policies but rarely ended on them, and archive selection restored their retention on validation ranking. The accuracy-constrained reward reduced destructive over-pruning, although feasibility certified on validation data held on the test set in only about two thirds of runs. Equal five-epoch recovery narrowed the differences between PPO and LAMP to within a pre-registered ±0.5 percentage-point decision margin in every setting. In the deployment analysis, parameter reduction proved a weak predictor of measured latency compared with the reduction in multiply–accumulate operations. Structured pruning reduced directly measured CPU package energy, but with accuracy losses beyond the pre-registered limit. The deployment-oriented structured agent reduced parameter memory by 63.4% and multiply–accumulate operations by 36.0% at a 1.12 percentage-point

validation-accuracy cost. It missed its accuracy criterion, and its direct energy remained unresolved because the measurement setup failed its stability qualification.

The novelty of the thesis lies not in a new pruning criterion but in treating adaptive pruning as a chain of connected problems (allocation, search guidance, policy retention, reward alignment, recovery and hardware validation) and in examining each with its own experiment and evidence. The results show that an improvement at one stage does not guarantee success at the next. Each stage must therefore be validated before a compression result can be called a deployment-efficiency result.

Keywords: neural network pruning; reinforcement learning; proximal policy optimisation; layer-wise pruning; sensitivity-aware pruning; structured pruning; model compression; deployment efficiency

সারসংক্ষেপ

Development of a Sensitivity-Aware Reinforcement Learning Framework for Layer-Wise CNN Pruning

নিউরাল নেটওয়ার্ক প্রুনিং (neural network pruning) মডেলের আকার ও গণনার পরিমাণ কমায়, কিন্তু প্রতিটি স্তর (layer) থেকে কতটুকু অপসারণ করা উচিত তা নির্ধারণ করা কঠিন, এবং কেবল ওজন-বিরলতা (weight sparsity) দিয়ে বাস্তব প্রয়োগে (deployment) সুবিধা প্রমাণিত হয় না। এই থিসিসে কনভোলিউশনাল নিউরাল নেটওয়ার্কের (convolutional neural network) স্তরভিত্তিক প্রুনিংয়ের জন্য একটি সংবেদনশীলতা-সচেতন (sensitivity-aware) রিইনফোর্সমেন্ট লার্নিং (reinforcement learning) কাঠামো উদ্ভাবন ও মূল্যায়ন করা হয়েছে। গবেষণার পরিধিতে রয়েছে CIFAR-10 ও CIFAR-100 ডেটাসেট, তিনটি ছোট কনভোলিউশনাল নেটওয়ার্ক এবং একটি ল্যাপটপ সিপিইউ (CPU)।

একটি প্রক্সিমাল পলিসি অপটিমাইজেশন (proximal policy optimisation, PPO) এজেন্ট স্তর-সংবেদনশীলতার (layer sensitivity) নির্দেশনায় প্রতিটি স্তর বা স্তরগুচ্ছকে একটি প্রুনিং অনুপাত (pruning ratio) নির্ধারণ করে। পূর্বনির্ধারিত সিদ্ধান্ত-মানদণ্ডসহ ধারাবাহিক পরীক্ষার মাধ্যমে এতে যুক্ত করা হয়েছে একটি কেন্দ্রীকৃত সংবেদনশীলতা পূর্বধারণা (centred sensitivity prior), অনুসন্ধানের সময় পরিদর্শিত নীতিগুলোর (policies) আর্কাইভভিত্তিক নির্বাচন (archive-based selection), একটি নির্ভুলতা-সীমাবদ্ধ পুরস্কার (accuracy-constrained reward) এবং প্রুনিং-পরবর্তী সমান পুনরুদ্ধার প্রোটোকল (equal recovery protocol)। পরবর্তী একটি প্রয়োগমুখী সম্প্রসারণে প্রুনিংকে কাঠামোগতভাবে (structured pruning) বাস্তবায়ন করা হয়েছে, সিপিইউ লেটেন্সি (latency) ও প্যাকেজ শক্তি (package energy) সরাসরি পরিমাপ করা হয়েছে, এবং একটি প্রয়োগমুখী কাঠামোগত এজেন্ট (deployment-oriented structured agent) প্রশিক্ষণ দেওয়া হয়েছে।

শেখা বণ্টনগুলো (allocations) সুষম (uniform) ও দৈব (random) বণ্টনের চেয়ে উল্লেখযোগ্যভাবে ভালো ফল দিয়েছে; সুষম প্রুনিংয়ের তুলনায় টেস্ট নির্ভুলতায় (test accuracy) এই ব্যবধান সর্বোচ্চ প্রায় ৩১.৪ শতাংশ-বিন্দু (percentage points)। তবে নির্ধারণমূলক লেয়ার-অ্যাডাপ্টিভ ম্যাগনিচুড-ভিত্তিক প্রুনিং (layer-adaptive magnitude-based pruning, LAMP) পদ্ধতি সমান পুনরুদ্ধারের আগে অধিক নির্ভুল ছিল। সংবেদনশীলতা নির্দেশনা অনুসন্ধান ও অন্বেষণের (exploration) পরিমাপ উন্নত করেছে, চূড়ান্ত নির্ভুলতা নয়। এজেন্ট উচ্চ-র‍্যাংকের নীতি খুঁজে পেলেও কদাচিৎ সেগুলোতে শেষ করেছে, এবং

আর্কাইভ নির্বাচন ভ্যালিডেশন র‍্যাংকিংয়ের (validation ranking) ভিত্তিতে সেগুলোর ধারণ (retention) পুনরুদ্ধার করেছে। নির্ভুলতা-সীমাবদ্ধ পুরস্কার ধ্বংসাত্মক অতিপ্রুনিং (over-pruning) কমিয়েছে, যদিও ভ্যালিডেশন ডেটায় প্রত্যয়িত সম্ভাব্যতা (feasibility) টেস্ট সেটে মাত্র প্রায় দুই-তৃতীয়াংশ রানে বজায় ছিল। পাঁচ ইপকের (epoch) সমান পুনরুদ্ধার প্রতিটি সেটিংয়ে PPO ও LAMP-এর পার্থক্যকে পূর্বনিবন্ধিত (pre-registered) ±০.৫ শতাংশ-বিন্দু সিদ্ধান্ত-সীমার (decision margin) মধ্যে নামিয়ে এনেছে। প্রয়োগ বিশ্লেষণে দেখা গেছে, গুণ-যোগ অপারেশন (multiply–accumulate operations) হ্রাসের তুলনায় প্যারামিটার হ্রাস (parameter reduction) পরিমাপকৃত লেটেন্সির একটি দুর্বল পূর্বাভাসক। কাঠামোগত প্রুনিং সরাসরি পরিমাপকৃত সিপিইউ প্যাকেজ শক্তি কমিয়েছে, তবে পূর্বনিবন্ধিত সীমার চেয়ে বেশি নির্ভুলতা-ক্ষতির বিনিময়ে। প্রয়োগমুখী কাঠামোগত এজেন্ট ১.১২ শতাংশ-বিন্দু ভ্যালিডেশন-নির্ভুলতা (validation accuracy) ব্যয়ে প্যারামিটার মেমরি ৬৩.৪% এবং গুণ-যোগ অপারেশন ৩৬.০% কমিয়েছে। এটি তার নির্ভুলতা-মানদণ্ড পূরণ করেনি, এবং পরিমাপ ব্যবস্থাটি স্থিতিশীলতা যোগ্যতা-যাচাইয়ে (stability qualification) ব্যর্থ হওয়ায় এর সরাসরি শক্তি-ফলাফল অমীমাংসিত রয়ে গেছে।

এই থিসিসের নতুনত্ব কোনো নতুন প্রুনিং মানদণ্ডে নয়; বরং অভিযোজিত প্রুনিংকে (adaptive pruning) পরস্পর-সংযুক্ত সমস্যার একটি শৃঙ্খল হিসেবে বিবেচনা করা (বণ্টন, অনুসন্ধান-নির্দেশনা, নীতি-ধারণ, পুরস্কার-সামঞ্জস্য, পুনরুদ্ধার ও হার্ডওয়্যার যাচাই) এবং প্রতিটিকে তার নিজস্ব পরীক্ষা ও প্রমাণ দিয়ে পরীক্ষা করাই এর মূল অবদান। ফলাফল দেখায় যে এক ধাপের উন্নতি পরবর্তী ধাপের সাফল্য নিশ্চিত করে না। তাই কোনো সংকোচন-ফলাফলকে (compression result) প্রয়োগ-দক্ষতার (deployment efficiency) ফলাফল বলার আগে প্রতিটি ধাপ আলাদাভাবে যাচাই করতে হবে।

ACKNOWLEDGEMENTS

[ACKNOWLEDGEMENTS TO BE WRITTEN BY THE AUTHOR. MIST TP.13 lists, in order: (a) the supervisor [SUPERVISOR] and co-supervisor, if any; (b) peers and colleagues in the laboratory; (c) other academic, technical or support staff of the Department of Computer Science and Engineering; (d) collaborators in other departments, institutions or organisations; (e) any financial assistance (fellowships, grants or contracts); (f) family and friends.]

TABLE OF CONTENTS

Abstract i
Abstract in Bangla (সারসংক্ষেপ) iii
Acknowledgements v
Table of Contents vi
List of Figures xi
List of Tables xii
List of Main Notation and Abbreviations xiv
CHAPTER 1: INTRODUCTION 1
1.1 Background 1
1.2 Motivation 2
1.3 Problem Statement 4
1.4 Research Gap 5
1.5 Research Questions 6
1.6 Research Objectives 7
1.7 Research Contributions 7
1.8 Scope and Boundaries 10
1.9 Thesis Organisation 11
1.10 Chapter Summary 11
CHAPTER 2: LITERATURE REVIEW 12
2.1 Neural Network Compression 12
2.2 Unstructured and Structured Pruning 13
2.3 Layer-Wise Adaptive Pruning 14
2.4 Magnitude-Based and Deterministic Pruning 15
2.5 LAMP and Related Layer-Adaptive Methods 16
2.6 Reinforcement Learning for Neural Network Compression 17
2.7 PPO-Based Optimisation 21
2.8 Sensitivity-Aware Pruning 21
2.9 Search and Policy Selection in Pruning 22
2.10 Post-Pruning Recovery and Fine-Tuning 23
2.11 Hardware-Aware Neural Network Compression 24
2.12 Parameter Count vs MACs/FLOPs vs Real Latency 24
2.13 Energy-Efficient and Sustainable Neural Network Inference 25
2.14 Research Gap 27
CHAPTER 3: PROPOSED FRAMEWORK 29
3.1 General 29
3.2 Problem Formulation 32
3.3 Layer-Wise Pruning Representation 32
3.4 Reinforcement Learning Formulation 33
3.5 State Representation 34

3.6 Action Space 35
3.7 Initial Reward Function 36
3.8 Layer Sensitivity Analysis 36
3.9 Sensitivity-Guided Action Prior 37
3.10 Centred Sensitivity Prior 38
3.11 Policy-Retention Problem 38
3.12 Archive-Based Policy Selection 39
3.12.1 Archive (Phase 4) 39
3.12.2 Selection rule (Phase 4) 39
3.12.3 Elite behavioural cloning (Phase 4 only) 39
3.12.4 Constraint-aware selection (Phase 5) 40
3.13 Reward Misalignment and Accuracy-Constrained Reward 40
3.13.1 Why R0 can encourage excessive pruning 40
3.13.2 Accuracy-constrained reward (Phase 5) 41
3.14 Final Core Framework 41
3.15 Equal Recovery and Fine-Tuning Protocol 42
3.16 Deployment-Oriented Structured Extension 43
3.16.1 From masks to physical structure 43
3.16.2 Measurement: three levels of deployment efficiency 44
3.16.3 Two uses of the extension 44
3.17 Overall Algorithm 45
3.18 Computational Workflow and Reproducibility 47
3.19 Chapter Summary 48
CHAPTER 4: EXPERIMENTAL SETUP 49
4.1 Experimental Design Overview 49
4.2 Datasets 49
4.3 Dataset Partitioning and Validation Protocol 49
4.3.1 Training, validation and test 49
4.3.2 Two validation configurations 50
4.3.3 Use of each split by phase 50
4.4 Neural Network Architectures 51
4.5 Dense Reference Models 52
4.6 Pruning Units and Policy Spaces 53
4.6.1 Masked pruning units (core framework) 53
4.6.2 Physically compacted structured units (deployment extension) 53
4.6.3 Policy spaces and enumeration 54
4.7 Baseline Pruning Methods 54
4.7.1 Core-framework baselines 54
4.7.2 Deployment-extension comparators 55
4.7.3 Matching rules 55
4.8 PPO Training Configuration 56

4.9 Experimental Conditions by Phase 57
4.9.1 Phase 3 59
4.9.2 Phase 4 59
4.9.3 Phase 5 60
4.9.4 Phase 6 60
4.9.5 Paper 3 60
4.9.6 Phase 7A 61
4.9.7 Phase 7B 61
4.9.8 Phase 7B energy confirmation 62
4.10 Sensitivity Conditions and Controls 62
4.11 Archive and Selection Protocols 62
4.12 Reward Configurations 63
4.13 Recovery and Fine-Tuning Protocol 63
4.14 Evaluation Metrics 64
4.15 Statistical Analysis and Decision Criteria 66
4.15.1 Paired tests (Phases 3–6) 66
4.15.2 The ±0.5 pp rule of Phase 6 67
4.15.3 Deployment-extension statistics 67
4.15.4 Pre-registered decision rules 68
4.16 Hardware and Software Environment 68
4.17 Structured Pruning and Latency Protocol 69
4.18 Energy Measurement Protocol 70
4.18.1 Backend 70
4.18.2 Phase 7A 70
4.18.3 Phase 7B energy surrogate 70
4.18.4 Phase 7B confirmatory measurement and its stopping rule 71
4.18.5 Energy confirmation study 71
4.19 Reproducibility and Pre-Registration 72
4.20 Chapter Summary 72
CHAPTER 5: RESULTS OF THE CORE FRAMEWORK 73
5.1 General 73
5.2 Dense Reference Performance 73
5.3 Phase 3 — PPO versus Baselines 73
5.4 Phase 3 — Effect of Sensitivity Guidance 77
5.5 Phase 4 — Centred Sensitivity and Exploration 78
5.6 Phase 4 — Policy Retention and Archive Selection 79
5.7 Phase 4 — Reward Misalignment 81
5.8 Phase 5 — Accuracy-Constrained Reward 82
5.9 Phase 5 — Validation-to-Test Transfer 86
5.10 Phase 6 — Equal Fine-Tuning 87
5.11 Answers to RQ1–RQ5 91

5.12 Chapter Summary 92
CHAPTER 6: DEPLOYMENT AND SUSTAINABILITY ANALYSIS 93
6.1 General 93
6.2 From Compression to Deployment Efficiency 93
6.3 Structured Physical Pruning Setup 94
6.4 Exhaustive CPU Latency Analysis 94
6.5 Parameter Reduction versus MAC Reduction versus Latency 96
6.6 Accuracy-Feasible Latency Headroom 97
6.7 Phase 7A — Structured Transfer and Energy Evaluation 99
6.8 Phase 7A — Accuracy–Energy Trade-off 101
6.9 Motivation for Phase 7B 103
6.10 Phase 7B — Deployment-Oriented Structured PPO 103
6.11 Phase 7B — Memory and Computation Results 104
6.12 Phase 7B — Accuracy Trade-off 104
6.13 Direct Energy Confirmation and Measurement Failure 106
6.14 Three-Level Sustainability Assessment 107
6.15 Answer to RQ6 108
6.16 Chapter Summary 108
CHAPTER 7: DISCUSSION 110
7.1 General 110
7.2 What Adaptive Layer-Wise PPO Actually Contributes 110
7.3 The Role of Sensitivity Information 111
7.4 Search Quality versus Policy Retention 112
7.5 Reward Design and Over-Pruning 112
7.6 Validation-to-Test Transfer 113
7.7 Why LAMP Remains a Strong Benchmark, and Why Equal Fine-Tuning Changes the Rankings 114
7.7.1 LAMP 114
7.7.2 Equal fine-tuning 114
7.8 Compression versus Real Deployment Efficiency 115
7.9 Accuracy–Efficiency Trade-offs and the Three Levels of Efficiency 116
7.10 Interpreting the Sustainability Experiments 116
7.11 What the Negative Results Establish 118
7.12 Generalisability 118
7.13 Limitations 119
7.14 Overall Scientific Contribution 120
7.15 Chapter Summary 121
CHAPTER 8: CONCLUSION AND FUTURE WORK 122
8.1 General 122
8.2 Conclusions by Research Question 122

RQ1. Does PPO learn non-uniform layer-wise pruning allocations that outperform simple allocation strategies, and how does it compare with a strong deterministic method such as LAMP? 122
RQ2. Does layer-sensitivity information improve the pruning search process, and does that improvement translate into higher final predictive accuracy?.. 122
RQ3. Does PPO retain the strongest pruning policies encountered during search, and can archive-based policy selection improve retention? 123
RQ4. How does reward design influence pruning aggressiveness and the accuracy–compression trade-off? 123
RQ5. How does equal post-pruning fine-tuning affect comparisons between PPO and deterministic pruning methods? 123
RQ6. Do compression gains translate into real deployment benefits in memory, computation, latency and measured operational energy? 124
8.3 Main Contributions 124
8.4 Overall Thesis Conclusion 125
8.5 Limitations 126
8.6 Future Work 127
Short-term 127
Medium-term 127
Long-term 128
8.7 Final Remarks 128
REFERENCES 129
APPENDIX A: ARCHITECTURES, HYPERPARAMETERS AND REPRODUCIBILITY A-1
APPENDIX B: SUPPLEMENTARY CORE-PRUNING RESULTS B-1
APPENDIX C: ARCHIVE, REWARD AND RECOVERY DIAGNOSTICS C-1
APPENDIX D: HARDWARE AND STRUCTURED-PRUNING PROTOCOL D-1
APPENDIX E: ENERGY MEASUREMENT AND QUALIFICATION E-1

LIST OF FIGURES

Figure 3.1: Overall thesis framework 31

Figure 5.1: Test accuracy against total sparsity (Phase 3) 75

Figure 5.2: Final-policy pruning ratio per unit (Phase 3) 76

Figure 5.3: Unit sensitivity, pruning tolerance and PPO allocation (Phase 3) 78

Figure 5.4: Share of runs whose selected policy lies in the top-k of the VAL-RL (I) landscape (Phase 4, Stage C) 81

Figure 5.5: ResNet-8 reward alignment: test-accuracy drop below dense against selected-policy sparsity 84

Figure 5.6: Rank score of the sampled policy over 512 episodes, C1 (centred prior) vs C2 (no prior) (Phase 5, VAL-RL) 85

Figure 5.7: Median selected sparsity of C0, C1 and C2 against the maximum feasible sparsity on VAL-RL ($q \geq \tau$), by setting (Phase 5) 87

Figure 5.8: PPO − LAMP test accuracy before and after equal 5-epoch fine-tuning (Phase 6) 90

Figure 6.1: Measured latency reduction against (a) physical parameter reduction and (b) MAC reduction, for all 1,296 structured policies of the CIFAR-10 SimpleCNN and ResNet-8 tables (Paper 3) 96

Figure 6.2: ResNet-8 (CIFAR-10): table speed-up against retained validation accuracy (Paper 3) 98

Figure 6.3: Phase 7A: VAL-RL accuracy after recovery against measured package energy per 1,000 inferences 102

Figure 6.4: Phase 7B: VAL-SELECT accuracy after the equal 5-epoch recovery against MACs per image (left) and parameter memory (right) 105

LIST OF TABLES

Table 1.1: Research questions, objectives, contributions and evidence 9
Table 2.1: Comparison of representative pruning-allocation and deployment-aware pruning works 19
Table 3.1: Development of the framework across phases 30
Table 3.2: Pruning units of the core framework (unstructured) 33
Table 3.3: State features by phase 35
Table 3.4: Mature core framework (as frozen after Phase 5, evaluated in Phase 6) 42
Table 4.1: Datasets and split configurations 50
Table 4.2: Data-split usage by phase 51
Table 4.3: Network architectures and pruning units (core framework) 52
Table 4.4: Dense reference models (accuracy, %) 53
Table 4.5: Baseline methods and matching rules 56
Table 4.6: PPO configuration by phase 57
Table 4.7: Phase-by-phase experimental design 58
Table 4.8: Reward configurations 63
Table 4.9: Evaluation metrics 65
Table 4.10: Pre-registered decision criteria 68
Table 4.11: Hardware and software environments 69
Table 5.1: Dense reference models (accuracy %, seed-0 reference) 73
Table 5.2: Phase 3: P0 minus baseline, test accuracy (pp, mean of 20 paired runs) 74
Table 5.3: Retention: the best policy encountered, the terminal policy (S0) and the archive-selected policy (S2A) 80
Table 5.4: ResNet-8 reward alignment across Phases 4 and 5 (test accuracy %, mean of 20 runs; sparsity = mean selected masked sparsity). Phase-4 and Phase-5 rows share the test set and dense reference 83
Table 5.5: Phase 5 by setting (test accuracy %, mean of 20 runs; differences in pp) 84
Table 5.6: Validation-to-test feasibility of the selected Phase-5 policies ($q \geq 0.98$; runs out of 120) 86
Table 5.7: Phase 6: test accuracy before → after equal fine-tuning, and the PPO–LAMP and PPO–Global gaps (pp; mean of 20 paired runs; 95% t-CI) 88
Table 5.8: Sensitivity guidance across Phases 3–5: exploration endpoints versus final-model endpoints (settings supported / opposite, of 6) 91
Table 6.1: Exhaustive structured latency tables: cost metrics against measured latency gain 95
Table 6.2: ResNet-8 (CIFAR-10): calibrated accuracy-feasible latency headroom 98
Table 6.3: Phase 7A: accuracy, compression, latency and measured energy 100

Table 6.4: Phase 7B: accuracy, memory and computation after the equal 5-epoch recovery (SimpleCNN, CIFAR-10) 104

Table 6.5: Three-level deployment-efficiency assessment..... 107

Table 7.1: Negative and partial results and what each establishes 118

Table 8.1: Thesis contributions and their boundaries..... 125

Table A.1: Pruning units of the three networks: weights, share of prunable weights and MACs per image (CIFAR-10 output layer) A-1

Table B.1: Phase 3 mean test accuracy (%) of PPO (P0) and the five baselines built at each PPO run's zero count (20 runs) B-1

Table B.2: Normalised loss sensitivity S_l of the four units by setting and the Phase-5 centred prior coefficients..... B-1

Table C.1: Phase 6 mean recovery (test accuracy after minus before equal fine-tuning, pp; 20 runs) by setting and method C-1

Table D.1: H0 calibration: decision quantities of the three candidate measurement configurations..... D-1

Table D.2: Exhaustive latency tables: accepted sessions, repeated attempts and anchor drift (limit 10%)..... D-1

Table E.1: Phase 7A energy: session summaries (five sessions per architecture).....E-1

Table E.2: Dense-only stability pilots and qualification (energy per 1,000 inferences, J; six dense blocks each) E-1

LIST OF MAIN NOTATION AND ABBREVIATIONS

Main notation

N_L	number of pruning units; $N_L = 4$ for every architecture
l	index of a pruning unit, $l = 1, \ldots, N_L$
r_l	pruning ratio assigned to unit l, $r_l = r(a_l) \in \mathcal{R}$
r (bold) / r(a)	**r**: the policy vector $[r_1, \ldots, r_{NL}]$. r(a): the map from action index to ratio
S_l	normalised sensitivity of unit l, $S_l \in [0, 1]$
$\tilde{S}_l$ (centred sensitivity)	$S_l - \bar{S}$, where $\bar{S} = (1/N_L)\ \Sigma\ S_j$ (Eq. 3.11); the centred prior uses $c_l = 1.0 \cdot \tilde{S}_l$
s_l	state observed by the agent before deciding unit l
a_l	action at unit l, $a_l \in \{0, \ldots, 5\}$
π_θ	PPO policy with parameters θ, $\pi_\theta(a \mid s)$ = softmax of the shifted logits (Eq. 3.10)
A_b	dense-reference accuracy on the reward split (VAL-RL)
A_p	accuracy of the pruned network $M_{\mathbf{r}}$ on the same split as A_b
q	retained accuracy A_p / A_b
ρ	masked weight sparsity, % of zero weights over all Conv2d/Linear weights (Eq. 3.2)
R	episode reward. Variants: R_0 (Eq. 3.7), R1a/R1b/R2 (Phase-4 Stage A), R_c (Eq. 3.15), R_{7B} (Eq. 3.19); $\hat{R}_t$ is the PPO return target
τ	retained-accuracy threshold of the constrained reward, $\tau = 0.98$ (Phase 5 and the selection rule of Eq. 3.14)
latency, T(**r**)	measured batch-1 CPU latency per inference (ms); $T_{table}(\mathbf{r})$ is the frozen Paper-3 table value used by the 7B surrogate
energy, E(**r**)	directly measured CPU package energy per 1,000 inferences (J)
$\hat{E}(\mathbf{r})$	surrogate (predicted) energy per 1,000 inferences, used only inside the Phase-7B search; never reported as a measurement
$\mathcal{R}$	ratio set {0, 0.1, 0.2, 0.3, 0.4, 0.6}

M, $M_{\mathbf{r}}$	dense reference network; network pruned by policy **r**
$\bar{S}$	mean sensitivity over units
c_l	prior coefficient (P1: $0.5 \cdot S_l$; centred: $1.0 \cdot \tilde{S}_l$)
C_l, k_l	channels before structured pruning; channels kept
D	number of distinct actions in **a**
$\mathcal{A}$	archive of unique visited policies
$\mathcal{C}$	top-K archived candidates (Eq. 3.14), $K_c = 10$
ε	accuracy tie-breaker weight in R_c and R_{7B} (0.01)
κ	penalty slope for infeasible policies ($\kappa = 10$)
α	weight of the predicted-saving term in R_{7B} ($\alpha = 4$)
a, b	surrogate intercept and slope (-0.117; 13.762)
η	weight of the behavioural-cloning (elite) loss, Phase-4 S2 only
$\hat{A}_t$, V_φ, $\mathcal{H}$, c_v, c_e	PPO advantage, value function, entropy, value and entropy coefficients
MAC(**r**)	multiply–accumulate operations per image
ρ^*_s, ρ_{60}	R1b target sparsity; sparsity of the maximal policy [5,5,5,5] (R2)

General abbreviations

AC	Alternating current (mains power)
AI	Artificial intelligence
ARM	ARM processor architecture
AUC	Area under the curve (trajectory AUC of a search measure)
CI	Confidence interval
CNN	Convolutional neural network
CPU	Central processing unit
CSV	Comma-separated values
CV	Coefficient of variation

DRAM	Dynamic random-access memory
EDP	Energy–delay product
FLOP	Floating-point operation (FLOPs: floating-point operations)
GAE	Generalised advantage estimation
GPU	Graphics processing unit
HW	Hardware (table header, Chapter 2)
JSON	JavaScript Object Notation
LLM	Large language model
MAC	Multiply–accumulate operation (MACs: multiply–accumulate operations)
MKL	Math Kernel Library (Intel oneMKL)
MLP	Multilayer perceptron
NAS	Neural architecture search
NCHW	Tensor layout: batch (N), channels (C), height (H), width (W)
NLP	Natural language processing
OS	Operating system
PDH	Performance Data Helper (Windows performance-counter interface)
pp	Percentage point(s)
PP0	RAPL power plane 0 (processor cores)
PPO	Proximal Policy Optimisation
RAPL	Running Average Power Limit (Intel energy-counter interface)
ReLU	Rectified linear unit
RGB	Red–green–blue colour channels
RL	Reinforcement learning
RQ	Research question
SB3	Stable-Baselines3

SD	Standard deviation
SHA-256	Secure Hash Algorithm, 256-bit digest
SNR	Signal-to-noise ratio
TDP	Thermal design power
VAL-RL	Validation split used for sensitivity, reward and PPO training (Chapter 4)
VAL-SELECT	Validation split used only to select among candidates already produced (Chapter 4)

Method, model and dataset names

AMC	AutoML for Model Compression
BESA	Blockwise Parameter-Efficient Sparsity Allocation
CHEX	CHannel EXploration
CIFAR-10, CIFAR-100	Image-classification datasets of the Canadian Institute for Advanced Research [10 and 100 classes]
DDPG	Deep Deterministic Policy Gradient
DECORE	Deep Compression with Reinforcement Learning
ERK	Erdős–Rényi-kernel layer-wise sparsity allocation
GNN	Graph neural network
HALP	Hardware-Aware Latency Pruning
HAP	Hessian-Aware Pruning
HAQ	Hardware-Aware Automated Quantization
LAMP	Layer-Adaptive Magnitude-based Pruning (score)
LeNet-5, ResNet-8, SimpleCNN	Network architectures (Chapter 4)
LSTM	Long short-term memory
MNIST, PASCAL VOC, ImageNet	Datasets named in the literature review
OPT, LLaMA, Vicuna, Mistral, ViT,	Model families named in the literature review

MobileNet V1/V2	
OWL	Outlier Weighed Layerwise sparsity
PQ Index	Compressibility index named in the literature review
QLP	Deep Q-Learning for Pruning
SMCP	Soft Masking for Cost-Constrained Channel Pruning
SNIP	Single-shot Network Pruning

Experiment labels (thesis notation)

C10, C100	CIFAR-10, CIFAR-100 (in tables) (Chapter 5)
C0, C1, C2	Phase-5 conditions (R0 + centred prior; constrained reward + centred prior; constrained reward, no prior) (Chapter 4 §4.9.3)
F, K	Phase-4 final pipeline (R0, P2, S2A) and control (R0, P0, S0) (Chapter 4 §4.9.2)
H0, H1, H2-A	Paper-3 stages: latency calibration, structured-pruning validation, exhaustive latency tables (Chapter 4 §4.9.5)
O1–O6	Specific research objectives (Chapter 1 §1.6)
P0–P3 (Phase 3)	No prior; correct prior; shuffled prior; constant prior (Chapter 4 §4.9.1)
P0–P5 (Phase 4)	No prior; Phase-3 prior; centred prior; permuted centred; magnitude-matched conservative; sign-reversed (Chapter 4 §4.9.2)
R0, R1a, R1b, R2, R_c	Reward functions (Chapter 4 §4.12)
RQ1–RQ6	Research questions (Chapter 1 §1.5)
S0, S1, S2, S2A	Phase-4 search variants (terminal; archive; elite cloning; elite cloning + archive) (Chapter 4 §4.9.2)
TC1–TC6	Thesis contributions 1–6 (written "Contribution 1" … "Contribution 6" in prose) (Chapter 1 §1.7; Chapter 8 §8.3)
7A, 7B, 7B-EC	Phase 7A, Phase 7B, Phase-7B energy confirmation study (Chapter 4 §4.9)

CHAPTER 1
INTRODUCTION

1.1 Background

The cost of deployment. Deep neural networks reach high predictive accuracy on many tasks, but the models that do so are often large. Deploying them has a cost on four fronts:

(a) the parameter memory needed to store and load them;

(b) the arithmetic they perform for every input;

(c) the latency of each prediction;

(d) the energy drawn by the processor that runs them.

These costs recur at every inference, and inference energy has become a subject of study in its own right (Desislavov, Martínez-Plumed and Hernández-Orallo, 2023; Luccioni, Jernite and Strubell, 2024).

Pruning. Network compression reduces these costs while keeping accuracy, through pruning, quantisation, distillation and efficient architecture design (Menghani, 2023). *Pruning* removes parts of a trained network that contribute little to its predictions. It is one of the oldest compression techniques (LeCun, Denker and Solla, 1989; Han et al., 2015), and it remains among the most studied; a recent survey counts thousands of pruning papers in a few years (Cheng, Zhang and Shi, 2024).

What is removed:

(a) **Unstructured (masked) pruning** sets individual weights to zero. The tensors keep their shapes, and the zeros are recorded in a mask.

(b) **Structured pruning** removes whole channels, filters or neurons, so the network becomes physically smaller and its layers perform less arithmetic (He and Xiao, 2024).

Only the second shrinks the dense computation that general-purpose hardware actually executes (Hoefler et al., 2021).

How much each layer should lose. The simplest rule prunes every layer by the same fraction, but layers differ in their share of parameters and computation and in the accuracy

lost when they are pruned. *Layer-wise adaptive* pruning gives each layer its own ratio, chosen by:

(a) **Magnitude-based rules,** which remove the smallest weights within each layer or across the whole network.

(b) **Layer-adaptive scores,** such as LAMP, which rescale weight magnitudes so that a single global threshold produces a non-uniform allocation (Lee et al., 2021).

(c) **Sensitivity-aware methods,** which measure how strongly each layer's pruning affects the loss.

(d) **Learned allocators,** which search for the ratios directly. Reinforcement learning (RL) agents that choose a ratio for each layer, rewarded by the accuracy and compression of the resulting network, are a well-known example (He et al., 2018).

This thesis uses Proximal Policy Optimisation (PPO) (Schulman et al., 2017) as such an allocator and studies what it contributes, what it needs to work, and whether its compression becomes deployment benefit.

Why a learned allocator, and what is hard about it. A learned allocator can optimise objectives that are not differentiable or per-weight scores, such as validation accuracy, a hard accuracy constraint or a measured hardware cost, and can be re-targeted without redesign. This flexibility has costs:

(a) **Evaluation cost.** Every candidate must be evaluated, so the search costs far more than a one-pass rule.

(b) **Sensitivity to design details.** Policy-gradient methods are sensitive to implementation and training choices (Engstrom et al., 2020; Andrychowicz et al., 2021).

(c) **Sensitivity to the reward.** The outcome depends on how the reward is written.

Whether the flexibility is worth the cost has to be measured against strong deterministic alternatives.

1.2 Motivation

Five linked problems lie between "a network can be pruned" and "a pruned network is a better deployment choice".

Problem A: uniform pruning ignores heterogeneous layer sensitivity. A single ratio treats a small early convolution, on which every later layer depends, like a large redundant fully connected layer. Strong deterministic layer-adaptive methods already exist, so a learned allocator must be compared with them under controlled conditions, not only with uniform pruning.

Problem B: a search may find strong policies without retaining them. An RL agent evaluates many allocations but returns its final policy; a good search can end on a mediocre allocation, and reporting only that policy cannot distinguish weak search from weak retention.

Problem C: a reward can encourage excessive pruning. RL pruning rewards typically trade accuracy against sparsity at a fixed rate, with no point at which accuracy loss becomes unacceptable; a better optimiser may then find exactly the over-pruned models the reward implicitly prefers.

Problem D: comparisons before equal recovery may overstate differences. Pruned networks are usually fine-tuned before use, and fine-tuning can repair much of the damage (Renda, Frankle and Carbin, 2020; Le and Hua, 2021). A method that looks better immediately after pruning may simply damage the network less before repair; its advantage after equal recovery is a separate question.

Problem E: sparsity does not necessarily mean speed or energy savings. A masked network still runs dense computations on a CPU. Even under structured pruning, the parameters removed and the computation saved need not coincide, and cost indicators can disagree (Dehghani et al., 2022). Speed and energy must be measured, with a reliable instrument.

A study that solves only the first can still return poor allocations, optimise the wrong trade-off, overstate its advantage or claim efficiency it does not deliver. The design controls each problem:

(a) **A:** every baseline is built at exactly the sparsity the agent chose;

(b) **B:** every policy the agent visits is recorded and ranked against an exhaustive landscape;

(c) **C:** rewards are compared on the same landscapes, and the accuracy boundary is made explicit;

(d) **D:** every method receives the same fixed-mask recovery with a shared data stream;

(e) **E:** pruning is realised physically and measured on hardware, with the measuring instrument qualified before any comparison.

1.3 Problem Statement

Existing pruning methods often optimise compression and accuracy through fixed rules or single saliency criteria. Reinforcement learning offers a flexible mechanism for learning non-uniform layer-wise allocations, but its effectiveness depends on four things beyond the learning algorithm itself:

(a) the quality of the guidance given to the search;

(b) whether good policies found during the search are retained;

(c) whether the reward encodes the intended accuracy–compression trade-off;

(d) how the pruned models are recovered before comparison.

Nor does the fraction of zero weights establish deployment efficiency. This thesis therefore investigates whether a PPO agent can learn adaptive layer-wise pruning policies, what sensitivity, retention, reward design and equal recovery contribute, and whether the compression becomes memory, computation, latency and measured energy benefits.

Research approach. The investigation was a sequence of phases, each with conditions, seeds, metrics and decision criteria committed before its confirmatory data were collected (Chapter 4). Four design choices make the results auditable:

(a) **Exhaustive ground truth.** Four pruning units with six ratios give 1,296 policies per setting, all evaluated, so the rank of any visited or returned policy is known exactly.

(b) **Strict data governance.** Training images update weights only; one validation split drives the reward, a second, disjoint split only selects among candidates; the test set is read only for frozen models.

(c) **Matched comparisons.** Every baseline is built at exactly the zero count of the PPO model it is compared with, and every compared model receives an identical recovery protocol.

(d) **A separate extension.** Pruning is realised physically and latency and energy are measured directly, under pre-registered stability checks for the instrument.

1.4 Research Gap

Chapter 2 identifies six gaps. RL has been applied to pruning (He et al., 2018); the gaps concern how such methods are evaluated and what their results establish.

(i) **Adaptive allocation against strong deterministic allocation.** Layer-specific allocation matters, but learned allocators are rarely compared with uniform, random, magnitude-based and layer-adaptive baselines built at exactly the same sparsity. → RQ1.

(ii) **Exploration effects versus final-model effects of sensitivity.** Sensitivity's effect on search efficiency is seldom separated from its effect on final accuracy, or compared with uninformative controls. → RQ2.

(iii) **Retention of strong policies found during search.** RL pruning is usually judged by its final policy. Whether stronger policies were encountered and lost has not been examined systematically for pruning. → RQ3.

(iv) **Incentives for excessive sparsity in pruning rewards.** Hand-weighted accuracy–sparsity rewards can be exploited by capable optimisers (Pan, Bhatia and Steinhardt, 2022). Whether better pruning search exposes this, and whether an explicit accuracy constraint controls it, has not been tested directly. → RQ4.

(v) **Comparisons without equal recovery.** Pruning methods are often compared under inconsistent experimental conditions (Blalock et al., 2020), and the retraining schedule alone can change which pruned network performs better (Le and Hua, 2021). Whether differences between a learned allocator and deterministic methods, measured before recovery or under different recovery budgets, survive identical post-pruning fine-tuning therefore remains to be tested. → RQ5.

(vi) **Compression versus deployment efficiency.** Several questions remain empirical:

(a) how well parameter and MAC reductions predict measured CPU latency across a whole pruning space;

(b) how much latency headroom remains under an accuracy constraint;

(c) whether a structured realisation of a learned allocation reduces directly measured energy without material accuracy loss.

Energy measurement on general-purpose CPUs adds its own reliability problem. → RQ6.

1.5 Research Questions

RQ1–RQ5 concern the **core framework**, which uses masked (unstructured) pruning. **RQ6** concerns the **deployment extension**, added after the core framework had been completed and evaluated.

RQ1. Does PPO learn non-uniform layer-wise pruning allocations that outperform simple allocation strategies, and how does it compare with a strong deterministic method such as LAMP?

RQ2. Does layer-sensitivity information improve the pruning search process, and does that improvement translate into higher final predictive accuracy?

RQ3. Does PPO retain the strongest pruning policies encountered during search, and can archive-based policy selection improve retention?

RQ4. How does reward design influence pruning aggressiveness and the accuracy–compression trade-off?

RQ5. How does equal post-pruning fine-tuning affect comparisons between PPO and deterministic pruning methods?

RQ6. Do compression gains translate into real deployment benefits in memory, computation, latency and measured operational energy?

The questions were not all fixed at the outset; each answer exposed the next problem (Chapter 3).

1.6 Research Objectives

General objective. To develop and systematically evaluate a sensitivity-aware reinforcement-learning framework for adaptive layer-wise neural-network pruning, including its search behaviour, recovery characteristics and deployment implications.

Specific objectives:

(a) **O1.** Develop a PPO-based framework for non-uniform layer-wise pruning, and evaluate it against uniform, random, global-magnitude, ERK and LAMP allocations at identical sparsity.

(b) **O2.** Evaluate whether sensitivity guidance improves the pruning search and the final model quality, using shuffled, constant, magnitude-matched and sign-reversed controls.

(c) **O3.** Investigate policy-retention behaviour, and evaluate archive-based selection of the policies visited during search.

(d) **O4.** Analyse reward misalignment, and evaluate an accuracy-constrained reward with validation-based selection.

(e) **O5.** Assess how an equal, fixed-mask post-pruning recovery protocol affects comparative conclusions between PPO and deterministic methods.

(f) **O6.** Evaluate whether pruning-related compression translates into memory, computation, latency and directly measured operational-energy improvements when realised as physically structured networks.

Each objective was addressed by completed, pre-registered experiments (Chapter 4); Table 1.1 maps objectives to research questions, contributions and evidence.

1.7 Research Contributions

The thesis makes six contributions. None of them rests on a claim that PPO is the best pruning method.

Contribution 1. An adaptive layer-wise pruning framework, evaluated against strong baselines. A PPO agent assigns one of six ratios to each of four units, evaluated against uniform, random, global-magnitude, ERK and LAMP allocations at exactly its

own sparsity, on two datasets and three architectures. Its non-uniform allocations strongly outperform uniform and random allocation; LAMP remains stronger before recovery.

Contribution 2. A separation of the exploration effect and the final-model effect of sensitivity guidance. Across three phases and against uninformative controls, sensitivity's measurable contribution is to the *search* (better-ranked policies earlier, fewer destructive ones); its contribution to final accuracy was not detectable.

Contribution 3. Identification of a policy-retention problem in RL pruning, and evaluation of archive-based selection. Using exhaustive ground-truth landscapes, the thesis shows that the agent visits near-optimal policies but rarely ends on them. An archive of visited policies, with selection on held-out validation data, restores retention in terms of validation rank. Its effect on test accuracy is mixed.

Contribution 4. Identification of reward misalignment, and evaluation of an accuracy-constrained reward. Improving search and retention exposed that a fixed-rate accuracy–sparsity reward drives fragile networks to over-prune. A constraint-first reward, with selection on held-out data, substantially reduced the most destructive over-pruning. Validation feasibility, however, did not reliably transfer to the test set.

Contribution 5. A controlled equal-recovery analysis. Every compared model received an identical fixed-mask fine-tuning protocol, with shared seeds and data order, at identical sparsity; the differences between methods became small, which grounds the thesis's main methodological recommendation.

Contribution 6. A deployment analysis that keeps four quantities apart: parameter compression, MAC reduction, measured latency and directly measured operational energy. It comprises:

(a) an exhaustive latency analysis of every structured policy of two networks and two output sizes;

(b) direct CPU package-energy measurement of a structured realisation of the learned allocation;

(c) a deployment-oriented structured PPO that learns physical channel allocations directly;

(d) pre-registered qualification of the energy instrument, reported including its failure.

Novelty. The novelty lies not in a new pruning criterion or a claim that PPO outperforms existing methods, but in **treating adaptive pruning as a sequence of connected optimisation and validation problems, rather than evaluating only the final compressed model**: allocation, exploration, policy retention, reward alignment, recovery, deployment translation and hardware validation, each examined with its own experiment and decision criteria fixed before the confirmatory data were collected. It shows how a weakness at one stage can hide or reverse improvements at another.

Table 1.1: Research questions, objectives, contributions and evidence.

RQ	Objective	Contribution	Evidence
RQ1	O1	TC1	Chapter 5 (Phase 3; Phases 4–5 LAMP comparisons)
RQ2	O2	TC2	Chapter 5 (Phases 3, 4, 5)
RQ3	O3	TC3	Chapter 5 (Phase 4)
RQ4	O4	TC4	Chapter 5 (Phases 4, 5)
RQ5	O5	TC5	Chapter 5 (Phase 6)
RQ6	O6	TC6	Chapter 6 (Paper 3, Phases 7A, 7B, energy confirmation)

TC1–TC6 denote Contributions 1–6. The labels C0, C1 and C2 used in later chapters denote Phase-5 experimental conditions, not contributions.

Preview of the main findings (Chapters 5–7):

(a) PPO substantially outperformed uniform (by up to about 31.4 percentage points of test accuracy) and random pruning, but LAMP remained stronger before equal recovery.

(b) Sensitivity guidance improved exploration but not final accuracy; archive selection restored validation-rank retention, with a mixed test effect.

(c) The accuracy-constrained reward substantially reduced destructive over-pruning, but its validation-certified feasibility held on the test set in only about two thirds of runs.

(d) Equal recovery narrowed the PPO–LAMP differences to within a pre-registered confidence-interval margin in every setting, although LAMP remained slightly ahead in three.

(e) Parameter reduction was a weak predictor of CPU latency compared with MAC reduction. Structured pruning reduced directly measured CPU package energy

(Phase 7A) at a substantial accuracy cost; on one network, uniform allocation was the better choice.

(f) The redesigned agent (Phase 7B) reduced parameter memory by 63.4% and MACs by 36.0% at a validation-accuracy cost of 1.12 percentage points; it missed the pre-registered accuracy criterion, and its direct energy could not be confirmed.

1.8 Scope and Boundaries

Included in the scope:

(a) **Datasets:** CIFAR-10 and CIFAR-100 image classification.

(b) **Networks:** three small convolutional networks (SimpleCNN, a CIFAR adaptation of LeNet-5, and ResNet-8), each with four pruning units and six pruning ratios.

(c) **Method:** PPO; masked (unstructured) pruning in the core framework, with all 1,296 policies per setting evaluated as ground truth; physically structured pruning on a subset of settings in the deployment extension.

(d) **Hardware evaluation:** parameter memory, MACs, batch-1 CPU latency, and CPU package energy measured through the processor's energy counters, on one laptop CPU.

Not established by this thesis:

(a) behaviour at ImageNet scale, or for larger CNNs, transformers or large language models;

(b) performance on GPUs, mobile processors, edge accelerators or cloud inference;

(c) whole-system or wall-plug energy, memory energy, or training energy;

(d) any carbon-footprint, lifecycle or environmental-impact claim.

The energy evaluation concerns operational inference energy; it is not an environmental assessment.

Boundaries that the reader should keep in mind throughout:

(a) PPO is **not** presented as universally better than deterministic pruning. LAMP remains a strong baseline, and was more accurate than PPO before recovery in nearly every setting.

(b) Sensitivity guidance is, on the evidence, primarily an **exploration aid**.

(c) Archive selection is a **validation-retention mechanism**, not a generalisation mechanism.

(d) Validation-based accuracy constraints **do not guarantee** feasibility on the test set.

(e) The ±0.5 percentage-point rule used after equal recovery is a pre-registered confidence-interval decision rule, and **not** a formal statistical equivalence test.

(f) **Phase 7A** directly measured energy reductions, but every pruned model lost more accuracy than the criterion allowed.

(g) **Phase 7B's** direct energy remains **unresolved**, because the measurement setup failed its stability check. No latency or energy reduction is claimed for it.

1.9 Thesis Organisation

Chapter 2 reviews the literature and ends with the research gaps. Chapter 3 presents the framework in the order in which it developed, with the deployment-oriented structured extension separately. Chapter 4 gives the experimental setup. Chapter 5 reports the core-framework results (Phases 3–6, RQ1–RQ5) and Chapter 6 the deployment analysis (RQ6). Chapter 7 interprets the findings and states the limitations; Chapter 8 concludes and outlines future work.

1.10 Chapter Summary

This chapter argued that adaptive layer-wise pruning is a chain of connected problems rather than a single choice of ratio, and set out six research questions: **RQ1–RQ5** for a PPO-based, sensitivity-aware core framework using masked pruning, and **RQ6** for a structured deployment extension with direct hardware measurement. The thesis does not claim superiority over existing methods, and its scope is limited to small CNNs, CIFAR data and one CPU platform.

CHAPTER 2
LITERATURE REVIEW

This chapter reviews compression and pruning granularity (Sections 2.1–2.2), the allocation problem (2.3–2.5), reinforcement learning (RL), PPO, sensitivity and policy selection (2.6–2.9), recovery (2.10), and hardware-aware compression, cost metrics and energy measurement (2.11–2.13); Section 2.14 states the research gap.

2.1 Neural Network Compression

The accuracy gains of deep neural networks have come with growing parameter counts, arithmetic cost and memory traffic, making deployment expensive on servers and difficult on resource-limited devices (Hoefler et al., 2021; Menghani, 2023). Compression reduces these costs while keeping predictive quality close to that of the original model, through pruning, quantisation, knowledge distillation or efficient architecture design (Menghani, 2023; Cheng, Zhang and Shi, 2024; He and Xiao, 2024).

Pruning is among the oldest of these ideas. Early work removed weights by a second-order estimate of the loss increase (LeCun, Denker and Solla, 1989); later, removing small-magnitude weights and retraining reduced the parameter count of large convolutional networks many times over with little accuracy loss (Han et al., 2015); the lottery ticket hypothesis argued that dense networks contain sparse subnetworks that train to comparable accuracy (Frankle and Carbin, 2019). One survey counts more than three thousand pruning papers between 2020 and 2024 (Cheng, Zhang and Shi, 2024). A systematic study found that quantisation usually gives a better accuracy–compression trade-off than pruning, except at very high compression ratios (Kuzmin et al., 2023); this thesis studies pruning because its questions concern *where* capacity is removed. Two observations shape the thesis.

(i) **"Efficiency" is not a single quantity.** A pruned model may need less storage, fewer arithmetic operations, less time per inference or less energy, and these do not necessarily move together (Hoefler et al., 2021; Dehghani et al., 2022). Sections 2.11–2.13 return to this point.

(ii) **Reported compression results are hard to compare.** Studies differ in architectures, datasets, training budgets and fine-tuning protocols, and these

differences can be larger than the differences between pruning methods (Blalock et al., 2020; Liu et al., 2023). This motivates the controlled comparisons in this thesis.

2.2 Unstructured and Structured Pruning

Pruning methods are classified by the *granularity* of what they remove: *unstructured* (individual weights; same tensor shapes with a sparse mask; smaller compressed storage, speed-up only with sparse kernels or hardware support), *semi-structured* (fixed patterns such as N:M within small blocks; speed-up on hardware that supports the pattern) and *structured* (whole channels, filters or neurons; physically smaller dense tensors with fewer MACs, usable by standard libraries).

Unstructured pruning can reach high sparsity at a given accuracy because the choice of weights is unconstrained (Han et al., 2015; Hoefler et al., 2021), but zeros are stored and computed unless sparse formats and kernels are used, and on general-purpose hardware these are effective only at high sparsity and favourable patterns, so weight-level sparsity often brings little or no latency reduction (Hoefler et al., 2021; He and Xiao, 2024).

Structured pruning removes filters, channels or neurons (Wen et al., 2016; He, Zhang and Sun, 2017; Li et al., 2017), giving a smaller dense network that directly reduces MACs and memory traffic, the route to "realistic acceleration" (He and Xiao, 2024). The price is a coarser search space: accuracy typically degrades faster than with unstructured pruning at the same parameter reduction.

Recent work on structured pruning has concentrated on two problems.

(a) **Dependency handling.** In residual and attention architectures, removing a channel in one layer forces matching removals elsewhere. DepGraph models these dependencies as a graph so that coupled parameters are pruned together across arbitrary architectures (Fang et al., 2023). Isomorphic pruning compares the importance of sub-structures only within groups of the same computational topology, because heterogeneous structures have incomparable parameter scales (Fang et al., 2024).

(b) **Pruning during training.** CHEX repeatedly prunes and regrows channels during training instead of pruning a fully trained model once (Hou et al., 2022). OTOv2 trains and compresses a model in a single pass without separate fine-tuning (Chen et

al., 2023). Submodular structured pruning offers guarantees in a limited-data setting (El Halabi, Srinivas and Lacoste-Julien, 2022).

Random structure as a baseline. A channel-pruning benchmark found that random search over channel configurations works well compared with other channel-pruning methods, with no clear winner among channel-importance criteria (Li et al., 2022). *How much* to prune in each layer, together with the training protocol, can matter as much as *which* channels are removed.

Relevance to this thesis. The core framework (Chapters 3–5) uses magnitude-based *unstructured* masks and learns the per-layer ratio, which made exhaustive evaluation possible; the deployment extension (Chapter 6) realises the allocations as *physically structured* networks so that memory, MACs, latency and energy can be measured.

2.3 Layer-Wise Adaptive Pruning

A pruning method must also decide *how many* parameters to remove from each layer: the layer-wise sparsity allocation. Uniform pruning applies one ratio everywhere, but small early layers are often fragile while large late layers are redundant, so it over-prunes some layers and under-prunes others; the allocation itself is a first-order factor in pruned-model quality (Lee et al., 2021; Liu et al., 2022; Gadhikar, Mukherjee and Burkholz, 2023).

Approaches to the allocation problem fall into four broad classes.

(i) **Rule-based allocations** fix the ratios from layer properties. An example is the Erdős–Rényi-Kernel (ERK) allocation, which scales density with the layer's fan-in and fan-out and was popularised in sparse training (Evci et al., 2020).

(ii) **Global ranking** applies one importance threshold across all layers, so that allocation emerges from where low-importance weights happen to lie. Global magnitude pruning is the simplest such method.

(iii) **Score-normalised global ranking** rescales importance scores so that a single global threshold produces a better allocation. LAMP is the best-known example (Section 2.5) (Lee et al., 2021).

(iv) **Search- or learning-based allocation** treats the per-layer ratios as variables to optimise, using RL (He et al., 2018; Camci et al., 2022), learned global filter

ranking (Chin et al., 2020), fast sub-network evaluation (Li et al., 2020) or bi-level optimisation (Zhang et al., 2022).

Recent results have sharpened the importance of allocation. In sparse training, randomly pruned networks can be competitive provided that the per-layer ratios are sensible; the layer-wise ratio, rather than the precise mask, was a key factor (Liu et al., 2022). A theoretical analysis complements this, explaining when random masks with appropriate layer-wise densities can approximate target networks (Gadhikar, Mukherjee and Burkholz, 2023). The same question has reappeared in large language models. One-shot LLM pruning methods traditionally prune every layer to the same sparsity, but OWL allocates higher sparsity to layers with fewer outlier activations and improves markedly at high sparsity (Yin et al., 2024). BESA learns block-wise sparsity allocations under a global budget (Xu et al., 2024). Other work proposes a sparsity-based compressibility index (PQ Index) to guide adaptive pruning (Diao et al., 2023).

What remains less studied is how a *learned* allocation compares with cheap deterministic allocations at the exact same sparsity and under identical recovery (Research Question 1).

2.4 Magnitude-Based and Deterministic Pruning

Magnitude pruning removes the weights of smallest absolute value. Despite its simplicity it remains one of the strongest and most widely used baselines (Han et al., 2015; Blalock et al., 2020). It requires no data, no gradients and no search, and it combines easily with any allocation rule.

Variants differ in the *scope* over which magnitudes are compared:

(a) **Layer-wise magnitude pruning** removes a fixed fraction within each layer, so it needs an external allocation.

(b) **Global magnitude pruning** compares magnitudes across all layers. Because weight scales differ between layers, it can prune some layers almost completely, which is especially harmful in small layers.

Systematic evaluations repeatedly find that carefully tuned simple methods are hard to beat. ShrinkBench documented that many papers compare against weak or inconsistently

configured baselines, and that simple magnitude pruning remains competitive with more complex methods when evaluated consistently (Blalock et al., 2020). Re-examinations of structured pruning found that training a pruned architecture from scratch can match fine-tuning an inherited one, which questions how much the specific inherited weights matter (Liu et al., 2019). Benchmarks of sparse networks on harder tasks have also shown that apparent gains can vanish under more demanding evaluation (Liu et al., 2023).

Deterministic methods (magnitude pruning, global ranking, ERK, LAMP) are cheap, reproducible and free of search hyperparameters, so a learned method must justify its cost against them, at the same sparsity rather than against numbers reported elsewhere.

2.5 LAMP and Related Layer-Adaptive Methods

Layer-Adaptive Magnitude-based Pruning (LAMP) rescales each weight's magnitude by the magnitudes of the weights in the same layer that would survive if it were pruned. This yields a score that reflects the model-level $\ell 2$ distortion caused by pruning (Lee et al., 2021). A single global threshold on the LAMP score then produces a layer-wise allocation automatically, without hyperparameters and with negligible computation. In its original evaluation LAMP consistently outperformed popular layer-wise sparsity schemes on image classification, and remained strong under weight rewinding, where the strongest other baseline fell behind even global magnitude pruning (Lee et al., 2021). As a near-free *deterministic* answer to the problem a learned allocator addresses, LAMP is the natural benchmark for RL-based allocation, which requires thousands of pruned-model evaluations.

Related layer-adaptive methods include:

(a) **Learned global ranking**, which learns a single ranking of filters across layers, from which architectures with different accuracy–latency trade-offs follow by pruning the lowest-ranked filters (Chin et al., 2020);
(b) **Outlier-based allocations for LLMs** (Yin et al., 2024);
(c) **Block-wise learned allocations** (Xu et al., 2024).

Like most allocators, these were compared under each paper's own fine-tuning protocol; Chapter 5 compares PPO and LAMP before and after identical recovery.

2.6 Reinforcement Learning for Neural Network Compression

Compression can be framed as sequential decisions: an agent visits the layers one by one, chooses a ratio for each, and is rewarded by the accuracy and cost of the compressed model. This connects compression to neural architecture search, where an RL controller generated architectures rewarded by their validation accuracy (Zoph and Le, 2017).

AMC made the formulation concrete for pruning (He et al., 2018). A DDPG agent receives an embedding of each layer and outputs a continuous sparsity ratio. The reward combines accuracy with resource constraints. AMC reported better accuracy than hand-crafted compression at a given FLOPs reduction, and measured speed-ups on a GPU and on mobile hardware (He et al., 2018). Platform-aware methods such as NetAdapt and HAQ extended the idea by placing measured latency or hardware feedback inside the optimisation loop (Yang et al., 2018; Wang et al., 2019).

Recent RL-based pruning research has explored several directions (Table 2.1):

(a) **Q-learning allocation.** QLP uses deep Q-learning to choose layer-wise sparsity ratios. It can revisit layers and prune them gradually instead of making a single first-to-last pass (Camci et al., 2022).

(b) **Lookahead search.** RL guided by a lookahead search has been used to select the filters to remove in channel pruning (Wang and Li, 2022).

(c) **Per-channel agents.** DECORE assigns a lightweight agent to each channel and trains the agents with a policy gradient method, so that channel retention is learned jointly with the network (Alwani, Wang and Madhavan, 2022).

(d) **Topology embeddings.** GNN-RL embeds the network's computational graph with a graph neural network and uses RL to find a compression policy; it reports results on ResNet, VGG, MobileNet and ShuffleNet models (Yu, Mazaheri and Jannesari, 2022).

(e) **Joint training and pruning.** An RL agent sets layer pruning ratios while the network is being trained, rather than after a full pre-training run (Ganjdanesh, Gao and Huang, 2024).

These methods show that RL can find competitive allocations and handle non-differentiable objectives, but they are usually evaluated on the final policy after method-

specific fine-tuning, with hand-built rewards and against published results under different recovery budgets (questions taken up in Section 2.14).

Table 2.1: Comparison of representative pruning-allocation and deployment-aware pruning works. "Struct." = structured pruning; "Sens." = an explicit sensitivity or saliency signal drives allocation; "HW metric" = a measured or table-based hardware metric (latency, throughput) enters the method or its evaluation; "Energy" = inference energy measured on hardware. Datasets and models are as stated in each paper's abstract. Limitations are stated relative to the questions of this thesis.

Work	Year	Pruning type	Selection / optimisation	Layer-adaptive	Sens.	RL	Struct.	HW metric	Energy	Datasets / models	Main limitation (for this thesis' questions)
Energy-aware pruning (Yang, Chen and Sze, 2017)	2017	Weight	Energy-model-guided greedy	Yes	No	No	No	Energy model	Estimated (model calibrated on hardware)	ImageNet CNNs (AlexNet, GoogLeNet)	Energy is modelled, not measured per inference
AMC (He et al., 2018)	2018	Weight / channel	DDPG agent per layer	Yes	No	Yes	Both	Latency (GPU, mobile)	No	ImageNet VGG-16, MobileNet	Final-policy evaluation; reward coefficients fixed by hand
NetAdapt (Yang et al., 2018)	2018	Filter	Iterative, measured-latency-driven	Yes	No	No	Yes	Measured latency	No	ImageNet MobileNet V1/V2	Platform-specific measurement loop at each step
LAMP (Lee et al., 2021)	2021	Weight	Rescaled-magnitude global threshold	Yes	Distortion score	No	No	No	No	Image-classification CNNs	Unstructured: no direct latency benefit
QLP (Camci et al., 2022)	2022	Weight	Deep Q-learning, multi-pass	Yes	No	Yes	No	No	No	CIFAR-10 ResNet-32/56; ImageNet ResNet-50, MobileNet-v1	Unstructured masks; deployment effect not measured
Lookahead RL (Wang and Li, 2022)	2022	Channel	RL with lookahead search	Yes	No	Yes	Yes	No	No	MNIST, CIFAR-10, ImageNet	Final-policy evaluation
DECORE (Alwani, Wang and Madhavan, 2022)	2022	Channel	Per-channel policy-gradient agents	Yes	No	Yes	Yes	FLOPs / memory constraints	No	CIFAR-10 ResNet-110, VGG; ImageNet ResNet-50	FLOPs used as cost, not measured latency
GNN-RL (Yu, Mazaheri and Jannesari, 2022)	2022	Channel	GNN embedding + RL	Yes	Topology	Yes	Yes	FLOPs-constrained	No	ResNet, VGG-16, MobileNet-v1/v2, ShuffleNet	Cost measured as FLOPs

Work	Year	Pruning type	Selection / optimisation	Layer-adaptive	Sens.	RL	Struct.	HW metric	Energy	Datasets / models	Main limitation (for this thesis' questions)
HAP (S. Yu et al., 2022)	2022	Structured	Second-order (Hessian) sensitivity	Yes	Yes	No	Yes	No	No	CIFAR-10 PreResNet29; ImageNet ResNet-50	Sensitivity used for selection; no search or retention question
HALP (Shen et al., 2022)	2022	Channel	Latency-constrained knapsack	Yes	Saliency	No	Yes	Latency lookup table	No	ImageNet ResNet-50/101; VOC detection	Needs per-device latency tables; energy not measured
SMCP (Humble et al., 2022)	2022	Channel	Soft masking under a cost constraint	Yes	Saliency	No	Yes	Cost (latency) constraint	No	ImageNet, PASCAL VOC	Energy not measured
CHEX (Hou et al., 2022)	2022	Channel	Prune-and-regrow during training	Yes	No	No	Yes	FLOPs	No	ImageNet ResNet-50; detection, segmentation	FLOPs as cost
DepGraph (Fang et al., 2023)	2023	Structured (any)	Dependency graph + norm criterion	Yes	No	No	Yes	No	No	CNNs, ViT, GNN, LSTM	Grouping, not allocation search
OTOv2 (Chen et al., 2023)	2023	Structured	Train-once optimiser	Yes	No	No	Yes	No	No	VGG, ResNet, ConvNeXt, DenseNet; CIFAR-10/100, ImageNet	Allocation implicit in training
OWL (Yin et al., 2024)	2024	Weight	Outlier-ratio layer allocation	Yes	Outlier statistic	No	No	Inference speed (sparse engine)	No	LLaMA, Vicuna, OPT, Mistral	LLM-specific signal
RL agent + joint training (Ganjdanesh, Gao and Huang, 2024)	2024	Channel	RL agent sets layer ratios during training; accuracy reward	Yes	No	Yes	Yes	No	No	CIFAR-10, ImageNet; ResNets, MobileNets	Final-policy evaluation
This thesis	—	Weight (core); channel (extension)	PPO + centred sensitivity prior + archive + constrained reward	Yes	Yes	Yes	Extension	Measured CPU latency	RAPL (7A); unresolved (7B)	CIFAR-10/100; SimpleCNN, LeNet-5, ResNet-8	Small models; one CPU platform; LAMP remains stronger before recovery

2.7 PPO-Based Optimisation

Proximal Policy Optimisation (PPO) is an on-policy policy-gradient algorithm. It stabilises learning by limiting how far each update can move the policy, using a clipped surrogate objective (Schulman et al., 2017). It is commonly combined with generalised advantage estimation (Schulman et al., 2016). PPO is popular because it is comparatively insensitive to hyperparameters, handles discrete and continuous actions, and has mature implementations such as Stable-Baselines3 (Raffin et al., 2021), which this thesis uses. Large empirical studies have qualified this picture:

(a) **Implementation details.** Code-level choices that are not part of the algorithm's description (for example value-function clipping and reward normalisation) can account for a large share of PPO's reported advantage over related methods (Engstrom et al., 2020).

(b) **Design choices.** A study of more than fifty design choices in on-policy actor-critic methods found that many settings matter considerably and offered practical recommendations (Andrychowicz et al., 2021).

(c) **Multi-agent settings.** PPO proved surprisingly strong in cooperative multi-agent games, where it had been assumed less sample-efficient than off-policy methods (C. Yu et al., 2022).

PPO is thus a reasonable optimiser whose behaviour must still be examined empirically. Pruning allocation differs from typical RL benchmarks: episodes are a handful of decisions, one per unit; the reward needs a noisy held-out evaluation of a pruned model; and here the space is small enough to enumerate (1,296 policies), so search quality (how highly ranked are the policies PPO visits?) can be measured separately from final-policy quality (how highly ranked is the policy it ends on?).

2.8 Sensitivity-Aware Pruning

Sensitivity, how much a model's loss or accuracy changes when a parameter, structure or layer is pruned, is one of the oldest ideas in pruning:

(a) **Second-order estimates.** Optimal Brain Damage estimated weight saliency from second-order derivatives of the loss (LeCun, Denker and Solla, 1989). Modern

second-order structured methods capture correlations across structures with Hessian-vector products (Nonnenmacher et al., 2022). Hessian-aware pruning uses second-order sensitivity to decide which structures to prune and which to replace (S. Yu et al., 2022). Global structural pruning of vision transformers uses a Hessian-aware saliency to redistribute parameters across blocks (Yang et al., 2023).

(b) **First-order (Taylor) estimates.** These approximate the loss change caused by removing a filter or neuron from gradients (Molchanov et al., 2017, 2019).

(c) **Connection sensitivity at initialisation.** SNIP scores connections by their effect on the loss before training (Lee, Ajanthan and Torr, 2019).

(d) **Post-training compression for large models.** Optimal Brain Compression and SparseGPT build on second-order reconstruction for accurate one-shot compression (Frantar and Alistarh, 2022, 2023). Wanda shows that a simple weight-times-activation score is a strong alternative (Sun et al., 2024).

In most of this work sensitivity is *the pruning criterion*. Used instead as *guidance to a search* (a state feature or an action prior), it can make the search more efficient or lead to a better final model, and these need not coincide when selection, reward or recovery is the bottleneck. Studies usually report only final accuracy; Research Question 2 measures the two effects separately, against shuffled and constant controls.

2.9 Search and Policy Selection in Pruning

Any search-based pruning method must decide *which* of the policies encountered during search is returned. The policy an RL agent ends on, the terminal or greedy policy, is not necessarily the best one it saw. The issue is well known in neighbouring fields:

(a) **Neural architecture search.** An evaluation of the NAS search phase found that state-of-the-art NAS algorithms performed, on average, similarly to a random selection policy, and that weight sharing degraded the ranking of candidates. This reveals a gap between the search process and the final selection (Yu et al., 2020).

(b) **Reinforcement learning.** Self-imitation learning reuses the agent's own best past trajectories to steer learning towards them (Oh et al., 2018). Go-Explore keeps an explicit archive of states it has reached and returns to promising ones before

exploring further, which overcomes the failure of agents to revisit good states (Ecoffet et al., 2021).

(c) **Pruning.** Evaluating sub-networks is costly, so proxies are needed. EagleEye showed that a fast evaluation step using adaptive batch-normalisation statistics correlates much better with final accuracy than direct evaluation of the pruned weights (Li et al., 2020). In NAS, reduced-training and "zero-cost" proxies are evaluated by how well they preserve the ranking of candidates (Abdelfattah et al., 2021).

For RL pruning, what the agent *retains* can differ from what it *finds*, and selection needs validation data separate from the search signal. The thesis measures the rank of the terminal policy against the best visited one and introduces archive-based selection on a held-out split (Research Question 3).

2.10 Post-Pruning Recovery and Fine-Tuning

Pruning usually reduces accuracy, which is then partly or wholly recovered by further training. The recovery procedure is not a detail:

(a) **Rewinding.** Weight rewinding and learning-rate rewinding can recover more accuracy than conventional low-learning-rate fine-tuning (Renda, Frankle and Carbin, 2020). Later work traced much of this effect to the learning-rate schedule used during retraining (Le and Hua, 2021).

(b) **Inherited weights.** Some structured-pruning results suggest that the inherited weights matter less than the pruned architecture, since training from scratch can match fine-tuning (Liu et al., 2019).

(c) **Benchmarks.** ShrinkBench identified inconsistent fine-tuning budgets as a major source of incomparable results (Blalock et al., 2020).

(d) **Recovery budgets in large models.** Retraining is costly enough that some methods avoid it entirely (Kwon et al., 2022; Frantar and Alistarh, 2023). Others use a small amount of recovery training (Ma, Fang and Wang, 2023) or substantial continued pre-training after structured pruning (Xia et al., 2024).

(e) **Effects beyond accuracy.** Pruning can affect a model in ways that test accuracy does not reveal: changes in behaviour under distribution shift (Liebenwein et al.,

2021), disparate effects across groups (Tran et al., 2022), and transfer performance (Iofinova et al., 2022).

A comparison before recovery measures immediate damage, and one after recovery with different budgets partly measures the budgets; only **identical** recovery isolates the allocation, so the thesis fixes the recovery protocol and mask across methods (Research Question 5).

2.11 Hardware-Aware Neural Network Compression

Hardware-aware methods put a hardware cost, rather than a proxy such as parameter count, into the optimisation.

(a) **Measured or table-based latency.** NetAdapt adapts a network to a target platform using measured latency (Yang et al., 2018). HAQ uses hardware feedback to choose per-layer quantisation precision (Wang et al., 2019). Platform-aware NAS includes measured device latency in the search objective (Tan et al., 2019). Once-for-all trains a single network from which specialised sub-networks are drawn for different hardware constraints (Cai et al., 2020).

(b) **Structured pruning under a latency budget.** HALP formulates pruning as a knapsack problem that maximises retained saliency subject to a latency budget estimated from per-device lookup tables (Shen et al., 2022). SMCP extends this with soft masks that let pruned channels return during training while respecting a cost constraint (Humble et al., 2022).

These methods need a hardware model or measurement loop per device, and their benefit depends on enough *accuracy-feasible* headroom. The thesis treats that headroom as an empirical question to answer before any hardware-aware optimisation (Chapter 6), checking whether accuracy-feasible configurations offer enough room for a hardware-aware RL agent to exploit.

2.12 Parameter Count vs MACs/FLOPs vs Real Latency

Network cost can be described at three related but not interchangeable levels:

(i) **parameter count**, or parameter memory;

(ii) **arithmetic count**, as multiply–accumulate operations (MACs) or floating-point operations (FLOPs);

(iii) **measured runtime** on specific hardware.

(d) **FLOPs are not speed.** ShuffleNet V2 showed early that networks with similar FLOPs can differ markedly in speed, because memory access cost and degree of parallelism also matter (Ma et al., 2018).

(e) **The efficiency misnomer.** Dehghani et al. argued that common cost indicators — parameter count, FLOPs and throughput — are often assumed to be correlated. In fact they can disagree, so reporting only one can give a misleading picture of efficiency (Dehghani et al., 2022).

(f) **Sparsity does not translate into speed.** Unstructured sparsity reduces parameter count but not necessarily runtime, unless the hardware and kernels exploit it (Hoefler et al., 2021). Even structured reductions translate into speed-ups that depend on layer shapes and library kernels (Shen et al., 2022; He and Xiao, 2024).

(g) **Model-level latency studies.** NetAdapt showed that optimising indirect metrics such as MACs or weights need not reduce direct metrics such as latency and energy (Yang et al., 2018). Recent LLM work reports speed-ups only when a sparse inference engine supports the sparsity pattern (Yin et al., 2024).

What is less common is an *exhaustive* measurement, across *all* configurations of one search space, of how well parameter and MAC reductions predict latency and how much accuracy-feasible headroom exists; Chapter 6 reports this for all 1,296 structured configurations of two architectures.

2.13 Energy-Efficient and Sustainable Neural Network Inference

The energy cost of machine learning has become a research topic in its own right. Early analyses drew attention to the energy and carbon cost of training large NLP models (Strubell, Ganesh and McCallum, 2019). Researchers were urged to report efficiency alongside accuracy, with the label "Green AI" used for this agenda (Schwartz et al., 2020). Frameworks for the systematic reporting of energy and carbon footprints followed (Henderson et al., 2020). Industrial analyses have discussed the environmental implications of AI across the full model lifecycle (Wu et al., 2022). They have also argued that best practices can substantially reduce training energy and emissions

(Patterson et al., 2022). Carbon estimates for large models depend on lifecycle assumptions as much as on measured energy (Luccioni, Viguier and Ligozat, 2023).

Inference has received growing attention because it is repeated for every query:

(a) Once hardware efficiency gains and consolidated algorithms are taken into account, inference energy has grown much more slowly than parameter-count trends would suggest (Desislavov, Martínez-Plumed and Hernández-Orallo, 2023).

(b) A systematic comparison of inference cost, measured as the energy needed for 1,000 inferences, found multi-purpose generative models to be orders of magnitude more expensive than task-specific models (Luccioni, Jernite and Strubell, 2024).

(c) Benchmarks of LLM inference energy on two GPU generations and multi-GPU configurations show how strongly energy depends on hardware and deployment configuration (Samsi et al., 2023).

(d) Fine-tuning, although cheap per run, contributes to energy use because it is performed so often (Wang et al., 2023).

(e) MLPerf Power defines a methodology for measuring the energy efficiency of ML systems from microwatt devices to data-centre clusters, and emphasises the role of system-level interactions (Tschand et al., 2025).

Energy-aware pruning. Within pruning, energy-aware pruning used an energy model, calibrated from hardware measurements, to guide which layers to prune. It showed that reductions in model size or computation did not necessarily reduce energy (Yang, Chen and Sze, 2017).

Measurement is central and difficult.

(a) **Software power meters.** On CPUs, Intel's Running Average Power Limit (RAPL) interface exposes energy counters for the processor package and its domains without additional hardware. Early experience reports discussed its accuracy and practical pitfalls (Khan et al., 2018). A comparison of software-based power meters distinguished tools that *measure* through hardware interfaces from tools that *estimate* from models such as thermal design power (TDP) (Jay et al., 2023). A recent comparative analysis of RAPL-based measurement mechanisms found that

they differ mainly in usability and robustness, and that RAPL is more accurate than statistical estimation models (Raffin and Trystram, 2025).

(b) **Package scope.** RAPL measures the *package*, not the program. Any other activity in the package (other cores, integrated graphics, uncore components, thermal state) enters the measurement. Stable measurement therefore requires controlled conditions, repeated runs and an explicit stability criterion.

The thesis therefore reports only directly measured package energy, never TDP- or model-based estimates, and reports energy as unresolved where measurement could not be qualified. Inference energy on one processor is only one component of a model's environmental footprint, and converting it into carbon or lifecycle impact needs further data and assumptions (Wu et al., 2022; Luccioni, Viguier and Ligozat, 2023); the thesis makes no environmental or carbon claims and uses "sustainability" only for reduced memory, computation and measured operational energy.

2.14 Research Gap

The literature establishes that layer-wise allocation is important, that RL can produce competitive allocations, that sensitivity carries useful information, and that compression metrics do not straightforwardly become deployment efficiency. Six narrower questions remain open.

(i) **Adaptive allocation versus strong deterministic allocation, under controlled conditions.** Allocation matters (Lee et al., 2021; Liu et al., 2022; Yin et al., 2024), but it is unclear how a learned PPO allocation compares with Uniform, Random, global magnitude, ERK and LAMP built at exactly the sparsity the agent chose. → *Research Question 1.*

(ii) **The separate contributions of sensitivity to exploration and to final accuracy.** Sensitivity is widely used as a pruning criterion (Molchanov et al., 2019; Nonnenmacher et al., 2022; S. Yu et al., 2022). When it is used to *guide* a search, its effect on search efficiency is rarely separated from its effect on final accuracy, or compared with uninformative controls. → *Research Question 2.*

(iii) **Retention of the best policies found during search.** RL-based pruning is usually judged by its final policy (He et al., 2018; Alwani, Wang and Madhavan, 2022; Camci et al., 2022; Ganjdanesh, Gao and Huang, 2024). Whether the optimiser

actually retains the best policies it encounters, a failure mode documented in neighbouring fields (Yu et al., 2020; Ecoffet et al., 2021), has not been examined systematically for pruning. → *Research Question 3.*

(iv) **Hidden incentives in pruning rewards.** Pruning rewards combine accuracy and sparsity with hand-chosen weights. RL research shows that more capable optimisers exploit misspecified rewards more strongly (Pan, Bhatia and Steinhardt, 2022; Skalse et al., 2022), and constrained formulations exist for this reason (Achiam et al., 2017; Gu et al., 2024). Whether better pruning search exposes such incentives, and whether an explicit accuracy constraint controls them, has not been tested directly. → *Research Question 4.*

(v) **Comparisons before equal recovery.** Recovery procedures strongly affect pruned accuracy (Blalock et al., 2020; Renda, Frankle and Carbin, 2020; Le and Hua, 2021). Differences between pruning methods measured before recovery, or under different recovery budgets, may therefore overstate the differences that remain after identical recovery. → *Research Question 5.*

(vi) **Compression metrics versus deployment efficiency.** Parameter sparsity, MACs, latency and energy can disagree (Yang, Chen and Sze, 2017; Ma et al., 2018; Yang et al., 2018; Dehghani et al., 2022). How strongly parameter and MAC reductions predict measured CPU latency across a whole pruning search space, how much accuracy-feasible latency headroom exists, and whether structured realisation of a learned allocation reduces *directly measured* energy without material accuracy loss all remain empirical questions. Energy measurement on general-purpose CPUs adds its own reliability problem (Khan et al., 2018; Raffin and Trystram, 2025). → *Research Question 6.*

Chapter 3 describes the framework that emerged, distinguishing original components from those introduced in response to diagnostic results.

CHAPTER 3
PROPOSED FRAMEWORK

3.1 General

This chapter describes the methodology developed in this thesis: a reinforcement learning (RL) framework that learns layer-wise pruning allocations with Proximal Policy Optimisation (PPO), guided by layer sensitivity.

Several components were added because a diagnostic analysis exposed a weakness of an earlier version, so the framework is presented **chronologically**, each component with the phase that introduced it and the problem it addressed (Table 3.1; Figure 3.1). The chapter explains *what* each component does and *why*; settings are in Chapter 4 and outcomes in Chapters 5 and 6.

Table 3.1: Development of the framework across phases. "Search" components affect which pruning policy is found or selected; "evaluation" components affect how a selected policy is assessed.

Component	Phase introduced	Role	Motivation
Layer-wise pruning units, discrete ratio actions, PPO agent	initial design (frozen in Phase 3)	search	learn a non-uniform allocation instead of one uniform ratio
Accuracy–sparsity reward R0	initial design (frozen in Phase 3)	search	reward retained accuracy and compression jointly
Normalised layer sensitivity	initial design (frozen in Phase 3)	search (guidance)	identify layers whose pruning is most damaging
Soft sensitivity prior on action logits	initial design; evaluated in Phase 3	search (guidance)	bias the agent towards gentler pruning of sensitive layers
Centred sensitivity prior	Phase 4	search (guidance)	remove the shared "prune less" pressure, keeping only *relative* sensitivity
Elite behavioural cloning (Phase 4 only)	Phase 4	search	steer PPO towards the best policies it has visited; dropped from Phase 5 on
Archive of visited policies and held-out selection	Phase 4	search (selection)	the terminal PPO policy may be worse than policies visited during search
Accuracy-constrained reward	Phase 5	search	the accuracy–sparsity trade-off reward encouraged excessive pruning
Constraint-aware archive selection	Phase 5	search (selection)	choose the most compressed candidate that meets the accuracy constraint on held-out data
Equal fixed-mask recovery	Phase 6	evaluation	compare methods after identical fine-tuning
Physical structured realisation; memory, MAC, latency and energy measurement	Paper 3, Phase 7A	evaluation (extension)	test whether compression becomes deployment efficiency
Deployment-oriented structured PPO	Phase 7B	search (extension)	learn directly in the structured deployment space

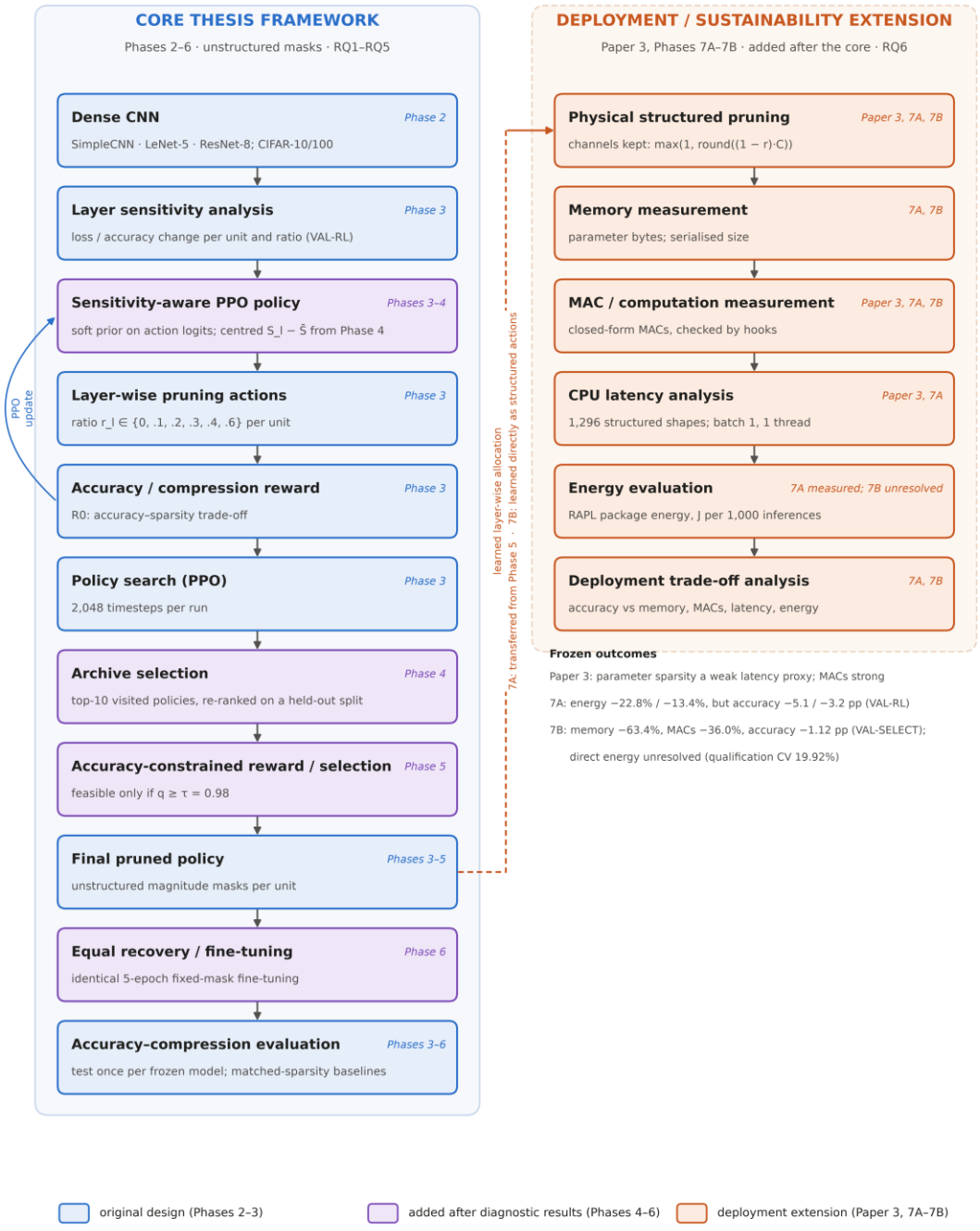


Fig. 3.1: Overall thesis framework. Left: the core framework (Phases 2–6, unstructured masks), each component tagged with its phase (blue: original design; purple: added after diagnostic results). Right: the later deployment extension (Paper 3, Phases 7A–7B). Dashed arrow: the learned allocation is transferred in Phase 7A and learned as structured actions in Phase 7B.

All symbols are defined in the List of Main Notation in the front matter.

3.2 Problem Formulation

Let M be a trained convolutional network whose prunable convolutional and fully connected layers are grouped into N_L **pruning units**, indexed l = 1, …, N_L. A pruning policy assigns each unit a pruning ratio:

$$\mathbf{r} = [r_1, r_2, \dots, r_{N_L}], r_l \in \mathcal{R} = \{0,0.1,0.2,0.3,0.4,0.6\}. \quad (3.1)$$

Applying **r** to M yields the pruned network $M_{\mathbf{r}}$. The uniform policy is one point in this space; the framework searches for **non-uniform** allocations, because units differ in weight share, computation, sensitivity and downstream effect. Compression is measured by the sparsity ρ over **all** convolutional and fully connected weight tensors, including the never-pruned output classifier:

$$\rho(M_{\mathbf{r}}) = 100 \cdot \frac{\sum_t \#\{w \in t : w = 0\}}{\sum_t |t|}, \quad (3.2)$$

where t ranges over every Conv2d and Linear weight tensor; biases and batch-normalisation parameters are neither counted nor pruned. **The goal** is conceptual, not a guaranteed optimum: a layer-wise policy that balances predictive performance and compression as specified by the reward and constraints in force. Formally, the agent maximises

$$\max_\theta \mathbb{E}_{\mathbf{a}\sim\pi_\theta}[R(M_{\mathbf{r}(\mathbf{a})})], \quad (3.3)$$

where R encodes the trade-off (Sections 3.7 and 3.13). Equation (3.3) defines what PPO optimises, not a claim that PPO finds the best policy: the space is searched stochastically, the returned policy is chosen by a separate selection rule (Sections 3.11–3.13), and enumeration (Section 3.6) shows how close it comes to the best one.

3.3 Layer-Wise Pruning Representation

Pruning units. Each architecture registers exactly four pruning units (Table 3.2), so every setting has the same action space. A unit consists of one or more weight tensors, and an

action applies the same ratio to every tensor in the unit; the output classifier is always excluded.

Table 3.2: Pruning units of the core framework (unstructured).

Architecture	Unit 1	Unit 2	Unit 3	Unit 4	Never pruned
SimpleCNN	conv1 (`features.0`)	conv2 (`features.3`)	conv3 (`features.6`)	fc1 (`classifier.1`)	output layer (`classifier.3`)
LeNet-5	conv1	conv2	fc1	fc2	output layer
ResNet-8	stem convolution	stage 1 (both block convolutions)	stage 2 (both block convolutions and the 1×1 shortcut)	stage 3 (both block convolutions and the 1×1 shortcut)	output layer

Within-unit pruning (core framework). In Phases 3–6, a ratio r_l is applied as **L1 unstructured** pruning to every tensor of unit l: the round($r_l \cdot |t|$) smallest-magnitude weights of each tensor t are masked to zero, using PyTorch's pruning reparameterisation (Paszke et al., 2019). Shapes are unchanged and the mask can be frozen during fine-tuning (Section 3.15), so every pruned network is a masked copy of the same dense network and the whole policy space can be evaluated once and cached. In SimpleCNN a unit is a single layer; in ResNet-8 it is a residual stage, so one action prunes several tensors at once. The structured extension (Section 3.16) redefines the ResNet-8 units around the residual coupling, so the two definitions are not interchangeable.

3.4 Reinforcement Learning Formulation

Choosing **r** is a **finite-horizon sequential decision process** with one decision per pruning unit, visited in the fixed order $l = 1, \ldots, N_L$ (Algorithm 3.1). At step l the agent observes a state s_l describing unit l and the decisions so far (Section 3.5), samples an action $a_l \sim \pi_\theta(\cdot \mid s_l)$ (Section 3.6), and the environment applies ratio $r(a_l)$ to unit l of the working copy, after which s_{l+1} is formed. After the last unit ($N_L = 4$ steps), $M_{\mathbf{r}}$ is evaluated on the reward split and the episode reward R is paid; intermediate rewards are zero, and each episode starts from an unpruned copy of M. Transitions are deterministic given the actions; the state summarises the current unit and the cumulative pruning, not the full history, so the process is not claimed to be strictly Markov in the state.

Why RL rather than enumeration? The small space is enumerated only to *evaluate* the search (Section 3.6); the same agent and reward apply unchanged to spaces too large to enumerate and to non-differentiable objectives such as measured hardware cost.

3.5 State Representation

Core framework (Phases 3–6). The state is the seven-feature vector of the historical environment (`src/rl`, `unit_state`), computed for the current unit l of N_L units:

$$s_l = [\frac{l-1}{N_L-1}, \log(1 + n_l), \frac{n_l}{\max_j n_j}, |w_l|, \sigma(w_l), \sum_{j<l} r_j, \sigma_l^{\text{slot}}], \quad (3.4)$$

where n_l is the number of weights in unit l, mean $|w_l|$ and $\sigma(w_l)$ are the mean absolute value and the standard deviation of its current weights, $\Sigma_{j<l}\, r_j$ is the cumulative pruning ratio so far, and σ^slot_l is a sensitivity slot. **The sensitivity slot was set to 0 in every condition of Phases 3, 4 and 5**, as pre-registered, so in the core framework sensitivity never entered PPO through its *observation*, only through the soft action prior (Sections 3.9–3.10). After the last unit the state is a zero vector.

Deployment extension (Phase 7B). The deployment-oriented structured PPO (Section 3.16.3) uses a different, eight-feature state that includes centred sensitivity. The two definitions are listed separately in Table 3.3 and must not be mixed.

Table 3.3: State features by phase.

#	State feature	Meaning	Phases used
1	$(l-1)/(N_L-1)$	position of the current unit	3, 4, 5
2	$\log(1+n_l)$	log weight count of the unit	3, 4, 5
3	$n_l / \max_j n_j$	relative weight count	3, 4, 5
4	mean $\lvert w_l \rvert$	mean weight magnitude (current, masked weights)	3, 4, 5
5	$\sigma(w_l)$	weight standard deviation	3, 4, 5
6	$\Sigma_{j<l}\, r_j$	cumulative pruning ratio so far	3, 4, 5
7	sensitivity slot	**set to 0 in all Phase 3–5 conditions**	3, 4, 5 (constant 0)
1′	$(l-1)/(N_L-1)$	position of the current unit	7B
2′	$\log(1+C_l)$	log channel count of the unit	7B
3′	unit share of dense parameters	parameter contribution	7B
4′	unit share of dense MACs	computational contribution	7B
5′	$\tilde{S}_l$	centred structured sensitivity	7B
6′	r_{l-1}	ratio chosen for the previous unit	7B
7′	$\text{mean}_{j<l}\, r_j$	mean ratio chosen so far	7B
8′	$\hat{E}$(partial policy) / $\hat{E}$(dense)	predicted energy of the policy so far (remaining units at ratio 0), relative to dense	7B

3.6 Action Space

At each unit the agent chooses one of six actions, which map to pruning ratios:

$$r(a) \in \{0,0.1,0.2,0.3,0.4,0.6\}, a \in \{0,1,2,3,4,5\}. \qquad (3.5)$$

That is, 0%, 10%, 20%, 30%, 40% or 60% of the unit's weights (core framework) or channels (structured extension); no action is ever masked. A policy is the sequence of the four unit actions, so the number of distinct policies is

$$|\mathcal{R}|^{N_L} = 6^4 = 1{,}296 \qquad (3.6)$$

in every setting. The highest ratio is 0.6, so even the most aggressive policy [5, 5, 5, 5] leaves 40% of every pruned tensor, and the never-pruned classifier caps each architecture's total sparsity ρ below 60%. The small space is deliberate: all 1,296 policies can be evaluated once per setting, giving an exact reward landscape that shows how good

the policies PPO *visits* and *returns* are. Policies do not correspond across architectures: in ResNet-8 an action masks a fraction of a residual-stage weight group, and the residual stage itself is not structurally removed (Table 3.2).

3.7 Initial Reward Function

The initial reward R0 was inherited from the original framework and frozen in Phase 3. It is paid at the end of each episode:

$$R_0 = 1.5 \cdot \frac{A_p}{A_b} + \lambda_s \cdot \rho + 0.05 \cdot D, \lambda_s = 0.01. \qquad (3.7)$$

Here A_p is the accuracy (%) of M_r on the reward split (VAL-RL); A_b is the dense reference's accuracy on the same split, so A_p/A_b is the retained accuracy q; ρ is the sparsity in **percent** (Equation 3.2); and D is the number of distinct actions (1 to 4), a mild incentive against trivially uniform policies. The coefficients were never tuned; every frozen phase uses $\lambda_s = 0.01$, not the library default 0.04 of the original notebook (Phase-4 pre-registration). Equation (3.7) trades accuracy against sparsity at a fixed exchange rate, with no threshold below which accuracy loss becomes unacceptable (Section 3.13).

3.8 Layer Sensitivity Analysis

The sensitivity of a unit measures how much the network's loss increases when that unit alone is pruned. It is computed once per setting on the dense reference, using only the reward split (VAL-RL), never the selection split or the test set (`src/sensitivity`, `loss_sensitivity`):

(i) **Baseline loss.** Evaluate the dense model's mean cross-entropy loss L_0 on VAL-RL.

(ii) **Single-unit probes.** For each unit l and probe ratio r ∈ {0.1, 0.2, 0.4, 0.6}, prune *only* unit l by r (L1 unstructured, as in Section 3.3) and evaluate the loss $L_{l,r}$.

(iii) **Raw sensitivity.** Average the relative loss increases over the probe ratios:

$$S_l^{\text{raw}} = \frac{1}{4} \sum_{r \in \{0.1,0.2,0.4,0.6\}} \frac{L_{l,r} - L_0}{L_0}. \qquad (3.8)$$

(iv) **Normalisation.** Min–max normalise across the units of the network:

$$S_l = \frac{S_l^{\text{raw}} - \min_j S_j^{\text{raw}}}{\max_j S_j^{\text{raw}} - \min_j S_j^{\text{raw}}} \in [0,1]. \tag{3.9}$$

Normalisation makes sensitivities comparable across architectures but discards their absolute scale, and loss registers small degradations that do not yet change predictions. The vector was recomputed per split configuration and, in Phase 7B, with structural probes (Section 4.10). Because the probes miss interactions between units, sensitivity is *guidance* for the search, not a substitute for it.

3.9 Sensitivity-Guided Action Prior

In the initial design, sensitivity guides the agent through a **soft prior on the action logits**. Let $z_l(a)$ be the logit that the PPO actor assigns to action a at unit l. Before the action distribution is formed, the logits are shifted:

$$\begin{aligned} \tilde{z}_l(a) &= z_l(a) - c_l \cdot \frac{r(a)}{0.6}, \pi_\theta(a \mid s_l) \\ &= \text{softmax}_a(\tilde{z}_l(a)). \end{aligned} \tag{3.10}$$

Here c_l is the prior coefficient of unit l, and r(a)/0.6 rescales the ratio so that the most aggressive action has weight 1. In Phase 3, $c_l = \beta \cdot S_l$ with $\beta = 0.5$ (condition P1, "correct prior"). The shift (`src/rl`, `SoftPriorPolicy`) is applied in every forward pass, both when actions are sampled and when log-probabilities are evaluated in the PPO update. The penalty grows with sensitivity and aggressiveness, and action 0 is never penalised. The prior **shifts preferences but does not replace the learned policy**: every action keeps a non-zero probability, the actor can override it, and with $c_l = 0$ the policy is exactly standard PPO. Its shuffled and constant controls (Section 4.9.1) separate *layer-specific* information from a generic bias towards lighter pruning.

3.10 Centred Sensitivity Prior

Since every $S_l \geq 0$, the total penalty $\Sigma_l c_l = \beta \cdot \Sigma_l S_l$ of the Section 3.9 prior is positive, so the correct, shuffled and constant priors carry the **same total conservatism**, and any benefit they share could come from this common "prune less" pressure rather than from information about *which* units are sensitive. Phase 4 therefore introduced a **centred** prior that keeps only relative sensitivity:

$$\tilde{S}_l = S_l - \overline{S}, \overline{S} = \frac{1}{N_L} \sum_{j=1}^{N_L} S_j, c_l = 1.0 \cdot \tilde{S}_l, \qquad (3.11)$$

used in Equation (3.10) in place of $c_l = 0.5 \cdot S_l$. Its coefficients sum to zero ($\Sigma_l c_l = 0$), so the prior carries no net conservatism: aggressive actions are penalised on units more sensitive than average ($\tilde{S}_l > 0$) and favoured on less sensitive ones ($\tilde{S}_l < 0$), while action 0 is unchanged. Pruning is redistributed from sensitive to insensitive units instead of being reduced everywhere. This centred prior (condition "P2" in the Phase-4 and Phase-5 records) became the standard prior of the core framework; its controls are listed in Section 4.10. The sensitivity state slot remained 0, so sensitivity acts on the search only through this prior.

3.11 Policy-Retention Problem

The initial design returns the agent's **terminal policy**: after training, the deterministic policy $a_l = \text{argmax_a } \pi_\theta(a \mid s_l)$ is rolled out once. PPO samples hundreds of complete policies during training, but a stochastic policy-gradient method updating on noisy reward estimates can sample an excellent policy early and then drift towards a region of high but not maximal average reward, so returning only the terminal policy discards evaluations already paid for. With exact landscapes (Section 3.6), the problem is the gap between the rank of the **best policy sampled** and that of the **policy returned**; when the first is consistently better, the limitation lies in *retention and selection*, not exploration. Phase 4 introduced archive-based selection to address it.

3.12 Archive-Based Policy Selection

3.12.1 Archive (Phase 4)

During each PPO run, an **archive** $\mathcal{A}$ records every unique policy sampled, with its action vector **a**, its VAL-RL reward and the episode of first sampling; recording is read-only and draws no random numbers.

3.12.2 Selection rule (Phase 4)

At the end of training, the archive is reduced to a candidate set and one candidate is selected on held-out data:

$$\mathcal{C} = \text{top}_{10}\{\mathbf{a} \in \mathcal{A} \text{ by } R_{\text{VAL-RL}}(\mathbf{a})\}, \mathbf{a}^{\star} = \text{argmax}_{\mathbf{a}\in\mathcal{C}} R_{\text{VAL-SELECT}}(\mathbf{a}). \quad (3.12)$$

$\mathcal{C}$ holds the 10 unique archived policies with the highest VAL-RL reward (ties: earlier discovery), and **a**⋆ is the candidate scoring highest under the same reward on the disjoint **selection split** (VAL-SELECT; ties: higher VAL-RL reward, then earlier discovery). The two stages avoid relying only on the final policy and only on the best VAL-RL reward, which would favour candidates whose reward is inflated by noise. **Archive selection is a retention and selection mechanism based on validation evidence**: it decides which visited policy is returned, does not change what the agent learns, and is defined relative to the validation splits, not the test set.

3.12.3 Elite behavioural cloning (Phase 4 only)

Phase 4 also tested a training-time complement to the archive, using the best policies seen so far as additional supervised targets for the PPO actor:

$$\mathcal{L} = \mathcal{L}_{\text{PPO}} + \eta\mathcal{L}_{\text{elite}}, \mathcal{L}_{\text{elite}} = -\frac{1}{4K}\sum_{e=1}^{K}\sum_{l=1}^{4}\log \pi_{\theta}(a_l^{(e)} \mid s_l^{(e)}), \quad (3.13)$$

where $(s^{(e)_l}, a^{(e)_l})$ are the state–action pairs of the K = 5 best unique VAL-RL-reward policies seen so far and η = 0.1 (active once five unique policies have been seen). Phase 4 combined it with the archive rule (S2A); Phase 5 removed it, so the mature core framework does not include Equation (3.13).

3.12.4 Constraint-aware selection (Phase 5)

With the accuracy-constrained reward (Section 3.13) the selection rule changed with it. The candidates are still the top 10 archived policies by VAL-RL reward, frozen and hashed *before* any VAL-SELECT evaluation, and the selected policy is

$$\mathbf{a}^{\star} = \operatorname{argmax}_{\mathbf{a}\in\mathcal{C}}\{\rho(\mathbf{a}) : q_{\text{VAL-SELECT}}(\mathbf{a}) \geq \tau\}, \qquad (3.14)$$

that is, the **most compressed candidate whose retained accuracy on the selection split meets the threshold** (ties: higher VAL-SELECT accuracy, higher VAL-RL reward, earlier discovery; if none qualifies, the highest $q_{\text{VAL-SELECT}}$, with the fallback reported). Training rewards compression inside the feasible region; selection confirms feasibility on held-out data.

3.13 Reward Misalignment and Accuracy-Constrained Reward

3.13.1 Why R0 can encourage excessive pruning

Equation (3.7) converts accuracy and sparsity into one scalar at a fixed rate: an extra percentage point of sparsity is worth 0.01, and an extra percentage point of retained accuracy 0.015. A policy that gives up accuracy for enough sparsity can therefore score *higher* than a gentler policy, so in fragile architectures the reward-optimal policy can be one a practitioner would reject. A weak optimiser hides this; once search and retention improve, the agent finds exactly what the reward specifies, a pruning instance of more capable optimisers exploiting misspecified rewards more strongly (Pan, Bhatia and Steinhardt, 2022; Skalse et al., 2022). The alternative trade-off rewards of Phase 4 (Section 4.12) met none of their criteria, so Phase 5 redesigned the reward around an explicit constraint.

3.13.2 Accuracy-constrained reward (Phase 5)

The Phase-5 reward is constraint-first. With q = A_p/A_b on VAL-RL and the sparsity written as a fraction (ρ/100):

$$R_c = \begin{cases} 1 + \dfrac{\rho}{100} + \varepsilon q, & q \geq \tau, \\ -\kappa(\tau - q), & q < \tau, \end{cases} \quad \tau = 0.98, \varepsilon = 0.01, \kappa = 10. \tag{3.15}$$

A policy is *feasible* if it keeps at least 98% of the dense VAL-RL accuracy; τ was chosen before any PPO run by a pre-registered rule on the VAL-RL landscapes (Section 4.9.3), without the test set. The offset 1 makes every feasible reward larger than every infeasible one; ρ/100 pays for compression only inside the feasible region; ε·q breaks ties towards higher accuracy; and κ sets a penalty proportional to the shortfall below τ, keeping a learning signal towards feasibility. There is no diversity term. **Compression is rewarded only inside the acceptable-accuracy region**: the reward-optimal policy is the most compressed feasible policy on VAL-RL, matching a 2% relative accuracy tolerance. Two limitations are inherent and examined in Chapter 5: VAL-RL feasibility does not imply feasibility on unseen data, and τ is a *relative* threshold, not an absolute accuracy bound. Phase 5 kept the centred prior, with a prior-free condition to measure its contribution.

3.14 Final Core Framework

Table 3.4 lists the components that survived the diagnostic sequence, separating **search** components, which determine the selected policy, from **evaluation** components, which assess it.

Table 3.4: Mature core framework (as frozen after Phase 5, evaluated in Phase 6).

Stage	Component	Type	Section
1	Dense reference model M, trained once per setting	input	3.2
2	Normalised loss sensitivity S_l on VAL-RL	search (guidance)	3.8
3	PPO over four units with six ratio actions; seven-feature state (sensitivity slot 0)	search	3.4–3.6
4	Centred sensitivity prior, $c_l = 1.0 \cdot \tilde{S}_l$, on the action logits	search (guidance)	3.10
5	Accuracy-constrained reward R_c with $\tau = 0.98$ on VAL-RL	search	3.13
6	Archive of unique visited policies; top-10 candidates by VAL-RL reward	search (retention)	3.12
7	Constraint-aware selection on VAL-SELECT	search (selection)	3.12.4
8	Selected pruning policy $\mathbf{a}\star$, applied as unstructured masks	output	3.3
9	Equal fixed-mask recovery (5 epochs, identical for every method)	**evaluation**	3.15
10	Test evaluation, once per frozen model; matched-sparsity baselines	**evaluation**	Chapter 4

Relative to the initial version, the prior is centred, the returned policy comes from an archive on held-out data, and reward and selection share one constraint; behavioural cloning was dropped. Equal recovery (stage 9) is **not part of PPO's decision making**: the reward is always computed *without* fine-tuning, and recovery is applied only after selection, identically to every compared method, so the final comparison reflects the allocation.

3.15 Equal Recovery and Fine-Tuning Protocol

Pruning methods differ in how much accuracy they lose *immediately*, but in deployment a pruned network is normally fine-tuned before use, and recovery can reduce or reverse those differences (Renda, Frankle and Carbin, 2020; Le and Hua, 2021). To compare allocations fairly after recovery, Phase 6 applied one fixed protocol to every compared model (`experiments/phase6/phase6_lib.py`, `PROTOCOL` and `finetune`): Adam with cosine annealing to zero over E epochs, stepped once per epoch,

$$\eta_e = \frac{1}{2}\eta_0(1 + \cos(\pi e/E)), \eta_0 = 10^{-4}, e = 0, \ldots, E-1, \quad (3.16)$$

with E = 5 epochs fixed in advance by a timing rule (all settings in Section 4.13). The mask is **frozen**, so no pruned weight regrows and none is newly pruned; batch

normalisation runs in training mode during fine-tuning and in evaluation mode for every evaluation; the models of one comparison triple **share the fine-tuning seed** and data order; and **no checkpoint is selected** (the final epoch is used). The protocol is deliberately modest: it maximises no method's recovered accuracy but gives every allocation the same opportunity to recover. The extension reuses it on physically compacted networks, which have no masks to freeze; the Phase-7B accuracy proxy shortens it to 2 epochs.

3.16 Deployment-Oriented Structured Extension

> **This section describes a later extension**, developed after the core framework (Phases 3–6), to test whether compression becomes real deployment efficiency (Research Question 6). Its deployment-oriented structured PPO is a separate variant, not the final form of the core method.

3.16.1 From masks to physical structure

The core framework's **unstructured** masks keep the dense tensor shapes. On general-purpose CPUs the zeros are still stored and multiplied unless sparse kernels are used, so parameter sparsity need not reduce latency or energy (Hoefler et al., 2021; He and Xiao, 2024). The extension therefore realises a layer-wise allocation as a **physically smaller dense network**, using structured channel (filter) and neuron removal (He, Zhang and Sun, 2017; Li et al., 2017). For a unit with C_l output channels or neurons and ratio r_l, the number of channels kept is

$$k_l = \max(1, \mathrm{round}((1 - r_l)C_l)), \tag{3.17}$$

and the k_l channels with the largest L1 filter norms on the dense model are kept, so the result does not depend on the order of compaction (`src/pruning/structured_units.py`); removing an output channel also removes the matching input of every layer that reads it, and the output layer is never pruned. **Residual coupling (ResNet-8):** layers sharing a residual stream must keep the same channel set, so the structured ResNet-8 units differ from those of Table 3.2 (Section 4.6.2), and a masked ResNet-8 policy maps onto them only approximately.

3.16.2 Measurement: three levels of deployment efficiency

A compacted network is assessed at three related but non-interchangeable levels, used throughout Chapter 6; a result at one level does not establish a result at the next:

(i) **Memory / compression:** parameter memory, the sum over parameters of (number of elements × element size) in bytes, with the serialised checkpoint size as a secondary measure;

(ii) **Computation:** MACs, the multiply–accumulate operations per image of all convolutional and linear layers from a closed form checked against forward hooks, and the measured batch-1 CPU latency T(**r**);

(iii) **Operational energy:** E(**r**), the CPU package energy per 1,000 inferences, read from the Intel RAPL counters through the Windows Energy Meter Interface, with no TDP-based or modelled estimate.

The measurement protocols are specified in Sections 4.17–4.18.

3.16.3 Two uses of the extension

(a) Structured transfer of the learned allocation (Phase 7A). The most frequently selected Phase-5 policy's action indices are read as structured ratios (Equation 3.17), and the compacted network is recovered with the equal protocol and measured against a dense model and identically recovered structured comparators. The agent is not retrained: Phase 7A tests the transfer of an allocation from *masked* to *structured* pruning, not a new search.

(b) Deployment-oriented structured PPO (Phase 7B). Phase 7B redesigns the agent to search directly in the structured space of SimpleCNN. Its search reward uses a *surrogate (predicted)* energy saving, not measured energy, so the name implies no measured energy result:

(a) **Actions, state and prior.** Structured ratios per unit, so every action vector is a physical channel-width configuration; the eight features of Table 3.3 (rows 1′–8′); and the centred prior of Equation (3.10) computed from structured sensitivity.

(b) **Accuracy proxy.** Feasibility is judged by $\hat{q}$ = A_2/A_2,dense, the VAL-RL accuracy after a 2-epoch version of the recovery protocol relative to the dense

model's; the proxy and its threshold $\hat{\tau}$ were validated against 5-epoch recovery before the search (Chapter 4).

(c) **Energy surrogate.** Energy cannot be measured inside the search, so the reward uses a surrogate fitted beforehand to directly measured energies of sampled policies, linear in the policy's measured batch-1 latency T_{table} (Paper 3 latency tables):

$$\widehat{E}(\mathbf{r}) = a + b \cdot T_{\text{table}}(\mathbf{r}), a = -0.117, b = 13.762(\text{J per 1,000 inferences; } T \text{ in ms}). \quad (3.18)$$

The surrogate makes the search feasible. **It is not a measurement**: a predicted saving is never reported as an energy result, and any energy claim about a final model requires direct measurement under a qualified protocol.

(a) **Reward.** Constraint-first, as in Equation (3.15), with the compression term replaced by the predicted energy saving:

$$R_{7B} = \begin{cases} 1 + \alpha \dfrac{\hat{s}(\mathbf{r})}{s_{\max}} + \varepsilon \hat{q}, & \hat{q} \geq \hat{\tau}, \\ -\kappa(\hat{\tau} - \hat{q}), & \hat{q} < \hat{\tau}, \end{cases} \hat{s}(\mathbf{r}) = 1 - \frac{\widehat{E}(\mathbf{r})}{\widehat{E}(\text{dense})}, \quad (3.19)$$

where s_{max} is the largest predicted saving over the 1,296 policies; α, ε and κ are given in Section 4.9.7.

(a) **Feasible archive and selection.** Every unique *feasible* policy visited is archived, and the policy with the lowest predicted energy is selected (tie rules in Section 4.11); the primary policy is the one selected most often across the confirmatory seeds.

Phase 7B thus carries sensitivity guidance, a constraint-first reward, archive selection and equal recovery into the deployment setting, adding a proxy and a surrogate only to make the search affordable.

3.17 Overall Algorithm

Algorithm 3.1 summarises the mature core framework and Algorithm 3.2, kept separate, the extension's evaluation workflow.

Algorithm 3.1 — Sensitivity-aware adaptive pruning with archive selection and accuracy constraint (core framework).

```
Input:  trained dense network M with pruning units 1..N_L; ratio set R = {0, .1, .2, .3,
.4, .6};
        reward split VAL-RL; selection split VAL-SELECT; threshold tau = 0.98;
        PPO hyper-parameters (Table 4.6); episodes E_ep = 512; archive size K_c = 10
Output: selected pruning policy a*; pruned network M_a*

 1  A_b <- accuracy of M on VAL-RL
 2  for each unit l: S_raw[l] <- mean_{r in {.1,.2,.4,.6}} (Loss(M with only unit l
pruned by r) - Loss(M)) / Loss(M)    (3.8)
 3  S <- min-max normalise S_raw; c[l] <- 1.0 * (S[l] - mean(S))
(3.9),(3.11)
 4  initialise PPO actor-critic pi_theta, V_phi; archive Arch <- empty map
 5  for episode = 1 .. E_ep:
 6      M' <- copy of M; actions <- []
 7      for l = 1 .. N_L:
 8          s_l <- state of unit l (seven features; sensitivity slot = 0)
(3.4)
 9          logits <- actor(s_l) - c[l] * r(a)/0.6 for every action a
(3.10)
10          a_l ~ softmax(logits); prune every tensor of unit l of M' by r(a_l) (L1
unstructured); append a_l
11      A_p <- accuracy of M' on VAL-RL; rho <- sparsity of M' (all conv/linear weights)
(3.2)
12      R <- R_c(A_p / A_b, rho; tau)
(3.15)
13      if actions not in Arch: Arch[actions] <- (R, episode)
14      store transitions (rewards 0 at l < N_L, R at l = N_L); every 64 steps: PPO
update (clipped objective)            (3.20)
15  C <- the 10 entries of Arch with the highest R (ties: earlier episode)
(3.12)
16  freeze C; evaluate every candidate on VAL-SELECT
17  a* <- argmax rho over {a in C : q_VAL-SELECT(a) >= tau}; ties: higher VAL-SELECT
accuracy, higher R, earlier;
        if the set is empty: a* <- argmax q_VAL-SELECT (reported fallback)
(3.14)
18  M_a* <- M pruned by a*
19  [evaluation] recover M_a* with the equal fixed-mask protocol (5 epochs; Section
3.15), identically for every method
20  [evaluation] evaluate on the test set once, after the model is frozen
```

Steps 1–18 are search; steps 19–20 are evaluation and do not influence the selected policy. Earlier versions follow by substitution: Phase 3 used $c_l = 0.5 \cdot S_l$, R0 and the terminal policy (steps 15–17 replaced); Phase 4 used $c_l = 1.0 \cdot \tilde{S}_l$, R0 and Equation (3.12), with behavioural cloning (Equation 3.13).

Algorithm 3.2 — Deployment-oriented structured evaluation (extension; Phases 7A–7B).

```
Input:  dense network M; a structured policy a (7A: transferred from the core framework;
        7B: selected by the deployment-oriented structured PPO); comparator policies;
recovery protocol
Output: accuracy, parameter memory, MACs, latency and measured energy of each model

 1  for each model (dense, a, comparators):
 2      k_l <- max(1, round((1 - r(a_l)) * C_l)) for every unit; keep the k_l channels
with largest L1 norm          (3.17)
 3      build the physically compacted network; propagate removed channels to the layers
that read them
        (ResNet-8: shared channel set across the residual stream)
 4      recover with the equal protocol (5 epochs, identical seed and data stream);
```

```
record VAL-RL / VAL-SELECT accuracy
 5      Level 1: parameter memory (bytes) and serialised size
 6      Level 2: MACs (closed form, hook-checked); batch-1 CPU latency
 7  qualify the energy instrument on the dense model (pre-registered stability
criterion)
 8  if qualified: Level 3: paired sessions of package energy per 1,000 inferences, dense
anchor in every session
    else: report energy as unresolved (no estimate is substituted)
```

PPO objective. PPO (Schulman et al., 2017), as implemented in Stable-Baselines3 (Raffin et al., 2021), updates the actor by maximising the clipped surrogate objective, with advantages estimated by generalised advantage estimation (Schulman et al., 2016):

$$\begin{aligned}\mathcal{L}^{\mathrm{PPO}}(\theta) = \mathbb{E}_t[\min(\kappa_t(\theta)\widehat{A}_t, \mathrm{clip}(\kappa_t(\theta), 1-\epsilon, 1 \\ +\epsilon)\widehat{A}_t)] - c_v\mathbb{E}_t[(V_\phi(s_t) - \widehat{R}_t)^2] \\ + c_e\mathbb{E}_t[\mathcal{H}(\pi_\theta(\cdot \mid s_t))],\end{aligned} \tag{3.20}$$

where $\kappa_t(\theta)$ = π_θ(a_t | s_t) / π_θold(a_t | s_t) is the probability ratio, $\hat{A}_t$ the advantage estimate, $\hat{R}_t$ the return target, $\boldsymbol{\mathcal{H}}$ the policy entropy, and $\epsilon = 0.2$, $c_v = 0.5$ and $c_e = 0.01$ (Table 4.6; the hyper-parameters are identical in Phases 3, 4, 5 and 7B). The sensitivity prior enters through π_θ, whose shifted logits (Equation 3.10) define both κ_t and $\boldsymbol{\mathcal{H}}$; the behavioural-cloning term (Equation 3.13) is added only in the Phase-4 S2 runs.

3.18 Computational Workflow and Reproducibility

Three design choices make the framework measurable and reproducible (implementation in Chapter 4). **Exhaustive landscapes:** all 1,296 policies of every setting are evaluated once on the reward and selection splits, and PPO's terminal evaluations are read from this cache, which returns exactly the live value and consumes the random stream as an uncached run does, so every visited or returned policy has a known rank. **Separated data roles:** VAL-RL drives sensitivity, reward and training; VAL-SELECT only chooses among frozen candidates; the test set is read once per frozen, hashed model. **Pre-registered configurations:** constants, thresholds, seeds, selection rules, the recovery protocol and decision criteria were fixed before each phase's confirmatory runs, and none was tuned on test data or changed after results were seen.

Computational cost. A run needs 512 cached episode evaluations, 64-step PPO updates and, in the core framework, one VAL-SELECT evaluation of 10 candidates; equal recovery costs 5 epochs per model, and Phase 7B adds a 2-epoch proxy landscape of all 1,296 structured policies. This cost, compared with search-free deterministic methods, is discussed in Chapter 7.

3.19 Chapter Summary

A PPO agent assigns one of six pruning ratios to each of four units, with sensitivity entering only as a soft prior on the action logits. Phase 4 centred the prior and added archive selection on held-out data; Phase 5 replaced R0 with a constraint-first reward that pays for compression only when at least 98% of the dense accuracy is retained; Phase 6 fixed an equal, mask-preserving recovery protocol. The later deployment extension realises allocations as physically smaller networks: Phase 7A transfers the core allocation, and Phase 7B's deployment-oriented structured PPO searches the structured space directly, using a validated accuracy proxy and a latency-based energy surrogate, neither treated as a measurement.

CHAPTER 4
EXPERIMENTAL SETUP

4.1 Experimental Design Overview

This chapter specifies how the framework of Chapter 3 was evaluated, in enough detail to reproduce and audit every experiment; it reports no results. Every phase committed its conditions, seeds, metrics, tests and decision criteria before any confirmatory run. Phase 1 labelled superseded an earlier pipeline whose test set had been used for reward, sensitivity and epoch selection, and Phase 2 fixed the splits, trained the dense references and verified the shared library. The **core framework** (Phases 3–6) uses mask-based pruning on six architecture–dataset settings on a desktop CPU; the **deployment extension** (Paper 3, Phases 7A and 7B, an energy confirmation study) uses physically compacted networks measured on a laptop CPU.

4.2 Datasets

The experiments use CIFAR-10 and CIFAR-100 (Krizhevsky, 2009) (Table 4.1), scaled to [0, 1] and normalised per channel with constants from the official 50,000-image training file: means (R, G, B) 0.4914, 0.4822, 0.4465 for CIFAR-10 and 0.5071, 0.4865, 0.4409 for CIFAR-100, and standard deviations 0.2470, 0.2435, 0.2616 and 0.2673, 0.2564, 0.2762. Training uses random 32×32 crops with 4-pixel zero padding and random horizontal flips; evaluation is never augmented. CIFAR-100 is used only in Phases 2–6 and the Paper 3 latency tables.

4.3 Dataset Partitioning and Validation Protocol

4.3.1 Training, validation and test

For both datasets, a fixed permutation of the 50,000 training indices (`torch.randperm`, seed 42) assigns 45,000 images to **training** and the last 5,000 to **validation**; the official 10,000 images are the **test** set. Index lists are identical for both datasets and recorded with hashes in split manifests.

4.3.2 Two validation configurations

Validation is split into two class-stratified, disjoint parts: **VAL-RL** (reward split) drives sensitivity, rewards and PPO training, and **VAL-SELECT** (selection split) only chooses among already-produced candidates. The sizes changed once (Table 4.1), as declared before any Phase-5 data existed: **configuration I** (Phases 3–4; about 60% of each class to VAL-RL, largest-remainder rounding, per-class permutation; recorded as V_{RL} and V_{SELECT}) and **configuration II** (Phase 5 onward; per class the halves differ by at most one image).

The change removed a Phase-4 bias (the final Phase-4 search variant had been chosen by the VAL-SELECT rank of policies the archive had itself selected on VAL-SELECT). Of the 3,000 configuration-I VAL-RL images, 1,521 (CIFAR-10) and 1,515 (CIFAR-100) fall into the new VAL-SELECT, so sensitivity was recomputed. The configuration is stated with every accuracy, and accuracies from different configurations or splits are never compared.

Table 4.1: Datasets and split configurations.

Item	CIFAR-10	CIFAR-100
Image size / classes	32×32 RGB / 10	32×32 RGB / 100 (fine)
Training / full validation	45,000 / 5,000	45,000 / 5,000
Configuration I (Phases 3–4): VAL-RL / VAL-SELECT	3,000 / 2,000 (seed 20260928)	3,000 / 2,000 (seed 20260928)
Configuration II (Phase 5 on): VAL-RL / VAL-SELECT	2,500 / 2,500 (seed 20260930)	2,500 / 2,500 (seed 20260930)
Test	official 10,000	official 10,000

4.3.3 Use of each split by phase

In every phase (Table 4.2) the reward split never selects the returned policy, and the test set never informs a search, selection or design choice.

Table 4.2: Data-split usage by phase.

Phase	Training	VAL-RL	VAL-SELECT	Test
2 Dense references	weight updates	— (full validation selects the epoch)	—	once per hashed reference
3	not used	I: sensitivity, reward, landscape, exploration metrics	not used	once per frozen final policy and matched baseline; confirmatory
4	not used	I: reward, training, search metrics, Stage-A/B selections	I: archive re-ranking, held-out ranks, Stage-C selection	locked until all selections and final policies were frozen; confirmatory
5	not used	II: sensitivity, choice of τ, reward, training, archive candidates	II: selection among frozen candidates only	after the final-policy list was committed; confirmatory
6	fine-tuning	II: recorded per epoch; no selection	II: recorded per epoch; no selection	after the 480 final checkpoints were committed; confirmatory
Paper 3	recovery	II: recovered accuracy, proxy validation, threshold calibration	II: stored, not analysed	not read
7A	recovery	II: **primary** accuracy criterion	II: secondary	not used
7B	proxy and final recovery	II: sensitivity, proxy, reward, archive	II: held out; **final** accuracy criterion	never read
7B energy confirmation	—	—	7B values, read only	—

The Phase-7A policy was selected on VAL-SELECT in Phase 5, so its validation accuracy may be optimistically biased relative to the baselines (recorded in its pre-registration); Phases 7A and 7B report validation accuracy only.

4.4 Neural Network Architectures

The networks are defined in `src/models`:

(a) **SimpleCNN** (the historical network): three 3×3 convolutions (padding 1, with bias) with 32, 64 and 128 filters, each with ReLU and 2×2 max-pooling; a 256-unit hidden layer with ReLU on the 2,048 flattened features; an output layer of C units; no batch normalisation or dropout.

(b) **LeNet-5, adapted to CIFAR** (LeCun et al., 1998): two 5×5 valid convolutions with 6 and 16 filters, each with ReLU and 2×2 max-pooling; fully connected layers 400 → 120 → 84 with ReLU; an output layer of C units.

(c) **ResNet-8** (He et al., 2016) (n = 1, 6n + 2 = 8 weight layers): a 16-channel 3×3 stem convolution with batch normalisation; three single-BasicBlock stages with 16, 32 and 64 channels and strides 1, 2 and 2; identity shortcuts in stage 1 and 1×1 projections in stages 2 and 3; global average pooling and an output layer of C units; bias-free convolutions with Kaiming-normal (fan-out) initialisation.

Table 4.3: Network architectures and pruning units (core framework). Counts are CIFAR-10 / CIFAR-100. "Weights" are the Conv2d and Linear weights, the denominator of sparsity (Equation 3.2).

Architecture	Datasets	Pruning units (share of prunable weights)	Dense parameters	Weights (prunable)	Main role
SimpleCNN	C10, C100	conv1 0.14%, conv2 2.99%, conv3 11.94%, fc1 84.93%	620,362 / 643,492	619,872 / 642,912 (617,312)	all phases; the only network of Phase 7B
LeNet-5	C10, C100	conv1 0.74%, conv2 3.94%, fc1 78.78%, fc2 16.54%	62,006 / 69,656	61,770 / 69,330 (60,930)	core phases 3–6 only
ResNet-8	C10, C100	stem 0.56%, stage 1 6.01%, stage 2 18.69%, stage 3 74.74%	78,042 / 83,892	77,360 / 83,120 (76,720)	core phases; Paper 3; Phase 7A

The output layer, biases and batch-normalisation parameters are never pruned.

4.5 Dense Reference Models

Each setting has one dense **reference model**, the starting point of every pruning method, trained in Phase 2 under a protocol committed before any training run: Adam, learning rate 10^{-3}, $\beta = (0.9, 0.999)$, $\varepsilon = 10^{-8}$, no weight decay; cosine annealing to zero over 30 epochs, stepped per epoch; batch 128, cross-entropy loss and the augmentation of Section 4.2; seeds 0, 1 and 2, where the seed sets the initial weights and a generator with seed + 10^6 sets data order and augmentation. The epoch with the highest full-validation accuracy is kept and hashed, and only then is the test set read, once. The seed-0 model is the reference (seeds 1 and 2 only measure variability), except the disclosed CIFAR-10 SimpleCNN historical checkpoint (Adam at 10^{-3} for 10 epochs, final epoch); epoch selection on the full validation set affects every method of a setting identically. Later

phases assert the reference hash (Table 4.4), and Phases 6, 7A and 7B add a dense model fine-tuned with the pruned models' protocol, separating recovery from extra training.

Table 4.4: Dense reference models (accuracy, %). The VAL-RL columns are the denominators A_b of retained accuracy in Phases 3–4 (configuration I) and Phase 5 (configuration II).

Setting	Checkpoint	SHA-256	Full validation	Test	VAL-RL (I)	VAL-RL (II)
CIFAR-10 SimpleCNN	historical, 10 epochs	`ca28f834`	77.32	77.03	76.9667	76.88
CIFAR-10 LeNet-5	seed 0	`5ca6e0fc`	64.32	64.96	64.3000	64.20
CIFAR-10 ResNet-8	seed 0	`41fc5ee5`	78.72	78.69	78.5333	78.92
CIFAR-100 SimpleCNN	seed 0	`d75dd19c`	49.08	50.50	50.3667	48.96
CIFAR-100 LeNet-5	seed 0	`8e1b1aee`	32.58	32.28	33.3000	32.76
CIFAR-100 ResNet-8	seed 0	`8b84bfad`	43.48	43.99	43.7667	43.76

The extension starts from the CIFAR-10 SimpleCNN and ResNet-8 references.

4.6 Pruning Units and Policy Spaces

4.6.1 Masked pruning units (core framework)

The four units per architecture are those of Tables 3.2 and 4.3, with ResNet-8's three **residual-stage weight groups** (stage 1: the two block convolutions; stages 2 and 3: plus the 1×1 projection). An action applies its ratio as **L1 unstructured** pruning to *each* tensor of the unit; a ResNet-8 stage action removes no stage, block or channel. For the exploration endpoints, a policy is **destructive** if it prunes the setting's most sensitive unit (highest raw sensitivity, Equation 3.8; ties to the earliest unit) by 40% or more, a rule fixed before any PPO run.

4.6.2 Physically compacted structured units (deployment extension)

A ratio removes whole channels or neurons (Equation 3.17) and the network is rebuilt with smaller layers. In **SimpleCNN** (C = 32, 64, 128, 256), conv1–conv3 remove output filters and the next layer the matching inputs (16 flattened columns per channel after conv3); fc1 removes hidden neurons. In **ResNet-8** (C = 16, 16, 32, 64) the **stem is a coupled residual pruning group**: one shared channel set, ranked by the summed L1 norms of the two writing filters, cuts the stage-1 residual stream (its writers, identity shortcut and readers). **Stages 1–3 are block-internal channel groups** (the outputs of the

block's first convolution and the matching inputs of its second); the stage-2 and stage-3 output widths and the output layer are not pruned. The masked and structured ResNet-8 units are therefore different objects, so a masked ResNet-8 policy maps onto structured units only approximately, a recorded Phase-7A limitation; for SimpleCNN the mapping is direct. Every compacted model is checked: no masks remain, parameter and MAC counts equal the analytic values, logits agree with the masked-equivalent model within 10^{-4}, and outputs are identical after reloading.

4.6.3 Policy spaces and enumeration

Every setting has $6^4 = 1{,}296$ policies; a PPO run samples 512 episodes, so it evaluates at most 512 (about 40% of the space). **Exhaustive landscapes** of masked policies without fine-tuning exist on both splits of each configuration; they cache PPO's terminal evaluations, define each policy's rank, and are never computed on the test set. Paper 3 timed all 1,296 structured shapes of SimpleCNN and ResNet-8 with 10 and 100 output classes (four tables, 5,184 shapes), and Phase 7B recovered all 1,296 structured SimpleCNN policies with a 2-epoch protocol as a proxy cache. A planned exhaustive 5-epoch Paper 3 landscape (5,184 policies) was stopped for compute after 126 CIFAR-10 SimpleCNN policies, used only as pilot data.

4.7 Baseline Pruning Methods

4.7.1 Core-framework baselines

The baselines are one-shot methods from `src/pruning`, each calibrated to an exact number K of zeroed prunable weights, with largest-remainder rounding of per-tensor counts; all are deterministic except Random:

(a) **Uniform:** every prunable tensor loses the fraction K/N of its smallest-magnitude weights (N prunable weights).

(b) **Random:** uniform allocation with weights removed at random inside each tensor; ten mask seeds (1000–1009), summarised by their mean accuracy.

(c) **Global magnitude:** the K smallest-magnitude weights over all prunable tensors, a standard reference point in pruning evaluations (Blalock et al., 2020).

(d) **ERK** (Evci et al., 2020): tensor t receives density $d_t = \min(1, \varepsilon \cdot (\text{sum of its dimensions})/n_t)$, with ε solved so that $N - K$ weights are kept (saturated tensors fixed dense, ε re-solved), removing the smallest-magnitude weights within each tensor.

(e) **LAMP** (Lee et al., 2021): each weight receives the score $w_(u)^2 / \Sigma_{v \geq u} w_(v)^2$ within its magnitude-sorted tensor, and the K smallest scores network-wide are removed; every tensor keeps its largest weight.

4.7.2 Deployment-extension comparators

Uniform-Structured uses one common channel ratio for all four units and **LAMP-Structured** a LAMP allocation translated into per-unit structured ratios. In Phase 7B, **old PPO** is the frozen Phase-7A transfer of the Phase-5 policy and **new PPO** the policy learned by the deployment-oriented structured PPO.

4.7.3 Matching rules

In the core framework, "matched" means an **identical zero count** (identical sparsity ρ); in the extension, MACs or parameter counts are matched approximately (Table 4.5).

Table 4.5: Baseline methods and matching rules.

Comparison	Methods	Matching rule	Exactness
Phase 3	P0 vs Uniform, Random, Global, ERK, LAMP	zero count of each PPO run's final model; PPO chooses its own sparsity, so there is no common target	exact, asserted
Phase 3, descriptive	the five baselines	fixed sparsity grid 10–60%	curves only; never treated as matched
Phase 4	F, K vs LAMP, Global; F, K vs Uniform, ERK, Random	zero count of each distinct selected model	exact
Phase 5	C0, C1, C2 vs LAMP, Global	zero count of each selected policy	exact (mismatch 0)
Phase 6	PPO vs LAMP, Global, dense control	zero count per (setting, policy seed)	exact (mismatch 0, 120/120 triples)
Phase 7A	PPO vs Uniform-Structured	**MACs**: common ratio on a 0.001 grid minimising $\|MACs - MACs_{PPO}\|$ (ties: smaller ratio)	approximate; residual gap recorded
Phase 7A	PPO vs LAMP-Structured	Phase-6 LAMP mask at PPO's zero count; per-unit sparsity realised as structured ratios	approximate; not MAC-matched
Phase 7B	new PPO vs Uniform-Structured	**MACs**, 0.001 grid over [0, 0.9]	approximate
Phase 7B	new PPO vs LAMP-Structured	**parameter count**: LAMP sparsity on a 0.1% grid over [0, 90]%, realised structurally; closest parameter count (ties: smaller sparsity)	approximate
Phase 7B	new PPO vs old PPO	none (frozen Phase-7A checkpoint)	not matched

4.8 PPO Training Configuration

Every agent uses the PPO implementation (Schulman et al., 2017) of Stable-Baselines3 2.8.0 (Raffin et al., 2021), with generalised advantage estimation (Schulman et al., 2016); Table 4.6 gives the hyper-parameters, identical in Phases 3, 4, 5 and 7B, and what differs by phase (Phases 3–6 spread the single-thread runs over four worker processes). The evaluation data loader is still created at each cached terminal step, so the random stream matches an uncached run.

Table 4.6: PPO configuration by phase.

Item	Phase 3	Phase 4	Phase 5	Phase 7B
Actor and critic; optimiser; n_{steps}; batch; epochs	MLP [64, 64] tanh, orthogonal initialisation; Adam, lr 3×10^{-4}; 64; 32; 10	same	same	same
γ; GAE λ; clip; entropy; value; grad norm	0.99; 0.95; 0.2; 0.01; 0.5; 0.5	same	same	same
Budget; threads	2,048 steps = 512 episodes; 1	same	same	same
Environment; state	masked; 7 features, sensitivity slot 0	same	same	structured SimpleCNN; 8 features with centred sensitivity
Prior	P0–P3, uncentred, β = 0.5	P0–P5; P2 centred, β = 1.0	P2 (C0, C1); none (C2)	centred structured, β = 1.0
Reward	R0	R0, R1a, R1b, R2	R0 (C0); R_c, τ = 0.98 (C1, C2)	R_{7B}, α = 4
Evaluation cache	VAL-RL (I) landscape	same	VAL-RL (II) landscape	2-epoch proxy landscape
Extra loss	none	elite behavioural cloning in S2 (K = 5, η = 0.1)	none	none
Returned policy	terminal	terminal (S0, S2) or archive (S1, S2A)	archive	feasible archive
Seeds	300–319	400–419	500–519	900–904 (development); 700–719 (confirmatory)

The Phase 3–5 seeds were unused by any earlier PPO run (except an unrecorded 64-timestep smoke test with seed 300 in the Phase-2 checks) and shared across conditions and settings, so comparisons are paired; no seed was removed, replaced or added. Elite behavioural cloning adds $\eta \cdot \mathcal{L}_{\text{elite}}$ (Equation 3.13) to a copy of the SB3 PPO update, bit-identical to standard PPO at $\eta = 0$.

4.9 Experimental Conditions by Phase

Table 4.7 summarises each phase; the subsections give only what the table cannot.

Table 4.7: Phase-by-phase experimental design.

Phase	Research purpose	Datasets	Architectures	Conditions	Seeds	Runs	Primary output
3	PPO vs baselines; sensitivity prior	C10, C100	all three	P0, P1, P2, P3	300–319	480	test accuracy vs matched baselines; exploration endpoints
4	reward, centred prior, retention	C10, C100	all three	Stage A: 4 rewards; B: 6 priors; C: 4 search variants	400–419	1,200	stage selections; search quality; final pipeline vs control and LAMP
5	accuracy-constrained reward	C10, C100	all three	C0, C1, C2	500–519	360	constraint, search and accuracy endpoints
6	equal fixed-mask recovery	C10, C100	all three	PPO, LAMP, global, dense control	fine-tuning 600–619	480 (+6 repeats)	accuracy before and after equal fine-tuning
Paper 3	structured CPU latency	C10, C100 (latency); C10 (accuracy)	SimpleCNN, ResNet-8	1,296 shapes per table	recovery seed 600	5,184 shapes	latency tables; GO/NO-GO for hardware-aware PPO
7A	structured transfer, measured energy	C10	SimpleCNN, ResNet-8	Dense, PPO-, Uniform-, LAMP-Structured	recovery seed 600	8 models; 5 sessions per architecture	accuracy, memory, MACs, latency, energy
7B	deployment-oriented structured PPO	C10	SimpleCNN	new PPO; Dense, Uniform-, LAMP-Structured, old PPO	900–904; 700–719	20 searches; 11 recovered models	accuracy, memory, MACs; planned latency and energy
7B energy confirmation	direct energy of frozen 7B models	C10	SimpleCNN	Dense vs new PPO	—	6 qualification blocks; 10 sessions planned	qualification; planned paired energy

4.9.1 Phase 3

Phase 3 compared P0 (no prior) and P1 ($c_l = 0.5 \cdot S_l$) with two controls of the same strength: P2, S permuted per seed so that every unit receives another unit's value, and P3, the mean of S for every unit. It returned the terminal policy, pruned without fine-tuning, and posed four separate questions: (A) PPO vs uniform; (B) PPO vs global magnitude, LAMP, ERK and random; (C) whether the correct prior improves exploration (primary endpoint: VAL-RL accuracy AUC); and (D) whether it improves the final model (test accuracy in excess of matched global magnitude, P1 − P0). "P2" is the *shuffled* prior in Phase 3 and the *centred* prior in Phases 4–5.

4.9.2 Phase 4

Phase 4 used a staged design with mechanical, validation-only selections:

(a) **Stage A, reward.** R0, R1a, R1b and R2 (Section 4.12), with P0 and the terminal policy. A redesigned reward qualified only if it at least halved the cross-setting spread of final sparsity (with a seed-bootstrap upper bound below 1) **and** kept VAL-RL retention ≥ 0.95 in at least five settings; otherwise R0 was kept.

(b) **Stage B, prior.** Six priors under the Stage-A reward: P0, none; P1, $c_l = 0.5 \cdot S_l$; P2, $c_l = 1.0 \cdot (S_l - \bar{S})$ (centred); P3, P2 permuted across units; P4, c = 1.0·mean_l $|S_l - \bar{S}|$ for every unit (magnitude-matched pure conservatism); P5, $c_l = -1.0 \cdot (S_l - \bar{S})$ (sign-reversed). A prior beating P0 on reward-rank AUC in at least three settings with none opposite was eligible; the eligible prior supported in more settings was carried forward (tie: larger mean d_z), otherwise P0.

(c) **Stage C, search. S0**, terminal policy; **S1**, archive rule on the S0 runs; **S2** and **S2A**, elite behavioural cloning with the terminal policy or the archive rule; the winner had the lowest mean median VAL-SELECT rank of the selected policy.

The selections kept R0, chose P2 and S2A, defining the final pipeline **F = (R0, P2, S2A)** and the control **K = (R0, P0, S0)**, the Phase-3 configuration on fresh seeds.

4.9.3 Phase 5

Phase 5 used configuration II and three conditions fixed in advance: **C0**, R0 + P2 + archive, selecting by the highest R0 score on VAL-SELECT (the Phase-4 pipeline without behavioural cloning); **C1 (proposed)**, R_c with $\tau = 0.98$, P2 and constraint-aware selection (Equation 3.14); and **C2**, as C1 without a prior. C1 − C2 is the sensitivity ablation and C1 − C0 the reward comparison. τ was chosen before any run, on the VAL-RL landscapes only, as the largest value in {0.95, 0.97, 0.98, 0.99} giving every setting at least 20 feasible policies, including one with ≥ 10% sparsity: $\tau = 0.98$.

4.9.4 Phase 6

Phase 6 tests whether accuracy differences persist under the same fine-tuning budget at exactly the same sparsity: **PPO** (the frozen Phase-5 C1 policy of each setting and policy seed), **LAMP** and **global magnitude** at PPO's exact zero count, and a **dense control**. **Phase 6 contains no uniform, ERK or random method, and no Phase-5 C0 or C2 policy** (excluded by the pre-registration to fit the compute budget). The 360 pruned runs and 120 dense controls of each (setting, policy seed) share the fine-tuning seed (policy seed + 100) and data stream, checked by hash, and six reproduction runs repeat seed 500's PPO run in each setting. The unit of analysis is the policy seed (n = 20 per setting), with a cluster-level sensitivity analysis over distinct policies where there are at least six.

4.9.5 Paper 3

Paper 3 tested whether structured compression becomes CPU latency: **H0** calibrated the primary timing configuration (Section 4.17); **H1** ran 50 checks of structured compaction, MAC and FLOP counters and structured sensitivity, with the Phase-2 library checks rerun on the laptop; and **H2-A** built the four latency tables. A 1-epoch recovery proxy was validated against 5-epoch recovery on frozen CIFAR-10 samples (156 SimpleCNN and 31 ResNet-8 policy pairs); an architecture could proceed only if the Spearman correlation was ≥ 0.90. The Youden-J (Youden, 1950) threshold rule needed at least three truly feasible validation pairs; ResNet-8 had fewer, triggering the pre-registered fallback to a decision by the principal investigator, and a pre-committed calibration on 48 frozen ResNet-8 policies set the threshold by a recall-first rule with a 0.005 margin. A latency-room rule then required at least 20 proxy-feasible policies and one with a speed-up ≥

1.10×. SimpleCNN failed the proxy-validity criterion and ResNet-8 the latency-room rule (Chapter 6), so **no hardware-aware reinforcement learning was run**.

4.9.6 Phase 7A

Phase 7A compares **Dense**; **PPO-Structured**, the most frequent Phase-5 C1 selected policy (ties: smaller action vector), read as structured ratios; **Uniform-Structured**, MAC-matched to PPO; and **LAMP-Structured**, from the Phase-6 LAMP mask, all recovered equally and measured in five energy sessions (Section 4.18). The fixed criteria are **accuracy**, $|A_{method} - A_{dense,recovered}| \leq 0.5$ pp on VAL-RL, and **consistent energy reduction**, lower energy than the same-session dense model in 5 of 5 sessions **and** a median saving above a noise threshold of twice the dense model's between-session coefficient of variation. **Case A** (both hold) is the primary success; **B** is consistent reduction with more than 0.5 pp loss; **C** accuracy within 0.5 pp without consistent reduction; **D** a baseline with the better trade-off, i.e. accuracy within 0.5 pp of the other method and lower energy in 5 of 5 sessions, or accuracy higher by more than 0.5 pp with median energy not higher.

4.9.7 Phase 7B

Phase 7B uses SimpleCNN on CIFAR-10 only. Its deployment-oriented structured PPO (Section 3.16.3) was rewarded with the *surrogate (predicted)* energy saving of Equation (3.18); no measured energy entered the search. **Stage A (development)** validated the proxy and surrogate (Section 4.18.3), built the 2-epoch proxy landscape and tuned $\alpha \in \{0.5, 1, 2, 4\}$ on the development seeds by the lowest median predicted energy of the selected policies (ties: smaller α), giving $\alpha = 4$; $\beta = 1.0$, $\kappa = 10$, $\varepsilon = 0.01$, the 1% tie band and the selection rule were fixed, not tuned. **Stage B (confirmatory)** ran 20 seeds, all reported, with structured sensitivity on VAL-RL (probes at {0.1, 0.2, 0.4, 0.6}) and $\hat{\tau} = 0.98849$, whose target was 5-epoch VAL-RL accuracy within 0.5 pp of the recovered dense model. All final models receive the equal 5-epoch recovery; old PPO is reused read only.

Joint deployment-efficiency criterion (named *GREEN-SUCCESS* in the pre-registration). A model succeeds only if all five conditions hold at once: (1) VAL-SELECT accuracy at least the recovered dense VAL-SELECT accuracy minus 0.5 pp; (2)

parameter-memory reduction > 0; (3) MAC reduction > 0; (4) latency reduced, with the median below dense and the model faster in 5 of 5 sessions; and (5) measured energy reduced beyond noise, lower in 5 of 5 sessions with a median saving above twice the dense model's between-session coefficient of variation. There is no override, and an empty archive counts as failure.

4.9.8 Phase 7B energy confirmation

This protocol, written after Phase 7B had closed, does not alter its verdicts. After hash verification and qualification of the instrument on the dense model (Section 4.18.5), ten paired Dense and new-PPO sessions would run; confirmation requires that the setup qualified, the PPO model used less energy in at least 8 of the 10 sessions, the median saving exceeded the noise threshold, and a leave-one-session-out check held for every omitted session.

4.10 Sensitivity Conditions and Controls

Sensitivity enters the core framework only through the prior (state slot 0). **Layer-specific information** is tested by priors that scramble the unit assignment of the same pressure (shuffled P2 in Phase 3; permuted P3 in Phase 4); **generic conservatism** by pressure without sensitivity information (constant P3 in Phase 3; magnitude-matched P4 in Phase 4); and **direction** by the sign-reversed P5. The sensitivity-specific statement held in a setting only if the centred P2 beat **both** P3 and P4 there. Phase 5 compares C1 with C2; Phase 7B ran no prior-free control. The vectors use configuration-I VAL-RL (Phases 3–4) or configuration-II VAL-RL (Phase 5; Phase 7B with structured probes), never VAL-SELECT or test data.

4.11 Archive and Selection Protocols

Phase 3 returns the terminal policy; in **Phase 4**, S0 and S2 return the terminal policy and S1 and S2A apply Equation (3.12). **Phase 5**: the archive of all 360 runs was frozen and hashed before any VAL-SELECT evaluation, each candidate's VAL-SELECT result had to equal the landscape, C1 and C2 selected by Equation (3.14) and C0 by the highest R0 score on VAL-SELECT, and the final-policy list was committed before the test set was read.

In **Phase 7B** the archive holds every distinct proxy-feasible policy visited. Per seed, the lowest predicted energy is selected; policies within 1% count as tied, and ties go to higher proxy accuracy, then fewer MACs, then lower parameter memory, then the smaller action vector. The primary new-PPO model is the most frequent selection over the 20 seeds; the same rule over all proxy-feasible landscape policies gives an enumeration reference. VAL-SELECT plays no part in Phase-7B selection.

4.12 Reward Configurations

All rewards are computed on VAL-RL from the pruned network without fine-tuning; Phase 7B uses its 2-epoch proxy.

Table 4.8: Reward configurations.

Reward	Definition	Used in
R0	$1.5 \cdot A_p/A_b + 0.01 \cdot \rho(\%) + 0.05 \cdot D$ (Equation 3.7)	Phases 3 and 4; Phase 5 C0
R1a	$1.5 \cdot A_p/A_b - \kappa_a \cdot \lvert\rho - 50\rvert + 0.05 \cdot D$	Phase 4 Stage A
R1b	$1.5 \cdot A_p/A_b - \kappa_b \cdot \lvert\rho - \rho^*_s\rvert + 0.05 \cdot D$, with ρ^*_s the largest sparsity retaining ≥ 95% of VAL-RL accuracy	Phase 4 Stage A
R2	$1.5 \cdot (A_p - A_60)/(A_b - A_{60}) + 1.5 \cdot \rho/\rho_60 + 0.05 \cdot D$, anchored on the maximal policy [5,5,5,5]	Phase 4 Stage A
R_c	constraint-first, $\tau = 0.98$ (Equation 3.15)	Phase 5 C1, C2
R_{7B}	constraint-first on the proxy, predicted energy saving, $\alpha = 4$ (Equation 3.19)	Phase 7B

κ_a and κ_b were set before any run as the smallest value in {0.005, 0.01, 0.02, 0.04, 0.08} whose VAL-RL optimum lies within ±5 pp of its target in all six settings.

4.13 Recovery and Fine-Tuning Protocol

The Phase-6 protocol (Section 3.15) is used unchanged in every later phase: Adam at 10^{-4}, $\beta = (0.9, 0.999)$, $\varepsilon = 10^{-8}$, weight decay 0, a new optimiser per run; cosine annealing to 0, stepped per epoch; batch 128 on the 45,000 training images, with crop and flip; one thread; no checkpoint selection. **E = 5** was fixed before any confirmatory run by a timing rule (E = 5 if the projected Phase-6 wall time was at most 30 hours, otherwise E = 3); the projection, from a pilot that used no accuracy, was 29.44 hours.

In Phase 6 the mask is fixed by PyTorch's pruning reparameterisation (Paszke et al., 2019), with the mask hash, the absence of non-zero masked weights and per-layer zero

counts asserted after every epoch. Compacted models (Paper 3, 7A, 7B) use recovery seed 600 with data generator $600 + 10^6$, so all models of a phase see the same minibatch sequence. The Paper-3 1-epoch proxy is the first epoch of the 5-epoch protocol, the Phase-7B 2-epoch proxy a separate two-epoch cosine run; final accuracy claims always use 5-epoch models.

4.14 Evaluation Metrics

Table 4.9 lists the metrics; accuracy always names its split, and every reduction its reference. Masked sparsity ρ, the channel removal ratio r_l, physical parameter reduction (over all parameters) and MAC reduction are kept apart.

Table 4.9: Evaluation metrics. ↑ higher is better; ↓ lower is better (in the order of the metrics listed).

Group	Metrics	Definitions	Phases	Better
Predictive	accuracy A; retained accuracy q; difference Δ	top-1 accuracy (%) on a named split; A / A_{ref}, with A_{ref} the dense or recovered dense model on the same split; A − A_{ref}, or method minus baseline (pp)	all; 3–7B; all	↑
	recovery; gap change ΔG; retention gap	$A_{post} - A_{pre}$; ($A_{PPO} - A_{base}$) after minus before fine-tuning; VAL-SELECT retention minus test retention	6	↑; ↑; ↓
	proxy accuracy $\hat{q}$	A_E / $A_{E,dense}$ on VAL-RL after an E-epoch recovery (E = 1 in Paper 3, 2 in 7B)	Paper 3, 7B	↑
Compression	masked sparsity ρ; parameter reduction	Equation 3.2; 1 − P/P_dense	3–6; Paper 3, 7A, 7B	context; ↑
	parameter memory; serialised size	Σ numel × element size (bytes, float32); saved state-dict bytes	7B; 6, 7A, 7B	↓
Computation	MACs; FLOPs	Conv2d/Linear multiply–accumulates per image, closed form checked against hooks; FLOPs = 2 × MACs	Paper 3, 7A, 7B	↓
Deployment	latency T; speed-up; latency gain g_T	median per-inference batch-1 time (ms); T_{ref} / T; 1 − T/T_identity within one table	Paper 3, 7A, 7B-EC; Paper 3, 7A	↓; ↑; ↑
Energy	E; energy per inference; energy saving	package J / N × 1,000 for a block of N inferences; package J / N; 100·(1 − E/E_dense), same-session dense model	7A, 7B-EC	↓; ↓; ↑
	EDP; predicted energy $\hat{E}$	(J/inference × latency) normalised to the same-session dense model, not a carbon metric; Equation 3.18, inside the search only, never a result	7A, 7B-EC; 7B	↓; —
Search	reward rank; rank score; constrained rank	rank among the 1,296 landscape policies (1 = best; ties get the minimum rank); (1,296 − rank)/1,295; rank under R_c on the VAL-RL landscape (common yardstick for C0–C2)	3–5; 3–5; 5	↓; ↑; ↓
	best-seen and selected rank; top-k reach; first entry	minimum rank over the 512 sampled policies and rank of the returned policy; best-seen or selected policy within the top k (100, 50, 20, 10, 5, optimum); first episode reaching it (censored at 513)	4, 5	↓; ↑; ↓

Group	Metrics	Definitions	Phases	Better
	trajectory AUC; destructive frequency; frontier regret; feasible-sample rate	trapezoidal integral over episodes of the sampled policy's rank score, VAL-RL accuracy or utility, normalised by the episode span; share of sampled policies that are destructive (Section 4.6.1); best accuracy at equal or higher sparsity minus own accuracy; share of samples with $q \geq \tau$	3–5; 3–5; 4; 5	↑; ↓; ↓; ↑

4.15 Statistical Analysis and Decision Criteria

4.15.1 Paired tests (Phases 3–6)

Comparisons are paired by seed (n = 20 per setting) with the **exact two-sided sign-flip test** over all 2^{20} sign patterns as primary test, reporting the mean, standard deviation and median, the 95% t-interval (df 19), a bootstrap 95% interval (10,000 resamples in Phase 3), d_z, the exact Wilcoxon signed-rank p (Wilcoxon, 1945) (Phases 3 and 4), and wins, ties and losses. Holm's step-down correction (Holm, 1979) is applied within each pre-registered family. A comparison is **supported** with the hypothesised direction, a Holm-adjusted $p < 0.05$ **and** a 95% t-interval excluding 0; **opposite** under the same criteria in reverse; otherwise **inconclusive**. Across settings (Phases 3–4) a statement is **STRONG** if supported in ≥ 5 of 6 settings and opposite in none, **MODERATE** if supported in 3–4 and opposite in none, **LIMITED** if supported in 1–2 (or ≥ 3 with one opposite), and **NOT SUPPORTED** otherwise; opposite in ≥ 3 settings is also labelled REVERSED. No subset result is described as general.

Families. Phase 3: A (6), B (24), C (6), C-specificity (12), one per C-secondary endpoint, and D (6), with McNemar counts supportive only. Phase 4: reward, prior, search and final-model families, a seed bootstrap for Stage A, and a **one-sided non-inferiority test** of F against LAMP with a 0.5 pp margin (a Holm-corrected sign-flip test on d + 0.5, plus a lower t-bound above −0.5). Phase 5: C1 − C0 on test accuracy (6); C1 − C2 on four primary exploration endpoints (24) and secondary endpoints (18); C1 − LAMP and its gap change (6 each); C1 − global magnitude **descriptive**. Phase 6: recovery families; PPO vs LAMP and PPO vs global (12 each); the retention gap; and an unpaired permutation

test (100,000 permutations) with a Welch interval for architecture dependence, plus a seed-paired version.

4.15.2 The ±0.5 pp rule of Phase 6

Phase 6 reports statistical support and practical size separately, with a fixed 0.5 pp margin: a difference is *practically relevant* only if |mean| ≥ 0.5 pp; two methods lie **within the ±0.5 pp margin** if the 95% t-interval of their paired difference lies inside (−0.5, +0.5) pp; and a supported difference below 0.5 pp is *statistically detectable but small*. This is a **pre-registered confidence-interval inclusion rule**, not a formal statistical equivalence test (none, such as two one-sided tests, was pre-registered); the pre-registration's phrase "practically equivalent" is therefore written "within the ±0.5 pp margin".

4.15.3 Deployment-extension statistics

Paper 3 is descriptive and rank-based: Spearman correlations of latency gain with parameter and MAC reduction over all 1,296 shapes, the fraction of policy pairs in which the policy with more parameter reduction is slower, and the H0 signal-to-noise rule; proxy validity uses Spearman correlation, error, bias, top-10 overlap and a confusion matrix. **Phases 7A and 7B** summarise sessions by median, mean, standard deviation and interquartile range, and test each method against dense with an **exact one-sided Wilcoxon signed-rank test** on five paired sessions, whose minimum attainable p of 0.031 makes p-values secondary to the 5-of-5 rule; the **energy confirmation** pre-registered an exact Wilcoxon and a sign test on ten pairs.

4.15.4 Pre-registered decision rules

Table 4.10: Pre-registered decision criteria. The commit is the one that first added the governing file.

Experiment	Frozen rule	Purpose	Source (commit)
Phase 2	seed-0 reference; best full-validation epoch; test once after hashing	dense references	`cpu_dense_training_protocol.md` (`366d21ae`)
Phase 3	outcome and class rules; four separate questions	PPO vs baselines; prior	`phase3_preregistration.md` (`2dbcc770`)
Phase 4	mechanical Stage A–C selections; criteria A–E (reward, specific prior, search, final model, LAMP)	reward, prior, retention	`phase4_preregistration.md` (`f52e87d9`)
Phase 5	τ rule; criteria A–F (accuracy safety, compression, search, sensitivity, final model, LAMP)	constrained reward	`phase5_preregistration.md` (`3cbaec1a`)
Phase 6	E timing rule; criteria A–F (fairness, recovery, competitiveness, generalisation, sparsity preservation, reproducibility); ±0.5 pp rule	equal recovery	`phase6_preregistration.md` (`b1b5396f`)
Paper 3 H0	SNR ≥ 5, reduction ≥ 10%, positive in every session; order A, C, B	primary latency configuration	`h0_protocol.md` (`c2dcb22a`)
Paper 3 H2, proxy	table protocol; proxy GO iff Spearman ≥ 0.90; Youden threshold with fallback	latency tables; proxy	`paper3_preregistration.md` (`45936846`); `paper3_proxy_protocol.md` (`ec7a7f27`)
Paper 3 ResNet-8	≥ 5 feasible calibration policies; threshold rule; latency room (≥ 20 feasible, one ≥ 1.10×)	GO/NO-GO for hardware-aware PPO	`resnet8_threshold_calibration_protocol.md` (`5b127804`)
Phase 7A	±0.5 pp on VAL-RL; consistent energy reduction; Cases A–D; pilot CV ≤ 3%	transfer and energy	`preregistration.json` (`b8c27665`)
Phase 7B Stage A	proxy acceptance; surrogate acceptance; calibration pilot CV ≤ 4%; α rule	development	`phase7b_design.md` (`f5abe039`)
Phase 7B Stage B	joint deployment-efficiency criterion; comparison rule; pilot CV ≤ 3%, N doubled once, then STOP	confirmatory	`phase7b_final_preregistration.json` (`0e101d55`)
7B energy confirmation	qualification CV ≤ 3%, one attempt; energy confirmation rule	direct energy	`protocol.json` (`df233be0`)

4.16 Hardware and Software Environment

All experiments ran on CPUs; no GPU was used (Table 4.11).

Table 4.11: Hardware and software environments.

Item	Core framework (Phases 2–6)	Extension (Paper 3, 7A, 7B, energy confirmation)
CPU	Intel Core i5-6400 @ 2.70 GHz, 4 logical CPUs (desktop)	Intel Core i7-1260P (laptop): 4 performance cores × 2 threads (logical 0–7), 8 efficiency cores (8–15)
Memory	not recorded	15.7 GB
OS	Windows 11, build 22621	Windows 11 Home, build 26200
Python; PyTorch	3.11.9; 2.7.1+cpu (torchvision 0.22.1+cpu)	3.11.9; 2.7.1+cpu with oneDNN 3.7.1 and oneMKL 2025.1
Other libraries	Stable-Baselines3 2.8.0, sb3-contrib 2.8.0, Gymnasium 1.2.3, NumPy 2.2.2, pandas 2.2.3, ptflops 0.7.5, psutil 7.2.2	Stable-Baselines3 2.8.0 in the pinned environment; the 7B records do not store package versions separately
Threads	1 per run (dense training: 4)	1 intra-op, 1 inter-op
Power	mains	AC required; Balanced plan

Package versions are pinned in `requirements_locked.txt`, and every phase records its environment and commit. **Results do not transfer between machines:** no desktop latency or energy value is used.

4.17 Structured Pruning and Latency Protocol

Measurement core (Paper 3, reused in Phases 7A and 7B): batch 1, float32, NCHW, `eval()` and `inference_mode()` on one preallocated, seeded input; one intra-op and one inter-op thread, set before PyTorch is imported, with the process pinned to logical CPU 2 (a performance-core thread other than core 0); each pass timed with `time.perf_counter_ns()` with garbage collection disabled; AC power required, system load ≤ 15% over 5 s before and after each session, and no other experiment running.

H0 calibration. Six SimpleCNN shapes were timed under configurations **A** (batch 1, 1 thread on CPU 2, 50 / 300 warm-up / timed passes), **B** (batch 1, 4 threads, 50 / 300) and **C** (batch 128, 4 threads, 10 / 50) over five fresh-process sessions; the rule selected configuration A.

H2-A latency tables. Configuration A in five fresh-process sessions per table, each timing all 1,296 shapes once in a seeded random order, with a dense anchor block before the first shape and after every 25; a session whose anchors drift by more than 10% from their median is repeated once (four attempts were repeated; all 20 accepted sessions met

the criterion). A shape's latency is the median over sessions of its per-session median; the tables were hashed and committed before use. The absolute dense latency on the table days was about 16% higher than on the H0 calibration day (a recorded cross-occasion failure), so only relative within-table latency is analysed and later sessions compare against a same-session dense model. **Phase 7A latency** is the median per-inference time over the energy block's N inferences, with speed-up computed within a session.

4.18 Energy Measurement Protocol

4.18.1 Backend

Energy is read from the **Intel RAPL package counter** (Khan et al., 2018) through the **Windows Energy Meter Interface** (counter `\Energy Meter(RAPL_Package0_PKG)\Energy`): raw PDH values in picowatt-hours (1 pWh = 3.6×10^{-9} J), refreshed about every 1 ms, read before and after each block. The core (PP0) domain is secondary; the DRAM domain reads zero and is not used. No TDP model, estimate or surrogate replaces a measurement, and no external meter was available; because software energy counters need careful protocol design (Jay et al., 2023; Raffin and Trystram, 2025), gates and qualification rules are applied.

4.18.2 Phase 7A

Per model, 1,000 warm-up inferences precede one block of N batch-1 inferences (E = package J / N × 1,000), in five fresh-process sessions per architecture with the four models in a seeded random order after 30 s of idle-power measurement. A dense-only pilot of 2 processes × 3 blocks set N = 100,000 if the coefficient of variation of the six values was ≤ 3% for both architectures (otherwise N doubled once, then stop); the rule was met at N = 100,000.

4.18.3 Phase 7B energy surrogate

The surrogate's target was **directly measured RAPL energy** of 200 structured policies fixed with seed 20261028 (dense, the H0 policies, 160 policies stratified over eight MAC-reduction bins, and similar-MAC pairs separating MACs from shape), measured in three fresh-process sessions on CPU 2 (200 warm-up inferences, N = 20,000 per policy, a dense

anchor every 25 policies) and taking the three-session median; a dense pilot with a 4% limit met the limit at N = 20,000. Candidate predictors were tried in order, the first to pass being accepted: (1) MACs; (2) table latency T_{table}; (3) MACs, parameters and T_{table}; (4) a degree-2 ridge model in the normalised widths plus MACs, parameters and T_{table}. Acceptance required, on 5-fold cross-validated (seed 20261029) predictions, Spearman ≥ 0.95, a median absolute percentage error ≤ 5%, and the same error ≤ 5% within the lowest-energy quartile. Candidate 1 failed; candidate 2 was accepted (Equation 3.18). **Predicted energy is never reported as measured energy.**

4.18.4 Phase 7B confirmatory measurement and its stopping rule

Five fresh-process sessions were planned (30 s idle, 1,000 warm-up inferences, N = 100,000 per block, 5 s pauses; dense three times per session, every other model once), preceded by the Phase-7A pilot rule with a 3% limit (N doubled once, then STOP). The pilot gave a coefficient of variation of 11.36% at N = 100,000; the first doubled-N attempt was halted by the load gate and, by recorded user decision, declared void; the permitted rerun at N = 200,000 with the laptop idle gave 4.01%, above the limit. Hardware validation therefore **stopped**: no confirmatory session was run, so the latency and energy conditions of the joint criterion could not be assessed. **No Phase-7B energy reduction is reported.**

4.18.5 Energy confirmation study

A new protocol, committed before any qualification data, used a 180 s thermal soak, 2,000 warm-up inferences and N = 200,000 per block with 5 s pauses, a fresh child process per run, all models hash-checked, and package and core energy, latency, system load, CPU-2 performance, thermal-zone temperature and AC status recorded per block; a run is void and repeated if load exceeds 15% or the laptop leaves AC (never on energy or latency values). **Qualification** used the dense model only (3 fresh runs × 2 blocks; coefficient of variation of the six values ≤ 3%; one attempt, no relaxation). The coefficient of variation was 19.92%, so the setup was **not qualified**, and the study stopped **before any paired Dense-versus-PPO measurement**: energy confirmation is NO. The stop reflects the instability of the measurement setup; no comparison of the two models was made.

4.19 Reproducibility and Pre-Registration

Each protocol was committed before its confirmatory data (Table 4.10) and never edited (necessary changes were separate amendments committed before the affected runs). There was no interim peeking and no result-based exclusion; infrastructure failures were repeated with the same seed and logged. Integrity checks before any statistic covered run matrices, hashes, test access, freeze order, matching, thread counts and environments; splits, references, final-model lists, checkpoints and latency tables are stored with SHA-256 hashes, and six Phase-6 reproduction runs test bit-identical reproduction. The complete per-seed records and machine-readable result files are retained in the associated research repository.

Test-set governance. Code enforces Table 4.2: a guard raises on any test access during training, or before a phase's final-model list is frozen. In Phases 3–5 the test set was read once per distinct frozen model and matched baseline; in Phase 6 only for the 480 frozen checkpoints and the pre-fine-tuning masks; in Paper 3, 7A and 7B never. No design choice used test accuracy. Two disclosures: (1) **Phase 6 is not a fully independent confirmation**, because the test sets had been read in Phases 2–5 for earlier frozen models; (2) **validation-to-test transfer is measured, not assumed**, with each Phase-5 policy's feasibility recorded separately on VAL-RL, VAL-SELECT and test (q_{test} = test / dense test), and Phase 6 testing whether fine-tuning changes the VAL-SELECT–test retention gap.

4.20 Chapter Summary

The design fixes two validation configurations, a test set read only for frozen models, exact zero-count matching, one PPO configuration, one 5-epoch recovery protocol, pre-registered tests including a ±0.5 pp confidence-interval rule that is not a formal equivalence test, and measurement gates that stopped the Phase-7B measurements.

CHAPTER 5
RESULTS OF THE CORE FRAMEWORK

5.1 General

This chapter reports the core-framework results (Phases 3–6) and answers RQ1–RQ5; each section gives the question, the result with its statistics, a short interpretation and a claim boundary. Conventions follow Chapter 4: accuracies are percentages and differences percentage points (pp); "test" is the official 10,000-image test set; "VAL-RL (I)" and "VAL-SELECT (I)" are the 3,000 / 2,000 configuration of Phases 3–4, and "VAL-RL" and "VAL-SELECT" the 2,500 / 2,500 configuration of Phases 5–6. Comparisons are paired by seed (n = 20) with Holm-adjusted exact sign-flip p-values; **supported**, **opposite**, **inconclusive** and the classes STRONG to REVERSED are defined in Section 4.15.1. Baselines are built at each PPO run's *own* zero count; ρ is masked sparsity (Equation 3.2); C10/C100 denote CIFAR-10/100.

5.2 Dense Reference Performance

Table 5.1: Dense reference models (accuracy %, seed-0 reference).

Dataset	Architecture	Full validation	Test	Test, 3 seeds (mean ± SD)	VAL-RL (I)	VAL-RL (II)
CIFAR-10	SimpleCNN†	77.32	77.03	— (1 model)	76.97	76.88
CIFAR-10	LeNet-5	64.32	64.96	66.21 ± 1.51	64.30	64.20
CIFAR-10	ResNet-8	78.72	78.69	78.69 ± 0.51	78.53	78.92
CIFAR-100	SimpleCNN	49.08	50.50	49.84 ± 0.65	50.37	48.96
CIFAR-100	LeNet-5	32.58	32.28	31.42 ± 0.75	33.30	32.76
CIFAR-100	ResNet-8	43.48	43.99	43.58 ± 0.53	43.77	43.76

† The historical 10-epoch checkpoint, not the 30-epoch Phase-2 recipe (Section 4.5).

Table 5.1 gives the six dense references; test accuracy varies by at most about 1.5 pp (SD) across seeds, and the two VAL-RL columns are the denominators A_b of retained accuracy q in Phases 3–4 and Phase 5.

5.3 Phase 3 — PPO versus Baselines

Question (RQ1). Does a PPO agent learn layer-wise allocations that are more accurate than uniform allocation and other one-shot baselines at the same sparsity?

Evidence. Phase 3 ran 480 PPO runs (6 settings × 4 conditions × seeds 300–319). The P0 (no-prior) final policy of each run was compared on test accuracy with Uniform, Random, Global magnitude, ERK (Evci et al., 2020) and LAMP (Lee et al., 2021) (Section 4.7), each built at the run's exact zero count, without fine-tuning. PPO chose its own sparsity: the mean final P0 sparsity was 55.06% (C10 SimpleCNN), 51.54% (C10 LeNet-5), 28.35% (C10 ResNet-8), 53.28% (C100 SimpleCNN), 34.59% (C100 LeNet-5) and 9.46% (C100 ResNet-8), so Phase 3 spans 9.5–55.1% rather than one fixed sparsity.

Table 5.2: Phase 3: P0 minus baseline, test accuracy (pp, mean of 20 paired runs). "+" = supported, "−" = opposite, "·" = inconclusive.

Setting (P0 test)	Uniform	Random	Global	ERK	LAMP
C10 SimpleCNN (76.44)	+31.37 +	+59.60 +	−0.07 ·	+0.75 +	−0.55 −
C10 LeNet-5 (61.67)	+16.29 +	+48.69 +	+2.74 +	+1.93 +	−1.64 −
C10 ResNet-8 (76.95)	+14.17 +	+64.36 +	+7.38 +	−0.06 ·	+0.15 ·
C100 SimpleCNN (49.66)	+29.69 +	+47.29 +	−0.01 ·	+4.35 +	−0.65 −
C100 LeNet-5 (31.18)	+3.67 +	+27.89 +	−0.13 ·	+0.30 ·	−0.68 −
C100 ResNet-8 (43.01)	+0.23 ·	+38.76 +	+0.28 +	−0.80 −	−0.75 −
Supported / opposite (of 6)	**5 / 0**	**6 / 0**	**3 / 0**	**3 / 1**	**0 / 5**
Class	**STRONG**	**STRONG**	**MODERATE**	**LIMITED**	**NOT SUPPORTED, REVERSED**

Uniform: STRONG (5 of 6 settings; Holm $p = 1.1 \times 10^{-5}$ and 20 of 20 seeds won in each supported setting). The largest gain, **+31.37 pp** (exact mean 31.365 pp, ≈ +31.4 pp; 95% CI 26.57–36.16), is on **CIFAR-10 SimpleCNN, test accuracy**, where at 55.06% mean sparsity uniform pruning fell to 45.08% while PPO kept 76.44%; the maximum is specific to that setting. C100 ResNet-8 is inconclusive (+0.23 pp): PPO chose the uniform policy [1, 1, 1, 1] in 19 of 20 runs. **Random: STRONG** (6 of 6), +27.89 to +64.36 pp (Holm $p = 4.6 \times 10^{-5}$ in every setting). **Global: MODERATE** (better in 3 settings, Holm $p \leq 0.0015$; within ±0.13 pp, inconclusive, in the other three). **ERK: LIMITED** (better in 3, Holm $p \leq 0.0087$; inconclusive in 2; worse in C100 ResNet-8, −0.80 pp, Holm $p = 5.3 \times 10^{-5}$). **LAMP** is significantly more accurate than PPO in **5 of 6 settings**, by 0.55–1.64 pp (Holm $p \leq 0.002$; LAMP won 19–20 of 20 seeds in four of them), and C10 ResNet-8 is inconclusive (+0.15 pp): "PPO > LAMP" is NOT SUPPORTED and, being opposite in five settings, REVERSED.

Allocations. PPO's final policies are non-uniform (Figure 5.2). On SimpleCNN it prunes conv3 and the large hidden layer at 40–60% and keeps the first convolution at 0–20%; on LeNet-5 the two fully connected layers absorb most of the pruning; on C10 ResNet-8 stage 3 is pruned most. Uniform must apply the same ratio to the small, sensitive first layers.

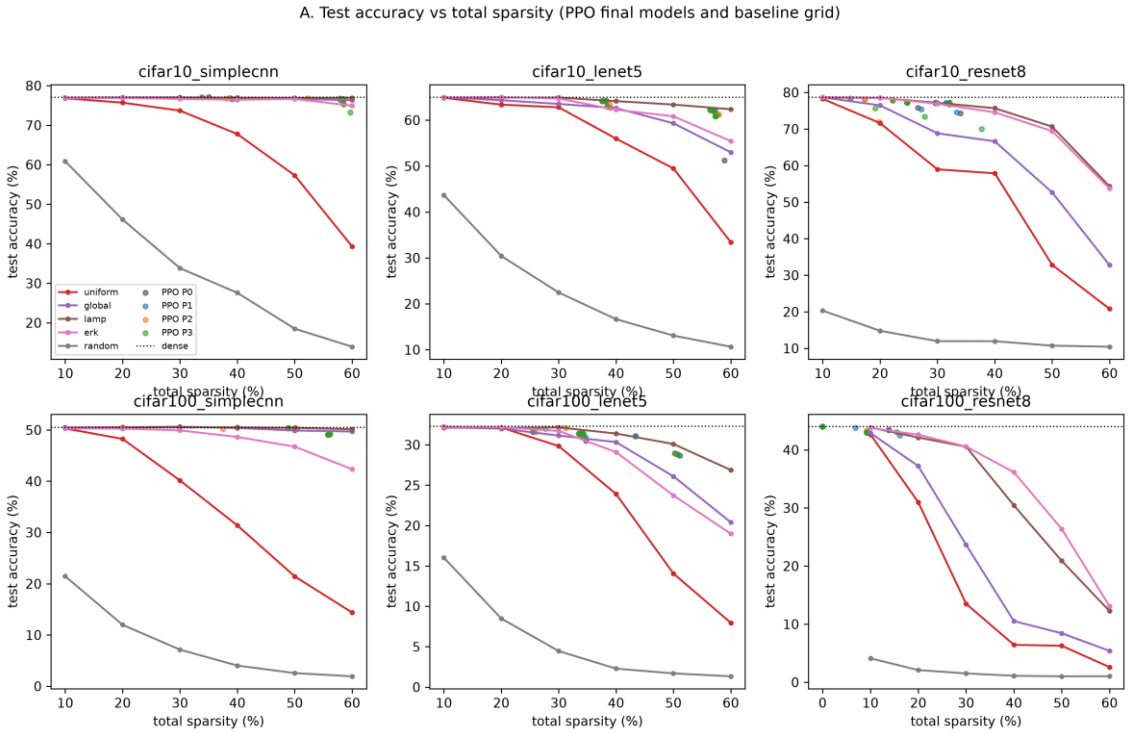


Fig. 5.1: Test accuracy against total sparsity (Phase 3). Lines: the five baselines on a fixed descriptive sparsity grid (10–60%); points: the final models of the 20 PPO runs per condition (P0–P3); dotted line: dense test accuracy. Table 5.2 uses baselines built at each PPO run's own zero count.

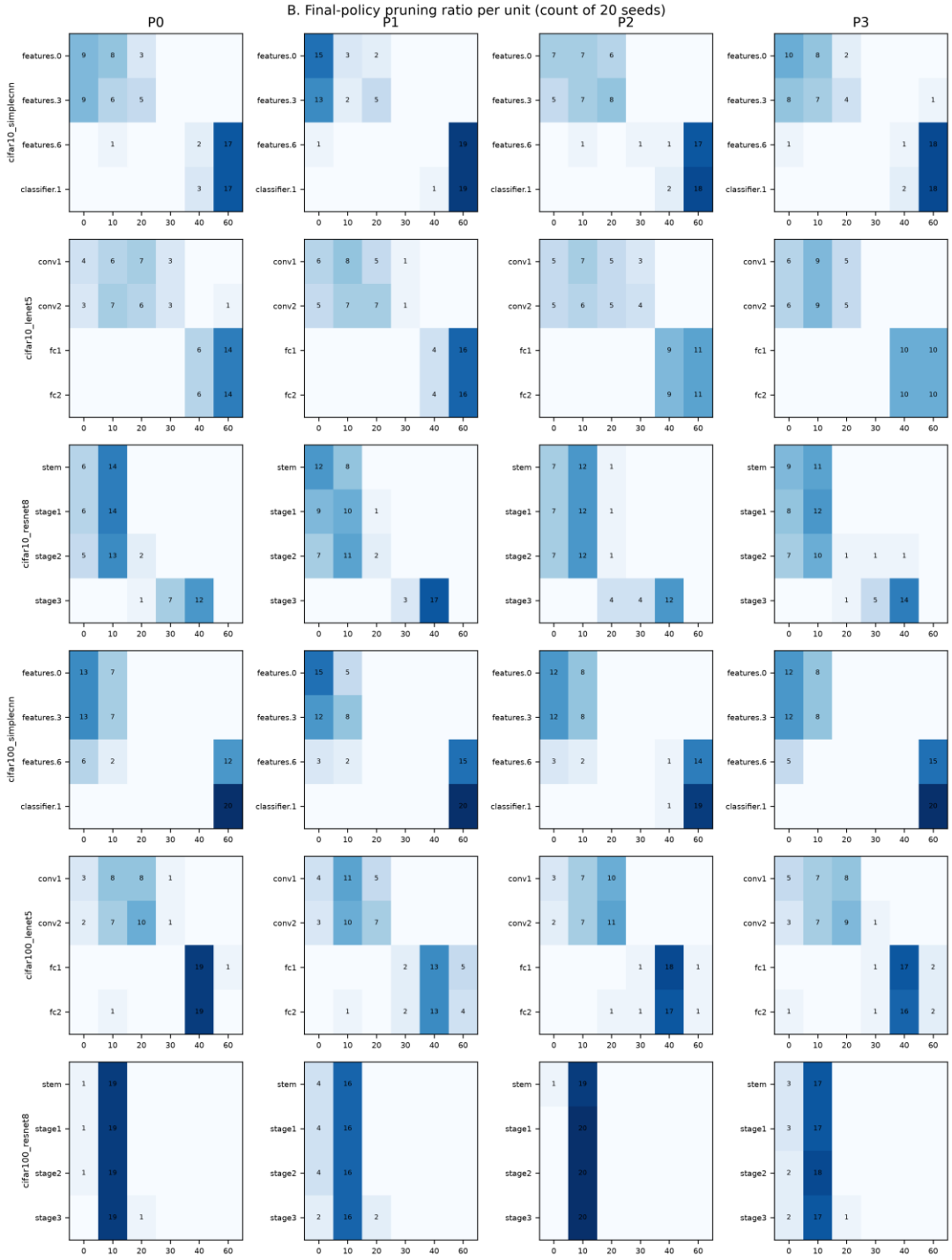


Fig. 5.2: Final-policy pruning ratio per unit (Phase 3). Count of the 20 seeds choosing each ratio, by setting (rows) and condition (columns P0–P3; P2 is the *shuffled* prior).

Interpretation and claim boundary. PPO's allocations are far more accurate than uniform or random allocation at the same sparsity; against magnitude-based baselines the advantage is mixed, and against LAMP it is reversed. The comparison is before fine-tuning, at the sparsity PPO chose; it does not show that PPO beats one-shot magnitude-

based allocation in general or matches LAMP, and says nothing about uniform pruning after fine-tuning (Phase 6 did not include Uniform).

5.4 Phase 3 — Effect of Sensitivity Guidance

Question (RQ2, first test). Does the sensitivity prior improve PPO's search and the final model? P0 (no prior), P1 (sensitivity), P2 (shuffled) and P3 (constant prior) were compared on the VAL-RL (I) accuracy AUC over the 512 sampled policies (primary) and on test accuracy in excess of matched global magnitude (final model, P1 − P0).

Results (Table 5.8, Phase-3 rows):

(a) **Primary exploration endpoint (P1 − P0, VAL-RL (I) accuracy AUC): MODERATE**, supported in 3 of 6 settings: C10 SimpleCNN (+0.46, Holm p = 0.002), C100 SimpleCNN (+0.35, Holm p = 0.0098) and C100 ResNet-8 (+0.54, Holm p = 5.7×10^{-5}). The other three settings are inconclusive, and none is opposite.

(b) **Secondary exploration endpoints: STRONG in 6 of 6** settings for each of reward-rank AUC (+0.013 to +0.032 in rank score, Holm p ≤ 5.5×10^{-4}), utility AUC (Holm p ≤ 3.6×10^{-4}) and destructive-policy frequency (lower under P1 by 0.9–3.8 percentage points of sampled policies, Holm p ≤ 0.0022).

(c) **Layer specificity** (P1 better than both the shuffled and the constant prior): **LIMITED**, supported only in C10 SimpleCNN.

(d) **Final model: NOT SUPPORTED** (0 of 6 supported, 0 opposite): the P1 − P0 differences in excess over global magnitude range from −0.10 to +0.86 pp, every Holm p ≥ 0.094, and the raw test differences are all inconclusive.

Control insight. On LeNet-5 the shuffled and constant priors did at least as well as the correct prior (C10 LeNet-5, VAL-RL (I) accuracy AUC: P1 − P2 = −0.21 (Holm p = 0.27), P1 − P3 = −0.21 (Holm p = 0.31); descriptive mean test accuracy, C10 LeNet-5: P1 62.17%, P2 62.54%, P3 62.83%; C100 LeNet-5: P1 30.81%, P2 31.15%, P3 31.05%).

All three Phase-3 priors carry the same total pressure towards lighter pruning, $\Sigma c_l = 0.5 \cdot \Sigma S_l$ (Section 3.10), so part of the exploration effect may come from a generic conservative

bias rather than layer-specific information. The Phase-3 data cannot separate the two; this possibility motivated the centred prior of Phase 4.

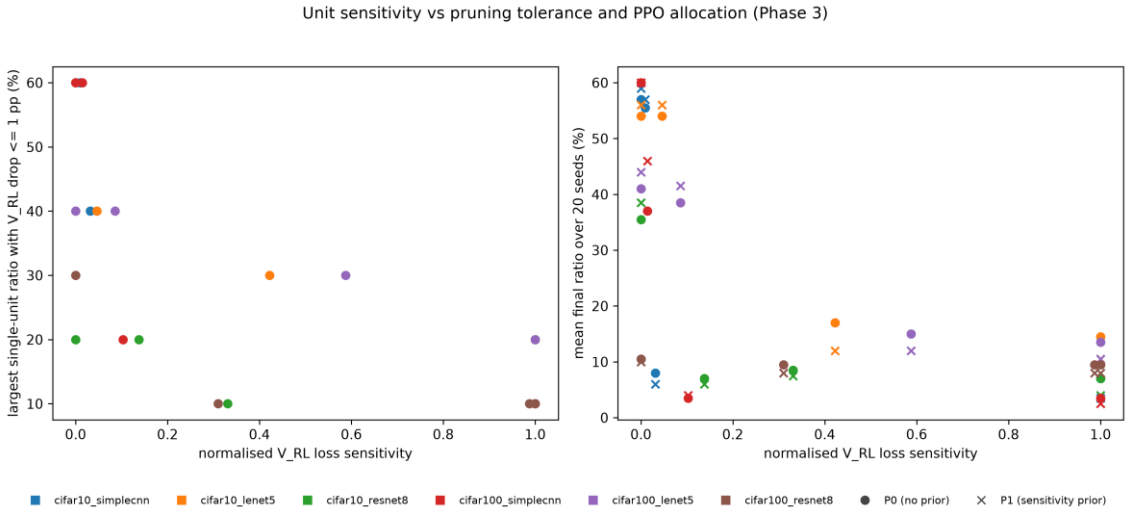


Fig. 5.3: Unit sensitivity, pruning tolerance and PPO allocation (Phase 3). Left: largest single-unit ratio with a VAL-RL (I) accuracy drop ≤ 1 pp against normalised loss sensitivity S_l. Right: mean final ratio of P0 (circles) and P1 (crosses) against S_l. Highly sensitive units tolerate only 10–20% pruning and receive low ratios.

Interpretation and claim boundary. The prior changes *how* PPO searches (better-ranked policies sooner, fewer destructive ones), not *what* it returns on test accuracy. "Sensitivity improves pruned-model accuracy" is not supported, and the layer-specific part of the effect is not established by Phase 3.

5.5 Phase 4 — Centred Sensitivity and Exploration

Question (RQ2, second test). Without net conservatism, does *relative* sensitivity still help the search?

Evidence. Phase 4 Stage B (600 runs under R0, seeds 400–419) compared the centred prior P2, $c_l = 1.0 \cdot (S_l - \bar{S})$, whose coefficients sum to zero, with P3 (centred values permuted across units), P4 (magnitude-matched pure conservatism), P5 (sign-reversed) and P0 (no prior), on the reward-rank AUC of the VAL-RL (I) landscape.

Results (Table 5.8, Phase-4 rows):

(a) **Sensitivity-specific statement** (P2 better than both P3 and P4): **STRONG**, supported in 5 of 6 settings and opposite in none. P2 − P3 ranges from +0.018 to +0.097 in rank-score AUC (Holm $p \leq 3.4 \times 10^{-4}$, all 6 supported); P2 − P4 from

+0.028 to +0.070 in the five supported settings (C100 ResNet-8: −0.002, inconclusive).

(b) **Against the sign-reversed prior** (P2 − P5): supported in **6 of 6** settings, by +0.021 to +0.145 (Holm $p = 1.1 \times 10^{-5}$); against no prior, P1 − P0 and P2 − P0 were each supported in 6 of 6.

(c) **One exploration measure was reversed.** On the VAL-RL (I) accuracy AUC, the statement "P2 better than P3 and P4" is NOT SUPPORTED and REVERSED. P2 was significantly *below* the magnitude-matched conservative prior P4 in 4 of 6 settings (C10 LeNet-5, C10 ResNet-8, C100 LeNet-5 and C100 ResNet-8; mean −0.26 to −0.76): pushing aggressive actions onto insensitive units samples policies better ranked under R0 but less accurate on average.

(d) **Other secondary endpoints:** destructive frequency MODERATE (4 of 6); utility AUC LIMITED (5 supported, 1 opposite); best-seen rank LIMITED (1 of 6); final-selected rank NOT SUPPORTED (0 of 6).

Interpretation and claim boundary. With the shared conservatism removed, relative sensitivity still makes the *average* explored policy better ranked under the reward, stronger evidence for a layer-specific effect than Phase 3 gave; it does not improve the best policy found, accuracy-based exploration (reversed) or the final model.

5.6 Phase 4 — Policy Retention and Archive Selection

Question (RQ3). Does PPO end on the good policies it visits, and does archive-based selection change this?

Evidence. Phase 4 Stage C compared four variants under R0 and P2 (20 seeds per setting): **S0**, the terminal deterministic policy; **S1**, archive selection over the S0 runs; **S2**, elite behavioural cloning ($K = 5$, $\eta = 0.1$) with the terminal policy; and **S2A**, cloning with archive selection. The archive rule takes the top 10 visited policies by VAL-RL (I) reward and returns the one with the highest VAL-SELECT (I) score (rank 1 = optimum).

Table 5.3: Retention: the best policy encountered, the terminal policy (S0) and the archive-selected policy (S2A). Reach is the share of the 20 runs whose policy is in the top-k of the 1,296-policy VAL-RL (I) landscape. Ranks are medians over runs. Δ test is S2A − S0 test accuracy (pp, mean).

Setting	A: best seen, top-10 reach (S0)	B: terminal (S0), top-10 reach	B: median rank	C: archive (S2A), top-10 reach	C: top-20 reach	C: median rank	Δ test (S2A − S0)
C10 SimpleCNN	100%	0%	92	100%	100%	3	+0.14
C10 LeNet-5	100%	0%	69	100%	100%	6	**+0.89**
C10 ResNet-8	95%	0%	96	100%	100%	1	**−5.66**
C100 SimpleCNN	100%	0%	65	100%	100%	6	+0.22
C100 LeNet-5	100%	5%	173	85%	100%	7	+0.45
C100 ResNet-8	95%	0%	114	80%	100%	10	**−4.00**

Results. A. PPO visits top policies: the best policy seen has a median rank of 1–2.5 and is in the top 10 in 95–100% of runs (Phase 3: median best-seen rank 1–4.5). **B. PPO does not end on them:** the terminal policy reaches the top 10 in **0% of runs in five settings and 5% in the sixth** (C100 LeNet-5), with median rank 65–173; in Phase 3 the terminal P0 policy reached the top 10 in 0% of runs in all six settings. **C. The archive retains them:** with S2A the selected policy reaches the top 10 in **80–100%** of runs and the top 20 in 100%, with a median rank of 1–10. "S2A better than S0 on final-selected VAL-RL (I) rank" is STRONG, supported in 6 of 6 settings, with the mean rank improving by 66–150 positions (Holm $p = 3.4 \times 10^{-5}$), and the same holds on VAL-SELECT (I) rank. The archive alone (S1) gives 75–100% top-10 reach and is likewise supported in 6 of 6, whereas elite behavioural cloning alone (S2) improved the final-selected rank in only 3 of 6 settings (MODERATE) and the best-seen rank in 1 of 6 (LIMITED): the retention gain is mainly a *selection* effect of the archive.

Test accuracy is mixed (Table 5.3, last column): small gains in four settings (+0.14, +0.89, +0.22 and +0.45 pp) and **large losses on ResNet-8: −5.66 pp (C10) and −4.00 pp (C100)**, with S1 showing the same pattern. On ResNet-8 the better-ranked policies are much sparser than the terminal ones: 32.2% → 47.8% (C10) and 8.5% → 27.7% (C100) mean sparsity for S0 → S2A. The pre-registered Stage-C criterion (VAL-SELECT (I) rank) favours the archive variants by construction; the test comparison does not share this bias.

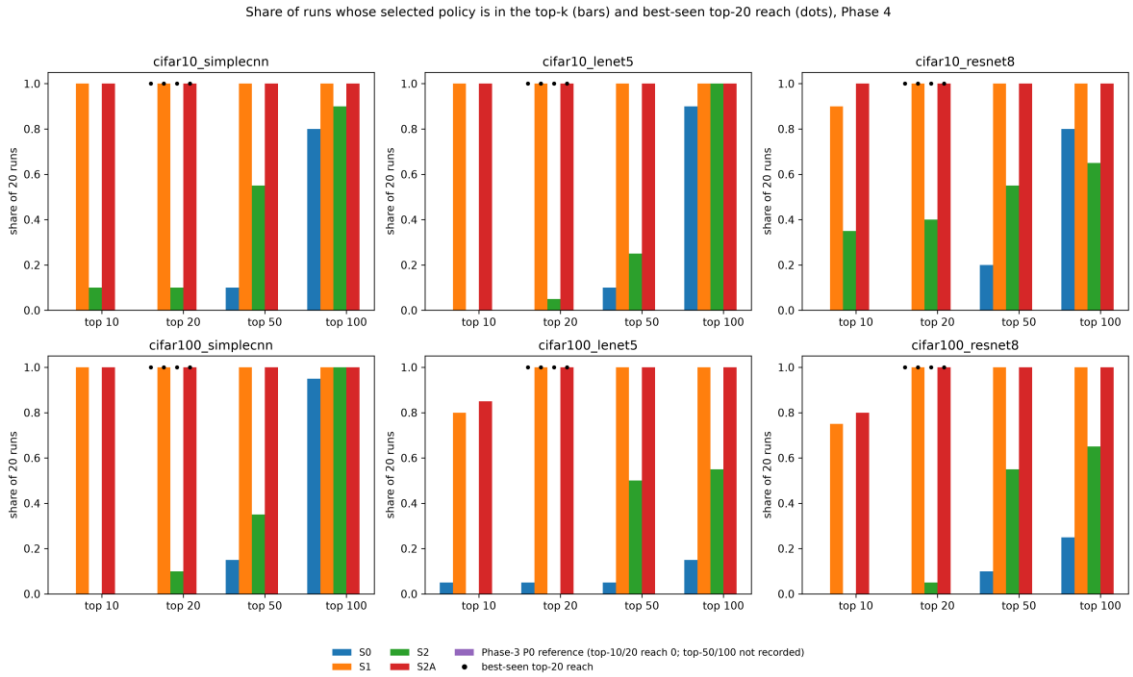


Fig. 5.4: Share of runs whose selected policy lies in the top-k of the VAL-RL (I) landscape (Phase 4, Stage C). Bars: S0, S1, S2 and S2A. Dots: best-seen top-20 reach (100% for every variant). The Phase-3 P0 series has no bars: its top-10 and top-20 reach is 0, and top-50/100 reach was not recorded.

Interpretation and claim boundary. Exploration is not the bottleneck; retention is. Archive selection does **not** consistently improve test accuracy: it lost 4–6 pp on ResNet-8, because the policies it retains are those the reward ranks highest, and that reward is misaligned (Section 5.7). The retention result is a validation-ranking result.

5.7 Phase 4 — Reward Misalignment

Question (RQ4, diagnosis). Once search and retention work, does the reward point PPO towards acceptable models? Stage A tested three redesigned trade-off rewards against R0; the final pipeline F = (R0, P2, S2A) was compared with the control K = (R0, P0, S0), the Phase-3 configuration on fresh seeds.

Redesigned rewards (criterion A: NOT MET). No redesigned reward met both pre-registered conditions: R1a (common 50% target) reduced the cross-setting spread of final sparsity to 0.29× that of R0 but kept ≥ 95% VAL-RL (I) retention in only 2 of 6 settings (C100 ResNet-8 retained 67.5%); R1b (95%-retention targets) kept retention but not the spread (1.05×); R2 (anchor-normalised) reached 0.66× and 4 of 6. R0 was carried forward unchanged.

R0's own optimum. On the VAL-RL (I) landscapes, R0's optimum lies at **23.5–56.4% sparsity**, depending on the setting. On ResNet-8 that optimum is an accuracy-losing policy: 48.8% sparsity retaining 89.7% of dense VAL-RL (I) accuracy (C10), and 23.5% sparsity retaining 93.4% (C100).

Better search finds that optimum (Table 5.4, Phase-4 rows). On ResNet-8, F's archive selection reached R0's optimum region, with median selected VAL-RL (I) rank 1 (C10) and 10 (C100) and mean sparsity rising from 29.0% to 47.8% (C10) and from 8.3% to 27.7% (C100) relative to K. Test accuracy fell accordingly: **C10:** 76.82% → 70.83%, a **7.86 pp** drop below dense; **C100:** 43.11% → 39.22%, a **4.77 pp** drop.

Final pipeline against the control and LAMP. Criterion D, F > K on test excess over matched LAMP, is **NOT MET** (LIMITED): supported in C10 SimpleCNN (+0.50), C10 LeNet-5 (+0.95) and C100 LeNet-5 (+0.67 pp), **opposite on both ResNet-8 settings** (−0.95 and −0.92 pp), inconclusive in C100 SimpleCNN. Criterion E, LAMP competitiveness, is met only as non-inferiority: LAMP is significantly better than F in **6 of 6** settings (F − LAMP = −0.11, −0.11, −0.68, −0.48, −0.08 and −1.62 pp; Holm $p \leq 0.012$), and the one-sided non-inferiority test with a 0.5 pp margin is supported in 3 settings, C10 SimpleCNN, C10 LeNet-5 and C100 LeNet-5 (Holm $p = 5.7 \times 10^{-6}$). F remains far ahead of Uniform: +12.05 to +36.04 pp at matched zero count.

Phase 4 pre-registered criteria: A. Reward **NOT MET**; B. Sensitivity-specific prior **MET** (STRONG); C. Search **MET** (STRONG for S1 and S2A); D. Final model vs control **NOT MET** (LIMITED); E. LAMP competitiveness **MET only through non-inferiority** in 3 settings, "F > LAMP" NOT SUPPORTED. Phase 4 thus meets its own success rule (at least one criterion) but not its two model-level criteria (A and D).

Interpretation and claim boundary. The optimiser did what the reward specified: once search and retention improved, PPO found R0's fixed-rate optimum (Section 3.13.1), which on fragile networks is over-pruned. This is evidence of reward misalignment for R0 on these landscapes, not a failure of PPO alone or of every trade-off reward.

5.8 Phase 5 — Accuracy-Constrained Reward

Question (RQ4, remedy; RQ2, third test). Does an accuracy-constrained, validation-aware reward remove the over-pruning? Does sensitivity guidance still help under it?

Evidence. Phase 5 ran 360 runs (configuration II, seeds 500–519) under **C0**, R0 + centred prior + R0-score archive selection (the Phase-4 pipeline without behavioural cloning); **C1**, constrained reward R_c with $\tau = 0.98$ + centred prior + constraint-aware selection; and **C2**, C1 without the prior. All three were evaluated once on test, after the policy list was frozen.

Table 5.4: ResNet-8 reward alignment across Phases 4 and 5 (test accuracy %, mean of 20 runs; sparsity = mean selected masked sparsity). Phase-4 and Phase-5 rows share the test set and dense reference.

Setting	Configuration	Sparsity	Test	Drop vs dense (pp)
C10 ResNet-8 (dense 78.69)	Phase-4 K (R0, no prior, terminal)	29.0%	76.82	1.87
	Phase-4 F (R0, P2, S2A)	47.8%	70.83	7.86
	Phase-5 C0 (R0, P2, archive)	46.2%	71.37	**7.32**
	Phase-5 C1 (R_c, P2, constrained selection)	24.5%	77.51	**1.18**
	Phase-5 C2 (R_c, no prior)	23.9%	77.57	1.12
C100 ResNet-8 (dense 43.99)	Phase-4 K	8.3%	43.11	0.88
	Phase-4 F	27.7%	39.22	4.77
	Phase-5 C0	25.9%	39.94	4.05
	Phase-5 C1	15.6%	42.83	1.16
	Phase-5 C2	15.5%	42.90	1.09

Over-pruning on ResNet-8. The constrained reward reduced the mean test-accuracy loss from **7.32 pp (C0) to 1.18 pp (C1)** on CIFAR-10 ResNet-8 and from 4.05 to 1.16 pp on CIFAR-100 ResNet-8, at much lower sparsity: C1 − C0 was +6.14 pp (95% CI 4.92–7.36; 20 of 20 seeds; Holm $p = 1.1 \times 10^{-5}$) and +2.89 pp (95% CI 2.30–3.48; 20 of 20; Holm $p = 1.1 \times 10^{-5}$). C1's selected policies lie at or next to the constrained optimum (median constrained rank 3 and 1).

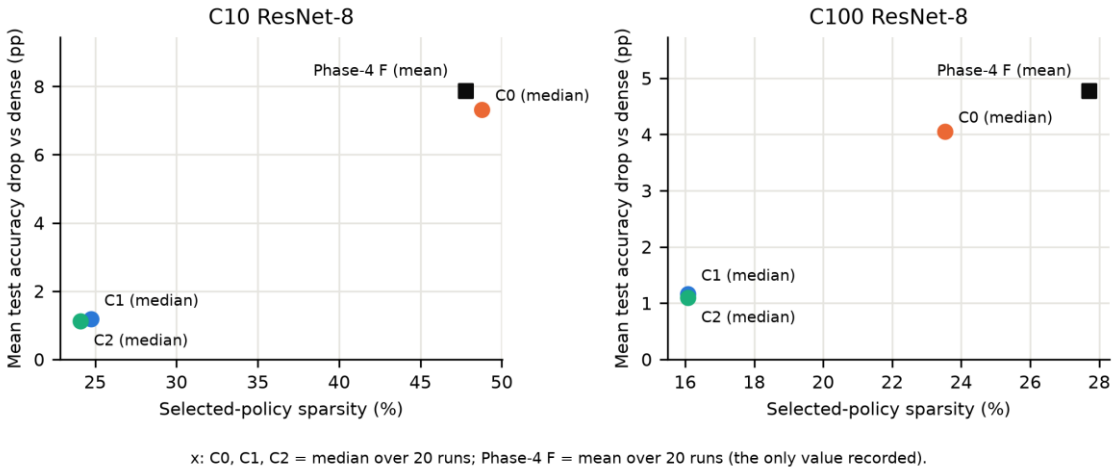


Fig. 5.5: ResNet-8 reward alignment: test-accuracy drop below dense against selected-policy sparsity. C0, C1 and C2 are plotted at their **median** sparsity (48.8%, 24.7% and 24.1% for C10; 23.5%, 16.1% and 16.1% for C100); the Phase-4 F reference at its **mean** sparsity (47.8% and 27.7%), the only value recorded.

Effect across all six settings (Table 5.5). C1 − C0 on test accuracy is supported in 4 settings, LeNet-5 (+0.70 and +1.12 pp) and ResNet-8, and **opposite in 2:** C10 SimpleCNN (−0.31 pp, Holm $p = 4.6 \times 10^{-5}$) and C100 SimpleCNN (−0.21 pp, Holm $p = 0.0022$). Where C1 is more accurate it prunes 10–22 pp less than C0; on SimpleCNN, C0 already chose near-safe policies and C1 spends the 2% accuracy allowance on slightly more sparsity. Safe sparsity is architecture-dependent: C1's mean sparsity runs from 15.6% (C100 ResNet-8) to 56.9% (C10 SimpleCNN).

Table 5.5: Phase 5 by setting (test accuracy %, mean of 20 runs; differences in pp).

Setting	C0 test (sparsity)	C1 test (sparsity)	C2 test	C1 − C0	C1 − LAMP	C1 − Global
C10 SimpleCNN	76.75 (56.1%)	76.45 (56.9%)	76.50	−0.31 −	−0.49 −	−0.09 ·
C10 LeNet-5	63.14 (53.9%)	63.83 (38.6%)	63.99	+0.70 +	−0.32 −	+1.42 +
C10 ResNet-8	71.37 (46.2%)	77.51 (24.5%)	77.57	+6.14 +	−0.53 −	+6.51 +
C100 SimpleCNN	50.28 (52.9%)	50.07 (53.1%)	50.13	−0.21 −	−0.29 −	+0.32 +
C100 LeNet-5	30.67 (46.3%)	31.79 (30.6%)	31.86	+1.12 +	−0.26 −	+0.36 +
C100 ResNet-8	39.94 (25.9%)	42.83 (15.6%)	42.90	+2.89 +	−0.51 −	+3.48 +

Comparison with global magnitude (descriptive by pre-registration). C1 is more accurate than global magnitude at the identical zero count in **5 of 6** settings, by +0.32 to +6.51 pp on test (sign-flip Holm $p \leq 0.0011$). C10 SimpleCNN is inconclusive (−0.09 pp). The comparison is not confirmatory.

Comparison with LAMP. LAMP is significantly more accurate than C1 in **all 6** settings, by **0.26–0.53 pp** on test (Holm $p \leq 5.9 \times 10^{-5}$; LAMP won 16–20 of 20 seeds); the gap is within 0.5 pp in four settings and 0.51–0.53 pp in both ResNet-8 settings. Relative to C0 it narrowed in C100 ResNet-8 (+0.66 pp, supported), widened on both SimpleCNN settings (opposite) and was unchanged elsewhere.

Sensitivity under the corrected reward (C1 − C2; Table 5.8, Phase-5 rows). The primary family (24 tests) has **0 opposite outcomes**: reward-rank AUC is supported in 5 of 6 settings (Holm $p \leq 0.019$; C10 ResNet-8 inconclusive), VAL-RL accuracy AUC supported in 5 of 6 (Holm $p \leq 0.012$; C100 ResNet-8 inconclusive), and episodes to the first top-20 and top-10 policy inconclusive in all six settings although every point estimate favours C1. Secondary: the destructive-action rate is lower in 6 of 6 (Holm $p \leq 1.5 \times 10^{-4}$), selected rank better in 3 of 6 and best-seen rank in 1 of 6. Final test accuracy (descriptive, not pre-registered as a test): C1 is **0.05–0.16 pp below** C2 in all six settings.

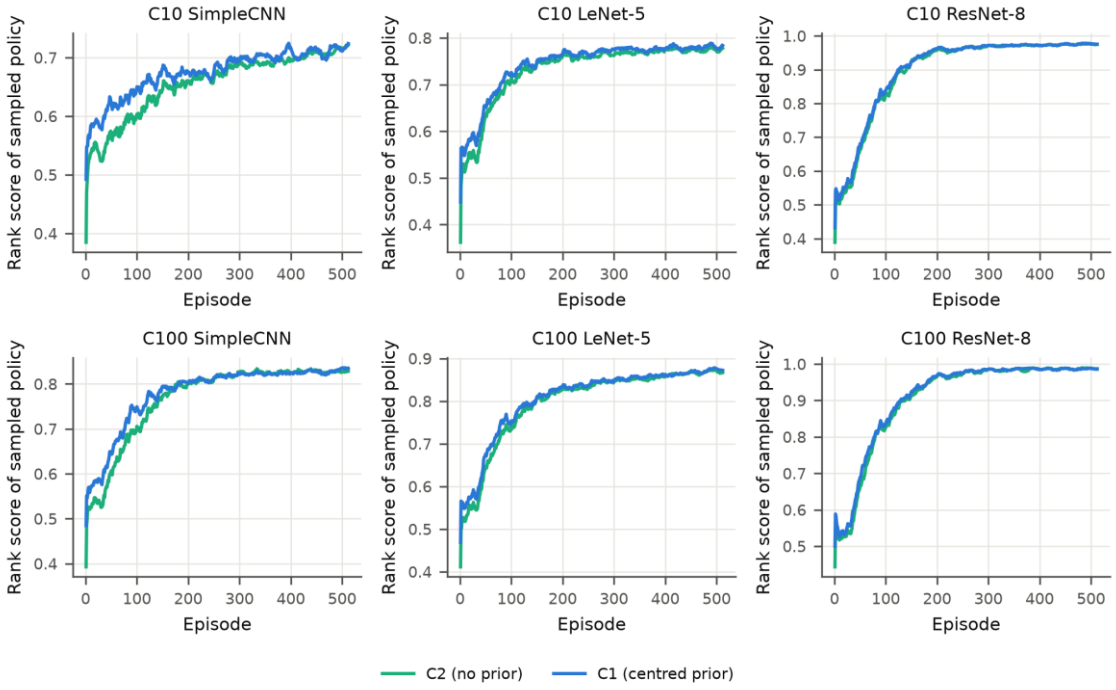


Fig. 5.6: Rank score of the sampled policy over 512 episodes, C1 (centred prior) vs C2 (no prior) (Phase 5, VAL-RL). Mean over 20 seeds, 16-episode rolling mean. The prior's advantage is concentrated early in training.

Phase 5 pre-registered criteria: A. Accuracy safety **PARTIALLY MET** (supported in both ResNet-8 settings, but opposite in 2 other settings; at most 1 allowed); B. Non-trivial compression **MET** (C1 median sparsity ≥ 10% in 6 of 6 and ≥ 20% in 5; 4 required); C. Search quality **PARTIALLY MET** (median selected constrained rank ≤ 20 in 5 of 6, but

only 76.7% of runs selected a top-20 policy, ≥ 80% required; 90.8% visited one); D. Sensitivity contribution **MET** (a primary exploration endpoint supported in every setting, 0 of 24 opposite); E. Final model improvement **PARTIALLY MET** (supported in 4 settings, opposite in 2; no degradation allowed); F-strong (LAMP) **NOT MET** (C1 never beats LAMP; LAMP better in 6 of 6); F-weak (LAMP) **MET** (within 0.5 pp in 4 settings, none worse than 1.0 pp).

Interpretation and claim boundary. Making accuracy a constraint largely removed the ResNet-8 over-pruning; the remaining effects are small and architecture-dependent, and sensitivity still improves exploration but not the final model. The constrained reward does not "guarantee" accuracy (Section 5.9), does not eliminate over-pruning in every setting and does not make PPO better than LAMP.

5.9 Phase 5 — Validation-to-Test Transfer

Question. Does a policy feasible on validation data remain feasible on test? Phase 5 recorded whether q ≥ 0.98 held on VAL-RL (enforced by the reward), VAL-SELECT (enforced by selection) and test.

Table 5.6: Validation-to-test feasibility of the selected Phase-5 policies (q ≥ 0.98; runs out of 120).

Condition	Feasible on VAL-RL	Feasible on VAL-SELECT	Feasible on test
C0	40 / 120	44 / 120	40 / 120 (33.3%)
C1	**120 / 120**	**119 / 120 (99.2%)**	**81 / 120 (67.5%)**
C2	120 / 120	119 / 120	90 / 120 (75.0%)

The single C1 policy infeasible on VAL-SELECT is the one selection-rule fallback (C100 LeNet-5). Of the 119 C1 policies feasible on VAL-SELECT, **39 were infeasible on test.** C1 test feasibility was 100% in both SimpleCNN settings and in C10 ResNet-8, 60% in C100 LeNet-5, 45% in C10 LeNet-5 and **0% in C100 ResNet-8**, where the mean test retention was 0.974, just below the 0.98 threshold.

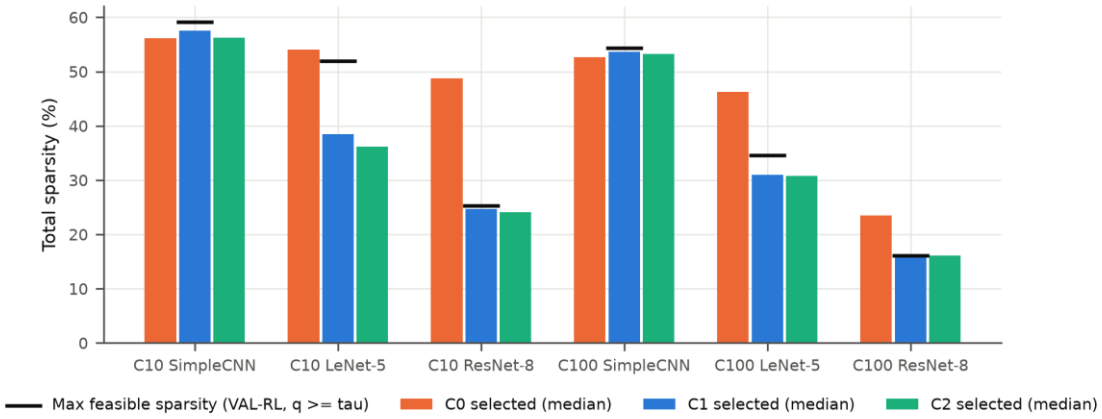


Fig. 5.7: Median selected sparsity of C0, C1 and C2 against the maximum feasible sparsity on VAL-RL ($q \geq \tau$), by setting (Phase 5). C0 selects beyond the feasible maximum in the LeNet-5 and ResNet-8 settings; C1 and C2 select at or just below it.

Interpretation and claim boundary. Validation feasibility is evidence about test feasibility, not a guarantee: the failures occur where C1 selects close to the VAL-RL feasibility boundary (Figure 5.7), and two 2,500-image splits carry roughly ±1 pp of sampling noise, comparable to the 2% relative margin. No claim of the form "the validation constraint guarantees generalisation" is made; Phase 6 shows that equal fine-tuning removes most of this gap (Section 5.10).

5.10 Phase 6 — Equal Fine-Tuning

Question (RQ5). Do the differences between PPO and the strongest deterministic methods persist when every pruned model receives the same fine-tuning?

Evidence. Phase 6 fine-tuned three pruned models in each of 120 (setting, policy seed) pairs: **PPO**, the frozen Phase-5 C1 policy, and **LAMP and global magnitude** at exactly PPO's zero count (mismatch 0 in 120 of 120 triples), alongside a dense control. **Phase 6 contains no Uniform baseline.** Every model received the same 5-epoch fixed-mask protocol with shared seed and data order (Section 4.13) and was evaluated once on test: 480 runs plus 6 reproduction runs.

Table 5.7: Phase 6: test accuracy before → after equal fine-tuning, and the PPO–LAMP and PPO–Global gaps (pp; mean of 20 paired runs; 95% t-CI).

Setting (sparsity)	PPO	LAMP	Global	PPO − LAMP before	PPO − LAMP after [95% CI]	PPO − Global after [95% CI]
C10 SimpleCNN (56.9%)	76.45 → 79.73	76.94 → 79.95	76.54 → 79.89	−0.49	−0.22 [−0.28, −0.15]	−0.16 [−0.21, −0.11]
C10 LeNet-5 (38.6%)	63.83 → 64.88	64.15 → 65.06	62.41 → 64.83	−0.32	−0.18 [−0.28, −0.09]	+0.05 [−0.05, 0.15]
C10 ResNet-8 (24.5%)	77.51 → 78.74	78.03 → 78.73	71.00 → 78.67	−0.53	+0.00 [−0.02, 0.03]	+0.07 [0.03, 0.11]
C100 SimpleCNN (53.1%)	50.07 → 50.53	50.36 → 50.58	49.75 → 50.51	−0.29	−0.05 [−0.09, −0.01]	+0.03 [−0.02, 0.07]
C100 LeNet-5 (30.6%)	31.79 → 32.36	32.05 → 32.49	31.43 → 32.24	−0.26	−0.14 [−0.19, −0.08]	+0.12 [0.06, 0.18]
C100 ResNet-8 (15.6%)	42.83 → 44.02	43.34 → 44.06	39.35 → 44.04	−0.51	−0.04 [−0.07, −0.01]	−0.02 [−0.07, 0.03]
Mean of 6 settings	**57.08 → 58.38**	**57.48 → 58.48**	**55.08 → 58.36**			

The dense control moved from 57.91 to 58.51 (mean over settings).

Recovery. Every method recovered in every setting: "fine-tuning improves accuracy over the same mask before fine-tuning" is supported in 18 of 18 method × setting tests (Holm $p < 10^{-4}$), and recovery beyond the dense control's own gain in 17 of 18 (the exception is LAMP on C10 SimpleCNN, 0.04 pp less). Mean recovery per setting was PPO: 0.46–3.28 pp; LAMP: 0.22–3.01 pp; global magnitude: 0.76–7.67 pp, inversely ordered to pre-fine-tuning accuracy. Every method ended within 0.14 pp of the dense reference on average; on C10 SimpleCNN the dense control itself gained +3.05 pp from the 5 extra epochs (Table 5.1, note †).

PPO versus LAMP. Before fine-tuning LAMP led by 0.26–0.53 pp in every setting; after it, the absolute PPO–LAMP difference was **0.00–0.22 pp.** The gap moved in PPO's favour (supported) in **5 of 6** settings (Holm $p \leq 0.027$; C10 LeNet-5 inconclusive, Holm $p = 0.055$; none opposite). LAMP was still significantly ahead in **3 of 6** settings: C10 SimpleCNN, −0.22 pp (Holm $p = 3.8 \times 10^{-5}$); C10 LeNet-5, −0.18 pp (Holm $p = 0.0026$); C100 LeNet-5, −0.14 pp (Holm $p = 4.6 \times 10^{-4}$); the other three were inconclusive, and PPO was significantly ahead in none. **The ±0.5 pp rule. The post-recovery PPO–LAMP differences fell within the pre-registered ±0.5 pp decision margin in all six settings**: each 95% t-interval lies inside (−0.5, +0.5) pp. It is a confidence-interval decision rule, not a formal statistical equivalence test (Section 4.15.2). Three of the six

settings were already inside the margin before fine-tuning (C10 LeNet-5, C100 SimpleCNN and C100 LeNet-5).

PPO versus global magnitude. Before fine-tuning PPO led global magnitude by up to 6.51 pp (Section 5.8); after equal recovery the differences were −0.16 to +0.12 pp. PPO's largest remaining advantage was **0.12 pp** (C100 LeNet-5; supported), it was ahead by 0.07 pp in C10 ResNet-8 (supported), global was ahead by 0.16 pp in C10 SimpleCNN (opposite), and the other three were inconclusive. The gap change was opposite in 5 settings, because global recovered much more, and inconclusive on C10 SimpleCNN; every post-recovery PPO–Global interval lies inside ±0.5 pp.

Fixed masks and integrity. No pruned weight regrew: per-layer zero counts were identical before and after fine-tuning, the mask hash was unchanged and no masked weight was non-zero, in 360 of 360 pruned runs; all 20 integrity checks passed.

Phase 6 pre-registered criteria, all **MET**: A. Fairness (one protocol and epoch count; shared seed and data stream in every pair; zero-count mismatch 0 in 120 of 120 triples; masks equal the frozen table); B. Recovery (supported for all three pruning methods in all 6 settings); C. PPO competitiveness (the PPO–LAMP gap moved towards PPO in 5 settings, towards LAMP in none; C concerns the change of the gap, not who is ahead, and LAMP remains ahead in 3 settings); D. Architecture generalisation (all six settings complete in every family); E. Sparsity preservation (360 of 360 pruned runs); F. Reproducibility (480 of 480 runs complete; 6 of 6 reproductions bit-identical).

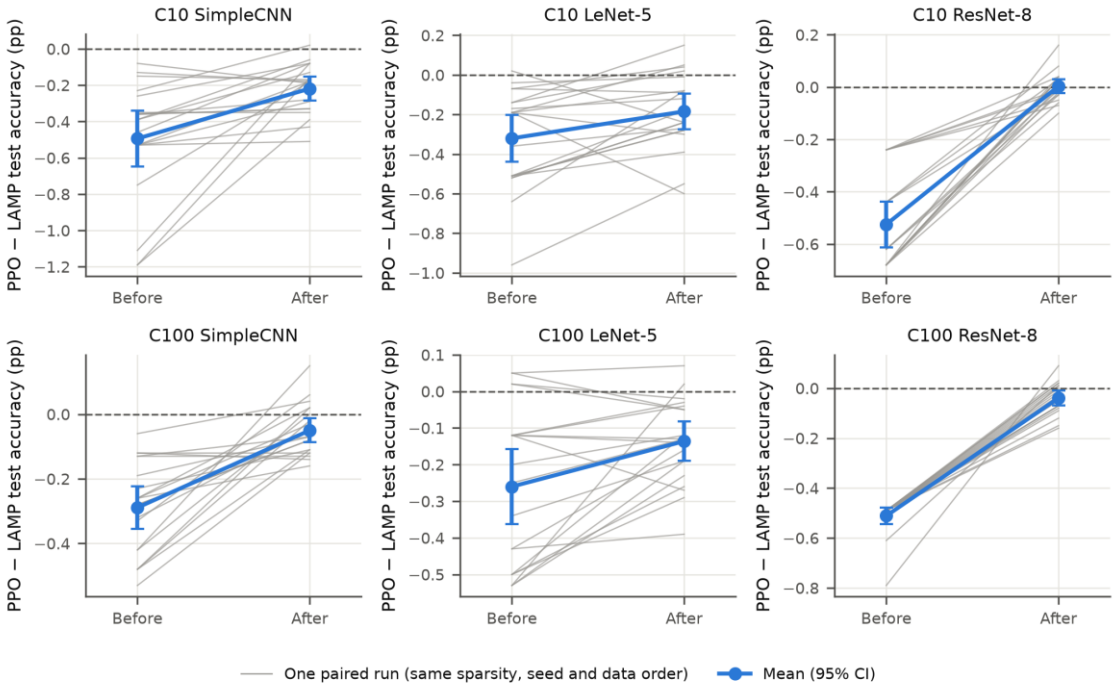


Fig. 5.8: PPO − LAMP test accuracy before and after equal 5-epoch fine-tuning (Phase 6). Grey: paired runs; blue: mean with 95% CI. Every post-fine-tuning interval lies inside the ±0.5 pp margin (not drawn; Table 5.7).

Interpretation. Equal, modest fine-tuning substantially shrinks the differences among pruning methods at these sparsities: PPO's advantage over global magnitude and LAMP's advantage over PPO both lay mainly *before* fine-tuning. After recovery every pruned model met retention ≥ 0.98 on test, and the PPO retention gap between VAL-SELECT and test narrowed significantly in 4 of 6 settings.

Claim boundary. PPO and LAMP are not claimed to be statistically equivalent, and LAMP remains significantly, though slightly, ahead in three settings. The result holds for these networks, sparsities and a 5-epoch budget, and says nothing about Uniform, ERK or Random after fine-tuning.

5.11 Answers to RQ1–RQ5

Table 5.8: Sensitivity guidance across Phases 3–5: exploration endpoints versus final-model endpoints (settings supported / opposite, of 6).

Phase	Comparison	Exploration endpoints	Final-model endpoint
3	P1 − P0 (prior vs none)	VAL-RL (I) accuracy AUC 3 / 0 (MODERATE); rank AUC 6 / 0; utility AUC 6 / 0; destructive frequency 6 / 0 (STRONG)	excess over matched Global 0 / 0 (NOT SUPPORTED)
3	P1 vs P2 and P3 (specificity)	VAL-RL (I) accuracy AUC 1 / 0 (LIMITED)	—
4	P2 vs P3 and P4 (centred, specific)	rank AUC 5 / 0 (STRONG); VAL-RL (I) accuracy AUC 0 / 4 (REVERSED); destructive 4 / 0	final-selected rank 0 / 0 (NOT SUPPORTED)
4	P2 − P5 (sign-reversed)	rank AUC 6 / 0	—
5	C1 − C2 (prior vs none, constrained reward)	rank AUC 5 / 0; VAL-RL accuracy AUC 5 / 0; time to top-20 / top-10 0 / 0; destructive rate 6 / 0	test accuracy: C1 0.05–0.16 pp below C2 in 6 of 6 (descriptive)

RQ1. Yes, with clear limits: PPO's allocations substantially outperform uniform (5 of 6 settings, up to +31.4 pp on C10 SimpleCNN test) and random allocation (6 of 6) at the sparsity PPO chose, are mixed against global magnitude and ERK, and are significantly less accurate than LAMP before recovery (5 of 6 settings in Phase 3; 6 of 6 for the Phase-4 final pipeline and Phase-5 C1). **RQ2.** Sensitivity improves the search but not the final model (Table 5.8), also against equally conservative controls and under the corrected reward; one Phase-4 exploration measure was reversed. **RQ3.** The terminal policy does not retain the strong policies PPO visits; archive selection restores retention on validation ranking, mainly through the archive rule, with a mixed test effect (Section 5.6). **RQ4.** Reward design materially determines how aggressively PPO prunes: R0's optimum over-prunes fragile networks (ResNet-8 lost up to 7.86 pp once search improved), the constrained reward largely removed this loss (Table 5.4), and validation feasibility transferred to test in only 67.5% of runs. **RQ5.** Equal 5-epoch fixed-mask fine-tuning narrowed the PPO–LAMP gap from 0.26–0.53 pp to 0.00–0.22 pp, within the pre-registered ±0.5 pp confidence-interval margin in all six settings, with LAMP still significantly ahead in three; PPO's advantage over global magnitude fell to at most 0.12 pp.

5.12 Chapter Summary

Each phase answered one question and exposed the next: PPO beat uniform and random pruning but not LAMP (Phase 3); sensitivity improved the search, never the final model; better search revealed a retention problem, which the archive solved on validation ranking, and then a misaligned reward (Phase 4); an accuracy constraint largely corrected it, with imperfect transfer to test (Phase 5); and equal fine-tuning made the methods' differences small (Phase 6). The negative and partial results are reported alongside: LAMP reversed against PPO (Phase 3); criteria A and D not met, ResNet-8 test losses from archive selection and one reversed exploration measure (Phase 4); criteria A, C and E partially met and F-strong not met (Phase 5); LAMP still ahead in three settings (Phase 6). Figures 5.3–5.5 are presentation-only re-renders of frozen phase figures from the same CSVs, every plotted value verified identical. Chapter 6 asks whether these results translate into deployment efficiency.

CHAPTER 6
DEPLOYMENT AND SUSTAINABILITY ANALYSIS

6.1 General

This chapter answers the question that masked pruning (Chapter 5) cannot:

RQ6. Does model compression translate into real deployment benefit?

The evidence comes from four later studies (Table 4.7): **Paper 3** measures the CPU latency of every structured policy (Sections 6.4–6.6); **Phase 7A** realises the learned Phase-5 allocation as a physically smaller network and measures its CPU package energy (Sections 6.7–6.8); **Phase 7B** searches directly in the structured space (Sections 6.9–6.12); and the **energy confirmation** study tests the frozen Phase-7B model (Section 6.13). Benefit is assessed at three levels (Section 3.16.2): compression and memory, computation and latency, and operational energy; **an improvement at one level does not guarantee an improvement at the next**, so each level needs its own measurement, stated with its accuracy cost.

Reporting conventions. Accuracies are configuration-II validation accuracies (2,500 images per split) after the equal 5-epoch recovery; no test-set number exists for these phases (Section 4.3). Phase 7A's pre-registered accuracy split is **VAL-RL**; Phase 7B's is **VAL-SELECT**. Both are reported and the split is always named. Latency and energy were measured on the laptop CPU of Table 4.11 (Sections 4.17–4.18).

6.2 From Compression to Deployment Efficiency

A masked network keeps every tensor at its dense shape and stores zeros in place of removed weights; on a CPU with dense kernels those zeros are still loaded and multiplied unless sparse formats and kernels are used (Hoefler et al., 2021). Weight sparsity is therefore a statement about the model, not its hardware cost, and cost indicators such as parameter count, FLOPs and measured speed can disagree (Dehghani et al., 2022); the thesis makes no latency or energy claim for its masked models. The extension instead realises each policy **structurally**, removing whole channels and neurons to build a smaller dense network (He, Zhang and Sun, 2017; Li et al., 2017; He and Xiao, 2024). The three levels then become measurable: parameter count, memory and serialised size

exactly (Level 1); MACs in closed form and measured batch-1 CPU latency (Level 2); and CPU package energy per 1,000 inferences from the RAPL hardware counter (Khan et al., 2018), with no TDP model or estimate (Level 3).

6.3 Structured Physical Pruning Setup

The structured space is the core space realised physically (Sections 4.6.2 and 4.17): the same six ratios per unit (1,296 policies per network); each unit keeps max(1, round((1 − r)·C)) channels with the largest L1 norm; in ResNet-8 the stem is a **coupled residual pruning group** and the stage units remove **block-internal channels**. Every compacted model was checked to contain no masks, to have analytic parameter and MAC counts and to reproduce the masked-equivalent logits within 10^{-4}, and accuracy always follows the equal 5-epoch Phase-6 protocol. Latency calibration chose configuration A (batch 1, one thread, one performance core; signal-to-noise ratio 12.3, relative reduction 61.6%), and each table was timed in five fresh-process sessions with dense anchors and a 10% drift limit.

6.4 Exhaustive CPU Latency Analysis

Paper 3 timed **all 1,296 physically compacted policies** in each of four tables (SimpleCNN and ResNet-8, with 10 and 100 outputs): **5,184 structured policies** in total. Four session attempts exceeded the drift limit and were repeated; all 20 accepted sessions met it. The median between-session coefficient of variation per shape was 1.2–1.6%, the Spearman correlation of latency gains between sessions 0.96–0.995, and raw and anchor-normalised gains agreed at Spearman ≥ 0.997. Because dense latency on the table days was about 16% above the calibration day, while the anchors within each table agreed with its identity shape to within 1%, latency is analysed *relative to the identity shape of the same table*.

Table 6.1: Exhaustive structured latency tables: cost metrics against measured latency gain (Spearman rank correlation over 1,296 policies per table; latency gain $g_T = 1 - T/T_identity$).

Table	Policies	Parameter reduction vs latency gain	MAC reduction vs latency gain	Pairs where the more parameter-sparse policy is slower	Maximum speed-up (unconstrained)
SimpleCNN, CIFAR-10	1,296	0.32	0.89	39.0%	2.61×
SimpleCNN, CIFAR-100	1,296	0.31	0.89	39.4%	2.59×
ResNet-8, CIFAR-10	1,296	0.55	0.98	30.6%	1.67×
ResNet-8, CIFAR-100	1,296	0.54	0.98	30.8%	1.72×

The maximum speed-up in every table belongs to the most aggressive policy [5, 5, 5, 5]; these maxima describe the latency space only and are **not** accuracy-preserving speed-ups (no accuracy constraint is applied; Section 6.6).

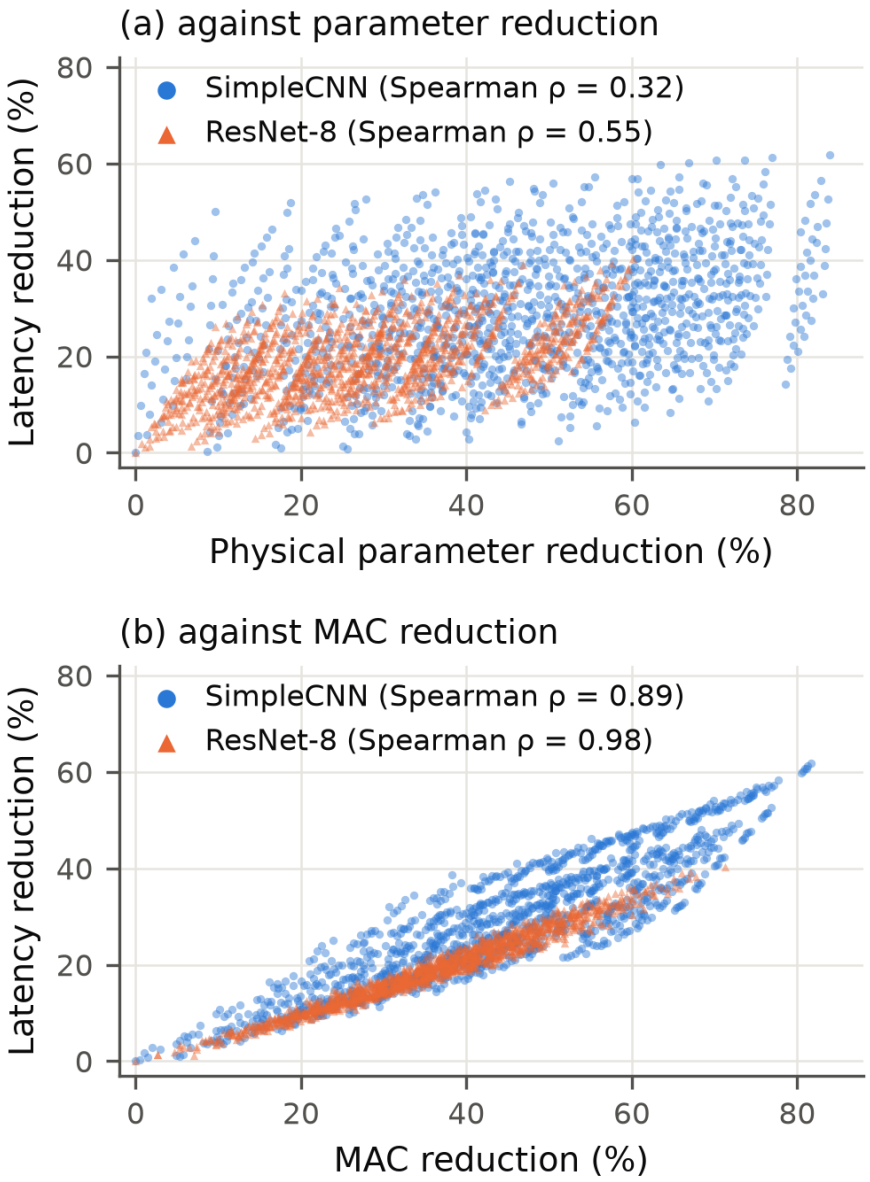


Fig. 6.1: Measured latency reduction against (a) physical parameter reduction and (b) MAC reduction, for all 1,296 structured policies of the CIFAR-10 SimpleCNN and ResNet-8 tables (Paper 3).

6.5 Parameter Reduction versus MAC Reduction versus Latency

Result. Across all four tables (Table 6.1), the Spearman correlation of latency gain was 0.31–0.55 (weak) with parameter reduction and 0.89–0.98 (strong) with MAC reduction. In **39.0–39.4%** of SimpleCNN policy pairs and **30.6–30.8%** of ResNet-8 pairs, the policy with more parameter reduction was *slower*. In the CIFAR-10 SimpleCNN table, [5, 5, 1, 0], which prunes the first two convolutions at 60%, removes 18.7% of parameters and 69.3% of MACs and runs **2.08**× faster, whereas the fc-heavy [0, 0, 0, 5], which prunes

only the 256-unit hidden layer at 60%, removes 51.1% of parameters but 2.9% of MACs for **1.03×**: the hidden layer holds most of the weights but little of the arithmetic. These examples describe cost only; pruning the first convolutions at 60% is destructive in this network (Section 5.4). ResNet-8 correlates more strongly with parameters (0.54–0.55 vs 0.31–0.32) but still has roughly one inverted pair in three.

Interpretation and claim boundary. Parameter sparsity is a weak proxy for CPU latency and MAC reduction a strong one; measured latency remains the deployment-grounded quantity, because even MACs leave part of the ordering unexplained (Spearman $\rho < 1$). These are batch-1, single-thread results on one laptop CPU and do not transfer to other CPUs, thread counts, batch sizes or accelerators.

6.6 Accuracy-Feasible Latency Headroom

Deployment needs the speed-up of policies that also keep accuracy. Paper 3 used the Phase-5 criterion, $q_5 \geq 0.98$ in retained VAL-RL accuracy after the 5-epoch recovery; because the exhaustive 5-epoch landscape was stopped for computational cost (Section 4.6.3), a 1-epoch accuracy proxy had to reach a Spearman correlation ≥ 0.90 with 5-epoch recovery. It was **rejected** for SimpleCNN (Spearman $\rho(A_1, A_5) = 0.844$ over 156 validation pairs; NO-GO) and accepted for ResNet-8 (0.980 over 31 pairs; GO), so **no accuracy-feasibility claim is made for the SimpleCNN latency tables**; its 2.61× maximum is an unconstrained figure only. For ResNet-8, too few validation pairs were truly feasible for the pre-registered threshold rule, so its fallback applied: a pre-committed calibration recovered 48 frozen policies for the full 5 epochs and set the proxy threshold by a recall-first rule with a fixed 0.005 margin (Table 6.2).

Table 6.2: ResNet-8 (CIFAR-10): calibrated accuracy-feasible latency headroom.

Item	Value
Calibration policies (5-epoch recovered)	48
Truly 5-epoch feasible ($q_5 \geq 0.98$)	14
Proxy threshold τ_{proxy}	0.960
Recall / precision at τ_{proxy}	1.00 / 0.875 (14 true positives, 2 false positives)
Proxy-feasible policies in the 1,296 landscape	52
Their parameter / MAC reduction	0–28.9% / 0–18.4%
Best table speed-up among proxy-feasible policies	**1.098×**
Best speed-up among the 14 truly feasible calibration policies	1.089× ([0, 3, 1, 1])
Proxy-feasible policies with speed-up ≥ 1.10×	0

The 52 proxy-feasible policies are *candidates*; only the 48 calibration policies were confirmed by 5-epoch recovery.

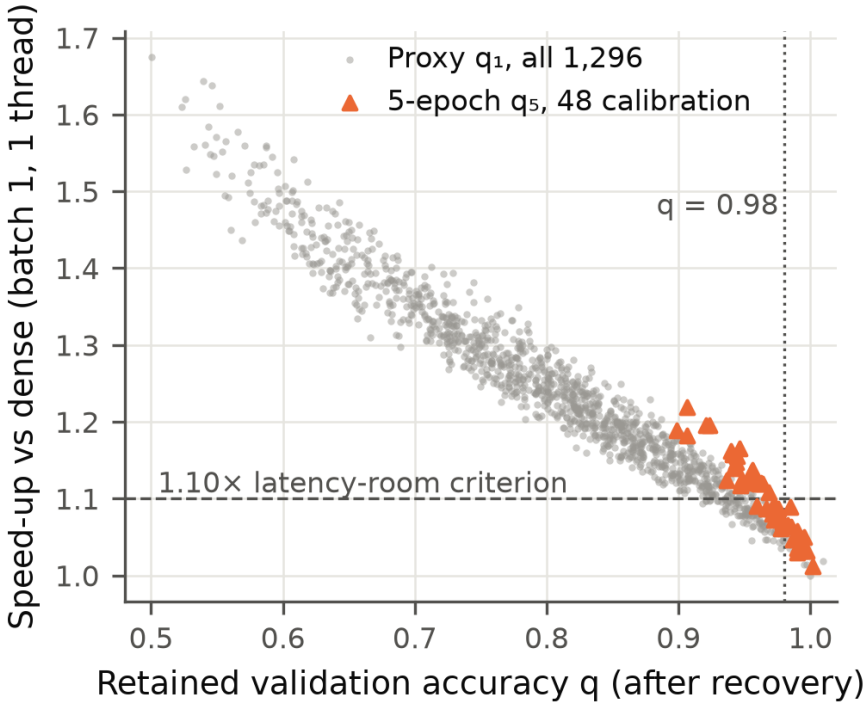


Fig. 6.2: ResNet-8 (CIFAR-10): table speed-up against retained validation accuracy (Paper 3). Grey: the 1-epoch proxy q_1 for all 1,296 policies. Orange: 5-epoch q_5 for the 48 calibration policies. Dotted: q = 0.98; dashed: the 1.10× latency-room criterion.

Decision. The pre-registered latency-room rule required at least 20 proxy-feasible policies **and** one with a speed-up ≥ 1.10×; the first held (52), the second did not. **Because the latency-room requirement was not met, no hardware-aware PPO search was started.** The gap between 1.098× and 1.10× is a threshold fixed in advance, not a

meaningful discontinuity; the substantive finding is that, under the 2% accuracy constraint, the best proxy-feasible candidate reached a 1.098× speed-up, while the best confirmed five-epoch-feasible calibration policy reached 1.089×. Neither satisfied the pre-registered 1.10× GO criterion. The accuracy-feasible latency room on this CPU is therefore small, and for SimpleCNN it could not be established. No hardware-aware reinforcement-learning result exists; Paper 3 contributes a hardware analysis, not an RL success.

6.7 Phase 7A — Structured Transfer and Energy Evaluation

Question. Does the learned PPO allocation, realised structurally, reduce directly measured CPU energy, and at what accuracy cost? **Design** (Section 4.9.6): CIFAR-10 SimpleCNN and ResNet-8, each with **Dense** (recovered identically), **PPO-Structured** (the most frequent Phase-5 C1 policy transferred to structured units: SimpleCNN [0, 2, 5, 5], 3 of 20 seeds, widths [32, 51, 51, 102]; ResNet-8 [1, 1, 1, 3], 9 of 20 seeds, widths [14, 14, 29, 45], an approximate mapping), **Uniform-Structured** (MAC-matched, common ratios 0.256 and 0.149) and **LAMP-Structured** (from the Phase-6 LAMP mask at PPO's zero count). After the identical recovery, RAPL package energy was measured over five fresh-process sessions per architecture (Section 4.18); the stability pilot passed its 3% rule (dense CV 2.30% for SimpleCNN and 1.74% for ResNet-8).

Table 6.3: Phase 7A: accuracy, compression, latency and measured energy. Δ in pp against the recovered dense model on VAL-RL, the pre-registered split; latency, energy and EDP are medians over 5 sessions against the same-session dense anchor.

Architecture	Method	VAL-RL (Δ)	VAL-SELECT	Params ↓	MACs ↓	Latency (ms)	Speed-up	J / 1,000 inf.	Energy ↓	EDP
SimpleCNN	Dense	80.60 (0)	80.00	—	—	0.7313	1.00×	11.05	—	1.000
	PPO-Structured	75.52 (−5.08)	76.00	80.1%	42.6%	0.5711	1.28×	8.54	22.8%	0.603
	Uniform-Structured	79.12 (−1.48)	78.48	44.7%	42.5%	0.5341	1.37×	7.91	27.9%	0.518
	LAMP-Structured	78.84 (−1.76)	79.44	65.5%	19.7%	0.6579	1.12×	9.76	12.1%	0.783
ResNet-8	Dense	78.40 (0)	78.52	—	—	0.6984	1.00×	10.88	—	1.000
	PPO-Structured	75.16 (−3.24)	74.72	25.0%	21.5%	0.6400	1.09×	9.59	13.4%	0.791
	Uniform-Structured	75.60 (−2.80)	76.00	16.1%	19.3%	0.6459	1.07×	9.64	11.8%	0.818
	LAMP-Structured	75.24 (−3.16)	75.72	25.0%	18.2%	0.6490	1.07×	9.82	8.9%	0.828

Energy results. Every pruned model used less package energy than the dense model in 5 of 5 sessions (exact one-sided Wilcoxon signed-rank p = 0.031, the smallest attainable with five sessions). The pre-registered decision rests on that consistency and on the median saving exceeding the noise threshold of twice the dense between-session coefficient of variation (4.3% for SimpleCNN; 3.7% for ResNet-8); all six pruned models met both. PPO's per-session saving ranged from 19.5% to 25.7% on SimpleCNN and from 5.6% to 14.5% on ResNet-8, and core-counter and net-of-idle energy agreed with the package result to within 2 percentage points.

Latency and EDP. Every pruned model was faster than the same-session dense model, speed-ups tracking MAC reduction more closely than parameter reduction. Median package power was 14.6–15.6 W for every model. The normalised energy–delay product (EDP; lower is better) is relative to the same-session dense model; it is not a carbon metric.

6.8 Phase 7A — Accuracy–Energy Trade-off

Accuracy cost. Every pruned model lost VAL-RL accuracy (Table 6.3): **the pre-registered ±0.5 pp accuracy-preservation condition was not met by any pruned method.** On VAL-SELECT, PPO's losses were 4.00 pp (SimpleCNN) and 3.80 pp (ResNet-8).

Pre-registered verdicts: primary success, Case A (accuracy preserved and energy consistently reduced), **not met** for either architecture; SimpleCNN: Case B (energy reduced, with a measurable accuracy trade-off) **and Case D** (a baseline offers the better trade-off); ResNet-8: Case B.

PPO against Uniform on SimpleCNN. At almost identical MACs (42.6% vs 42.5% removed), MAC-matched Uniform-Structured was **3.60 pp more accurate** than PPO-Structured on VAL-RL. It also **used less energy in 5 of 5 sessions** (7.91 vs 8.54 J per 1,000 inferences; median per-session difference 8.0%). **PPO does not dominate Uniform in Phase 7A: on SimpleCNN, Uniform was better on both axes.** PPO concentrated its 80.1% parameter removal in conv3 and the hidden layer, which bought less time than Uniform's broad channel removal at the same MAC count (Section 6.5).

Other comparisons (descriptive: two-sided exact Wilcoxon tests on five session pairs, minimum attainable p = 0.0625). On SimpleCNN neither PPO nor LAMP dominates: PPO used less energy in 5 of 5 sessions (12.5% median difference) but was 3.32 pp less accurate. On ResNet-8, PPO, Uniform and LAMP showed **no clear difference**: against Uniform, accuracy differed by 0.44 pp and PPO used less energy in only 3 of 5 sessions; against LAMP, accuracy differed by 0.08 pp and PPO used less energy in 4 of 5.

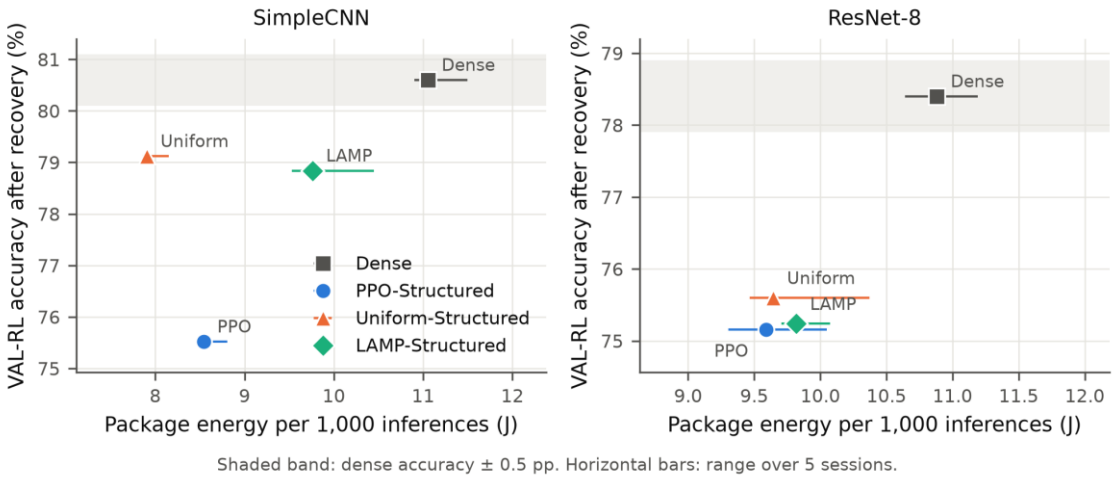


Fig. 6.3: Phase 7A: VAL-RL accuracy after recovery against measured package energy per 1,000 inferences. Medians with session ranges; no pruned model lies inside the shaded dense ± 0.5 pp band.

Interpretation. Structured realisation made the compression real: parameters, MACs, latency and directly measured package energy all fell, consistently across sessions. The learned masked allocation did not transfer well: its structured form lost 5.1 and 3.2 pp of VAL-RL accuracy, and on SimpleCNN plain uniform allocation was both more accurate and more energy-efficient.

Claim boundary.

(a) Phase 7A does not show accuracy-preserving energy reduction.

(b) It does not show that PPO is more energy-efficient than Uniform.

(c) It says nothing about the masked Phase-5/6 models themselves.

(d) The energy is CPU package energy on one laptop, not whole-system or lifecycle energy.

6.9 Motivation for Phase 7B

Phase 7A exposed a design problem, not a measurement problem: a policy learned for *masked* pruning, realised as channel removal and recovered, was neither accurate nor efficient enough, and for ResNet-8 the masked units do not match the structured units. Phase 7B was therefore designed **after** Phase 7A, as a separate variant that chooses structured actions directly; it was not part of the original framework.

6.10 Phase 7B — Deployment-Oriented Structured PPO

Design (Sections 3.16.3 and 4.9.7). The deployment-oriented structured PPO searches directly in the structured space of SimpleCNN on CIFAR-10, with an eight-feature state including centred structured sensitivity and the centred prior ($\beta = 1.0$). Its search reward used a *surrogate* (predicted) energy saving, so the name implies no measured energy result. Feasibility was judged by a 2-epoch recovery proxy on VAL-RL, accepted before the search (Spearman 0.948 against 5-epoch recovery; threshold $\hat{\tau} = 0.98849$; recall 1.00, precision 0.655). The surrogate, $\hat{E} = -0.117 + 13.762 \cdot T_{table}$ (J per 1,000 inferences, T in ms), was fitted to directly measured energy of 200 calibration policies and accepted by cross-validation (Spearman 0.996, median absolute percentage error 1.05%); the first candidate, MACs alone, had failed (Spearman 0.946 against the required 0.95; lowest-energy-quartile error 8.55% against 5%). The constraint-first reward used $\alpha = 4$ (fixed on development seeds), selection took the lowest-predicted-energy feasible policy, 20 confirmatory seeds (700–719) were run, and every final model received the equal 5-epoch recovery, with accuracy judged on VAL-SELECT, held out from search and selection. **The surrogate is not measured energy, and it is not evidence for the energy of any final Phase-7B model.**

Selection outcome. Every one of the 20 seeds selected a policy. The primary policy, [2, 2, 2, 5], was selected in **5 of 20** seeds; its widths **[26, 51, 102, 102]** remove 20% of the convolution channels and 60% of the hidden neurons. The six other policies (1–4 seeds each) were also recovered and evaluated. The primary policy's proxy retention, 0.9890, lies just above the threshold: the selections sit at the proxy feasibility boundary.

6.11 Phase 7B — Memory and Computation Results

Table 6.4: Phase 7B: accuracy, memory and computation after the equal 5-epoch recovery (SimpleCNN, CIFAR-10). Parameter memory = Σ numel × element size (float32). Latency and energy were **not measured**.

Model	Widths	VAL-SELECT (Δ pp)	VAL-RL (Δ pp)	Parameter memory	Memory ↓	MACs (M)	MACs ↓
Dense	[32, 64, 128, 256]	80.00 (0)	80.60 (0)	2,481,448 B	—	10.85	—
New PPO [2, 2, 2, 5]	[26, 51, 102, 102]	**78.88 (−1.12)**	80.00 (−0.60)	908,916 B	**63.4%**	6.94	**36.0%**
Uniform, r = 0.208	[25, 51, 101, 203]	78.76 (−1.24)	79.76 (−0.84)	1,555,908 B	37.3%	6.93	36.2%
LAMP, s = 56.2%	[32, 58, 103, 94]	79.64 (−0.36)	79.12 (−1.48)	909,932 B	63.3%	8.76	19.3%
Old PPO (7A) [0, 2, 5, 5]	[32, 51, 51, 102]	76.00 (−4.00)	75.52 (−5.08)	493,836 B	80.1%	6.23	42.6%

Table 6.4 compares the recovered model with dense, matched baselines and the old Phase-7A model. **Level 1:** the new PPO model needs **63.4% less parameter memory** (908,916 vs 2,481,448 bytes); its serialised checkpoint is 63.3% smaller. **Level 2:** it needs **36.0% fewer MACs** (6.94 M vs 10.85 M per image). The seven distinct per-seed selections all removed between 34.1% and 37.5% of the MACs, but their parameter-memory reduction ranged from 29.5% to 63.4%: the search rewarded predicted energy, which tracks latency and therefore MACs, and the selections differ mainly in hidden-layer neurons, which change memory a great deal and MACs very little (Section 6.5). **Latency was not measured for Phase 7B**, so Level 2 is established for MACs only; the Paper-3 table latency was a surrogate input.

6.12 Phase 7B — Accuracy Trade-off

Result on the pre-registered split (VAL-SELECT). The new PPO model reached 78.88% against the recovered dense model's 80.00%, a loss of **1.12 pp**; every other per-seed selection lost between 1.08 and 1.28 pp. On the same split and protocol the old Phase-7A PPO model lost **4.00 pp**, so the redesign reduced the loss by 2.88 pp. Phase 7A's 5.08 pp loss is on VAL-RL, its criterion split, and must not be compared with the 1.12 pp VAL-SELECT figure; on VAL-RL the new model's loss was 0.60 pp, against 5.08 pp for the old model.

Joint deployment-efficiency success criterion (named GREEN-SUCCESS in the pre-registration, Section 4.9.7): **met by 0 of 20 seeds.** Every selected policy lost more than

the allowed 0.5 pp on VAL-SELECT, which alone decides the outcome; the latency and energy conditions could not be assessed, because no measurement session took place (Section 6.13). Descriptively, the new PPO model is 0.12 pp more accurate than Uniform on VAL-SELECT at similar MACs (36.0% vs 36.2% removed) and much lower memory (63.4% vs 37.3%), and 0.76 pp less accurate than LAMP while removing more MACs (36.0% vs 19.3%) and about the same memory (63.4% vs 63.3%). LAMP-Structured is the only pruned model within 0.5 pp of dense on VAL-SELECT (−0.36 pp); its joint criterion is not assessable, because energy was never measured.

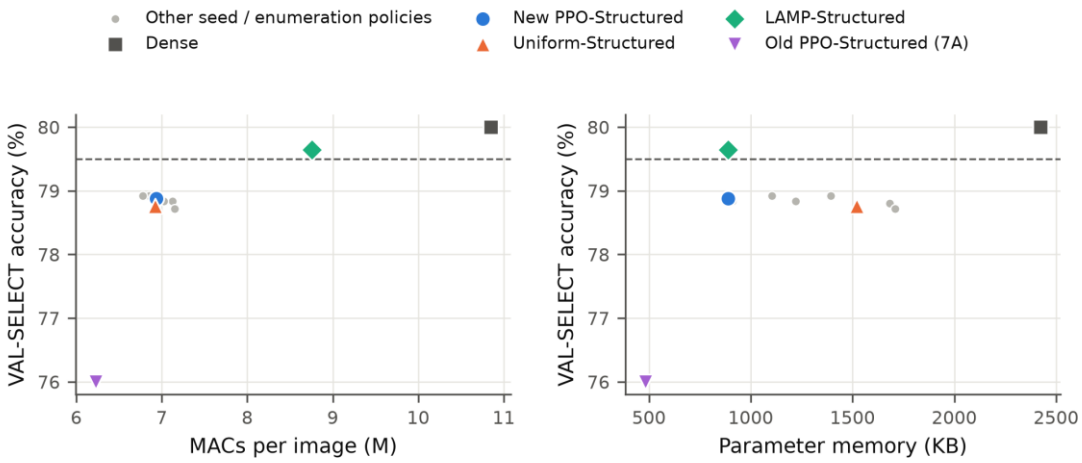


Fig. 6.4: Phase 7B: VAL-SELECT accuracy after the equal 5-epoch recovery against MACs per image (left) and parameter memory (right). Dashed: dense accuracy − 0.5 pp (the joint criterion's accuracy condition); grey: the other per-seed selections and the enumeration reference.

Interpretation. Learning directly in the structured space substantially improved the memory–computation–accuracy trade-off over transferring a masked policy, but did not reach the 0.5 pp criterion; that the selections sit at the proxy feasibility boundary (proxy precision 0.655) is consistent with the shortfall, although the data do not isolate the proxy as its cause.

Claim boundary.

(a) Phase 7B shows no measured latency or energy reduction.

(b) It is not accuracy-preserving.

(c) It does not beat Uniform or LAMP.

(d) The joint deployment-efficiency criterion was not achieved.

6.13 Direct Energy Confirmation and Measurement Failure

Both attempts to measure Phase-7B energy directly stopped at the pre-registered stability check, before any model comparison.

Attempt 1, the Phase-7B confirmatory pilot (dense model only; limit CV ≤ 3%; N may be doubled once, then STOP). The first pilot (N = 100,000 per block) gave a coefficient of variation of the six dense blocks of **11.36%**, so N was doubled; the doubled-N attempt (200,000) was voided by recorded user decision after the CPU-load gate stopped it during foreground activity; and the permitted rerun with the laptop idle (200,000) gave **4.01%**, still unstable, so **hardware validation STOPPED**. No confirmatory energy session was run.

Attempt 2, the separate energy confirmation study. The protocol was committed before any data, for the frozen Phase-7B models (all five checkpoint hashes verified), with a fixed 180 s thermal soak and 200,000 inferences per block. Qualification used the dense model only, 3 fresh runs × 2 blocks = **6 blocks**: energy per 1,000 inferences ranged from 9.47 to 15.22 J (mean 12.14 J), a coefficient of variation of **19.92%** against the **3%** criterion, although all runs passed the validity gates (in-block system load 7.6–10.2%, below the 15% limit). The setup was **not qualified**; there was one permitted attempt and the limit was not relaxed. **No Dense-versus-PPO paired session ran.**

Result. Direct energy confirmation for Phase 7B remains unresolved. No estimate from latency or from the surrogate is substituted.

What the qualification recorded (descriptive). Median latency stayed within 1.042–1.052 ms across the six blocks, but package power ranged from 9.1 to 14.5 W (core PP0 6.7 to 9.7 W, the rest of the package 2.4 to 5.1 W) and the thermal-zone reading from 72 to 92 °C, the coolest run giving the lowest energy: the instability lay in the package's power draw under identical work, not in inference timing. Its source is not identified. The same software counter had passed its pilot in Phase 7A (CV 2.30% and 1.74%); the thesis records this contrast without claiming a cause. A software counter on a laptop is not a guaranteed-stable instrument, which is why each study had to qualify it (Jay et al., 2023; Raffin and Trystram, 2025).

6.14 Three-Level Sustainability Assessment

Table 6.5 brings the three levels together for Phases 7A and 7B, with the accuracy cost of each.

Table 6.5: Three-level deployment-efficiency assessment.

Level	Metric	Phase 7A evidence (structured transfer)	Phase 7B evidence (deployment-oriented structured PPO)	Conclusion
1. Compression / memory	parameter count / parameter memory; serialised size	parameters −80.1% (SimpleCNN) and −25.0% (ResNet-8)	parameter memory −63.4%; serialised −63.3%	**Established** in both phases
2. Computation / latency	MACs; measured batch-1 latency	MACs −42.6% / −21.5%; **measured** speed-up 1.28× / 1.09×	MACs −36.0%; latency **not measured**	MACs established in both; latency measured for 7A only. Paper 3: MACs are a strong latency proxy, parameters a weak one
3. Operational energy	RAPL package J per 1,000 inferences; EDP	**Measured:** −22.8% / −13.4%, lower in 5/5 sessions; EDP 0.603 / 0.791	**Unresolved:** pilot CV 11.36% → 4.01%; confirmation CV 19.92% (limit 3%)	Real energy reduction demonstrated in 7A only
Accuracy (criterion ±0.5 pp)	recovered validation accuracy	**Failed:** VAL-RL −5.08 / −3.24 pp; no pruned method within 0.5 pp; Uniform dominated PPO on SimpleCNN	**Failed:** VAL-SELECT −1.12 pp; 0/20 seeds met the joint criterion	The accuracy trade-off was too large in 7A and reduced, but not removed, in 7B

Level 1 does not imply Level 2: the Phase-7A SimpleCNN PPO model removed 80.1% of the parameters but only 42.6% of the MACs, and MAC-matched Uniform ran faster. **Level 2 tracked Level 3 in Phase 7A,** because package power was nearly constant across models. **Level 3 must be measured:** in Phase 7B the instrument could not be qualified, and neither the MAC reduction nor the energy surrogate substitutes for a measurement. A deployment saving bought with a material accuracy loss is not presented as a sustainability result.

6.15 Answer to RQ6

Compression can translate into real deployment savings, but not automatically, and not at every level at once.

(i) **Masked sparsity alone gives no hardware evidence**; physical structured pruning is needed for dense-kernel CPU cost to fall.

(ii) **Even structurally, parameter reduction is a weak guide to latency**: across 5,184 measured structured policies, Spearman ρ was 0.31–0.55 for parameter reduction and 0.89–0.98 for MAC reduction, and in 31–39% of policy pairs the sparser policy was slower.

(iii) **Accuracy-feasible headroom is small**: for ResNet-8, the best proxy-feasible candidate reached a 1.098× speed-up, while the best confirmed five-epoch-feasible calibration policy reached 1.089×. Neither satisfied the pre-registered 1.10× GO criterion, so the hardware-aware search was not started.

(iv) **Phase 7A: real energy savings, with material accuracy loss**: directly measured CPU package energy fell by 22.8% (SimpleCNN) and 13.4% (ResNet-8), with VAL-RL losses of 5.08 and 3.24 pp; on SimpleCNN, MAC-matched uniform allocation was both more accurate and more energy-efficient.

(v) **Phase 7B: a much better trade-off, but neither criterion met**: parameter memory fell by 63.4% and MACs by 36.0% at a 1.12 pp VAL-SELECT loss (4.00 pp for the transferred policy on the same split); no seed met the 0.5 pp accuracy criterion, and direct energy confirmation failed its stability qualification.

Deployment benefit must therefore be established separately for memory, computation, latency and energy, each with its accuracy cost: an improvement at one level is a hypothesis about the next, not evidence for it.

6.16 Chapter Summary

Parameter reduction is a weak latency proxy and MAC reduction a strong one, and ResNet-8's accuracy-feasible latency headroom was too small for a hardware-aware search (Paper 3). Structured pruning reduced directly measured CPU package energy, but with VAL-RL losses beyond the pre-registered limit (Phase 7A). Learning in the structured space improved the memory–computation–accuracy trade-off, but no seed met

the joint criterion and its energy remains unresolved (Phase 7B). Chapter 7 discusses these results.

CHAPTER 7
DISCUSSION

7.1 General

Chapters 5 and 6 reported the core pruning experiments (Phases 3–6) and the hardware realisation and measurement of the models (Paper 3, Phases 7A and 7B, and the energy confirmation study). This chapter interprets that evidence across phases, relates it to Chapter 2, and states where the framework succeeds and where it does not.

No new numbers are introduced, and causal language is reserved for effects the design isolated, such as the C1-versus-C2 contrast of Phase 5; where only an association was observed, the text says so.

7.2 What Adaptive Layer-Wise PPO Actually Contributes

The evidence (Section 5.3). PPO learned visibly non-uniform allocations, pruning large, low-sensitivity layers heavily and keeping small early layers almost intact (Figure 5.2). At the sparsity it chose, these were far more accurate than uniform allocation (up to about +31.4 pp) and random allocation; against global magnitude and ERK the picture was mixed; and LAMP was significantly more accurate than PPO before recovery in nearly every setting.

What this establishes.

(a) **The agent solves the allocation problem it was given.** Without any hand-written allocation rule it discovers that layers should be pruned differently, confirming the premise behind RL-based compression (He et al., 2018) that layer-wise allocation is learnable from a scalar reward.

(b) **It does not establish superiority.** The thesis does not show that a learned allocation beats the best deterministic rule; the comparison with LAMP points the other way at the sparsities studied.

Where the agent's value lies. One interpretation, supported by the later phases, is that PPO's value lies in its *flexibility*: Phases 4, 5 and 7B changed the prior, the reward and the action space while the agent stayed the same, whereas a fixed rule such as LAMP optimises one implicit criterion. This evidence is indirect: PPO *can* be pointed at a new

objective and will pursue it (in Phase 5 this paid off for ResNet-8), but no objective tested showed a better final model than a tailored deterministic method. RL pruning should therefore be presented as an adaptive optimisation mechanism for objectives that are hard to express as a single saliency score, not as a replacement for strong deterministic pruning, nor as necessary for pruning in general.

7.3 The Role of Sensitivity Information

The pattern (Sections 5.4, 5.5 and 5.8). Across three phases, sensitivity guidance improved the *search* but not the *final model*. The prior sampled better-ranked policies earlier, with fewer destructive ones, in every phase; in Phase 4 the centred prior beat its shuffled, magnitude-matched and sign-reversed controls on reward-rank AUC; yet no final-accuracy benefit was found in any phase, and under the corrected reward C1's test accuracy was slightly *below* C2's in every setting (descriptive).

The strongest defensible reading. Sensitivity is useful as a **search bias**: its measurable contribution lies in exploration guidance, not in predicting which final model will be most accurate. A natural explanation, which the thesis can describe but not prove, is that the prior shapes *which* policies are sampled early, whereas the returned policy is decided by the reward landscape and the selection rule. In these small spaces the search finds near-optimal regions with or without the prior; a better route to the same destination need not change the destination.

Why centring mattered. All three Phase-3 priors shared the same total pressure towards lighter pruning, so part of any benefit could be generic conservatism. Centring removes that net pressure by construction, so its advantage over permuted and magnitude-matched controls is stronger evidence for a layer-relative exploration effect than Phase 3 gave, although not complete causal isolation. On the accuracy-based exploration measure, however, the *conservative* control was significantly better than the centred prior in most settings, consistent with aggressive actions on insensitive units raising the reward rank of sampled policies while lowering their accuracy. Centring made the prior more layer-specific; it did not make it a better predictor of final accuracy.

Implication for the literature. Sensitivity is often treated as a direct proxy for how much a layer can be pruned; the dissociation found here suggests evaluating sensitivity-based

methods separately on the *search* and on the *outcome*. Its contribution is of a different kind than often assumed, not absent.

7.4 Search Quality versus Policy Retention

The finding (Section 5.6). One of the clearest results of the thesis is that **finding a good policy and ending on it are different problems.** PPO's best visited policy almost always reached the top 10 of the 1,296 policies, whereas the terminal policy almost never did; archive selection restored retention on validation ranking (80–100% top-10 reach). A possible mechanism, not isolated, is that a stochastic policy-gradient method updating on noisy estimates may sample an excellent policy early and then settle on a region of high average reward that does not contain it.

What the finding establishes is methodological. A study that reports only the terminal policy measures retention together with search; had the thesis done so, it would have concluded that PPO searches poorly, whereas the exhaustive landscapes showed that it searches well and retains poorly. This mirrors the gap between search and final selection in architecture search (Yu et al., 2020) and the value of archives of visited states in exploration (Ecoffet et al., 2021). RL pruning studies should report the best visited policy, the retained policy and the selection procedure that links them, separately.

Elite behavioural cloning. An imitation term towards the best policies seen so far, in the spirit of self-imitation learning (Oh et al., 2018), improved the final-selected rank in only half the settings, against all six for the archive rule; with one configuration tested, this does not show that behavioural cloning is ineffective in general.

The boundary. Archive selection improves *validation-rank* retention. Its test effect was mixed, with large losses on ResNet-8, where the archive retained exactly what the reward ranked highest: an over-pruned model. It is a retention mechanism, not a generalisation mechanism, and it amplifies whatever the reward specifies.

7.5 Reward Design and Over-Pruning

A stronger optimiser exposes the reward (Section 5.7). Once search and retention improved in Phase 4, the agent reliably reached the optimum of R0, whose fixed exchange rate between accuracy and sparsity makes that optimum an accuracy-losing

policy on ResNet-8. The optimiser did not fail; it optimised what the reward asked for, whereas a weaker search had hidden the misalignment. This is a pruning instance of the observation that more capable optimisers exploit misspecified objectives more strongly (Pan, Bhatia and Steinhardt, 2022; Skalse et al., 2022). Search performance must be judged together with objective quality, and a pruning reward must encode the acceptable accuracy–compression boundary explicitly.

The accuracy-constrained reward as a correction (Section 5.8). Phase 5 made accuracy a constraint rather than a traded quantity, in the spirit of constrained reinforcement learning (Achiam et al., 2017). The C1-versus-C0 comparison isolates the reward as the manipulated factor: on ResNet-8 the accuracy loss fell markedly, at more moderate sparsity. On SimpleCNN it was slightly *worse*, LAMP remained more accurate in all six settings, and the constraint held on test in only 67.5% of runs. A validation-based constraint is therefore an effective **optimisation constraint**, not a **generalisation guarantee**.

7.6 Validation-to-Test Transfer

The gap (Section 5.9). Almost every constrained policy passed the constraint on the selection split, but only about two thirds remained feasible on test, and none in CIFAR-100 ResNet-8. Three explanations are plausible, none isolated:

(i) **Validation sampling variability:** the sampling noise of the validation split is of the same order as the 2% relative margin.

(ii) **Selection at the boundary:** the constrained reward drives the agent to the *most compressed* feasible policy, which sits close to the threshold by design.

(iii) **Architecture and dataset dependence:** the failures concentrated where the feasible region is small and the dense accuracy low.

Constraint-based pruning therefore needs margins, uncertainty-aware constraints or larger selection splits before a validation-certified constraint can be read as a promise about unseen data. Equal recovery removed most of the gap for this pipeline (Phase 6), but that is a property of recovery, not of the constraint.

7.7 Why LAMP Remains a Strong Benchmark, and Why Equal Fine-Tuning Changes the Rankings

7.7.1 LAMP

LAMP (Lee et al., 2021), deterministic and search-free, was the strongest method before recovery throughout the core framework. Its strength calibrates the thesis rather than invalidating it. One untested interpretation is that LAMP's rescaled magnitude score, which sets layer-wise sparsity automatically (Lee et al., 2021), already captures much of what a learned allocation can achieve under an ordinary compression objective. RL must then justify its additional cost: repeated pruned-model evaluations (512 per run), many seeds, reward design, prior calibration, archive management and selection splits, against one pass over the weights for LAMP. An adaptive search becomes the more natural tool when the objective includes accuracy constraints, deployment metrics or competing goals; the thesis does not show that this advantage materialises, and Phase 7B does not prove it (Section 7.10).

7.7.2 Equal fine-tuning

What changed (Section 5.10). Phase 6 changed the comparison more than any other experiment. After an identical five-epoch, fixed-mask recovery, the PPO–LAMP difference fell to 0.00–0.22 pp, within the pre-registered ±0.5 pp confidence-interval margin in all six settings, and PPO's large advantage over global magnitude almost vanished. This is a decision rule, not a formal equivalence test, and LAMP remained slightly but significantly ahead in three settings; the methods are not equivalent, their differences became small.

One interpretation. Much of the apparent advantage of one pruning rule over another reflects how much accuracy the *initial* masked model preserves: recovery appears to compensate most where the initial loss was largest. Because the methods differ only in their masks, with shared seeds and data streams, the narrowing is an effect of equal recovery; the mechanism inside fine-tuning is not isolated.

This is the thesis's most important methodological implication for pruning comparisons. Methods should be compared after equal post-pruning recovery budgets, not only immediately after pruning; otherwise the comparison partly measures how much

each method damages the network before the damage is repaired, consistent with work showing that retraining choices can change pruning conclusions (Blalock et al., 2020; Renda, Frankle and Carbin, 2020; Le and Hua, 2021). The claim holds for small CNNs at the moderate sparsities PPO selected under the accuracy constraint, with a five-epoch budget; it does not extend to very high sparsity, ImageNet-scale models, transformers or large models, where recovery may compensate less.

7.8 Compression versus Real Deployment Efficiency

Masked versus structured pruning. Masked pruning suits search and analysis, but masked zeros stay inside dense tensors: without sparse kernels, a masked model is not smaller or faster on a CPU (Hoefler et al., 2021), and the thesis makes no latency or energy claim for its masked models. Only structured pruning lets parameters, MACs, latency and energy fall physically; it is coarser, and in Phase 7A it cost several points of accuracy.

Sparsity is not speed (Sections 6.4–6.5). Parameter reduction correlated only weakly with measured CPU latency gain and MAC reduction strongly, and in roughly a third of policy pairs or more the more parameter-sparse policy was slower. In SimpleCNN the reason is arithmetic: the hidden fully connected layer holds most of the weights but little of the computation, and the early convolutions are the reverse. This is consistent with the observation that cost indicators can disagree (Dehghani et al., 2022); the thesis quantifies the disagreement exhaustively for small structured CNNs on one CPU.

Accuracy-feasible headroom (Section 6.6). Under an accuracy constraint the useful latency headroom shrank sharply: for ResNet-8 the best proxy-feasible candidate reached 1.098× and the best confirmed five-epoch-feasible calibration policy 1.089×, so neither satisfied the pre-registered 1.10× GO criterion; for SimpleCNN none could be established, because its proxy failed validation. **Unconstrained speed-ups say little about the speed-ups available to a model that must keep its accuracy.**

7.9 Accuracy–Efficiency Trade-offs and the Three Levels of Efficiency

The deployment results are best read through the three levels of Chapter 3: compression and memory (Level 1), computation and latency (Level 2), and measured operational energy (Level 3). **Each level needs its own evidence:**

(i) **Memory does not guarantee latency.** In Phase 7A the SimpleCNN PPO model removed most of the parameters but was slower than a uniform model that removed far fewer, the same weak link Paper 3 shows across all its shapes.

(ii) **MACs predict latency better, but do not replace measurement.** The correlation is strong but below 1, and at nearly equal MACs Uniform was still faster than PPO in Phase 7A.

(iii) **Latency does not mathematically guarantee energy.** Energy is power times time, so a faster model saves energy only if its average power does not rise to compensate. In Phase 7A average package power was nearly the same for every model, so energy savings followed latency savings; that is an observation about this platform, not a law.

Accuracy must be reported at every level. The pre-registered criteria of Phases 7A and 7B required accuracy within 0.5 pp of dense. **No pruned model met that requirement in either phase, except LAMP-Structured in Phase 7B, whose energy could not be measured.** Every deployment saving in the thesis therefore comes with an accuracy cost, and is reported with it.

7.10 Interpreting the Sustainability Experiments

Phase 7A: real energy savings, but not a better model (Sections 6.7–6.8). Phase 7A firmly establishes that physically structured pruning reduced **directly measured** CPU package energy per inference, in all five sessions. But energy savings alone are not evidence of a better model: the SimpleCNN PPO model lost 5.08 pp of VAL-RL accuracy, and a MAC-matched uniform model was both more accurate and lower in energy. Phase 7A shows that structural compression reaches the hardware, not successful sustainability-oriented optimisation by PPO, and that a transferred masked allocation is a poor structured allocation, plausibly because the masked policy concentrated pruning in

the hidden layer, where it costs little accuracy *as a mask* but is neither accurate nor computationally efficient as channel removal (consistent with Paper 3, not isolated).

Phase 7B: what improved (Sections 6.10–6.12). The **deployment-oriented structured PPO** of Phase 7B chose physical channel ratios, and its search was guided by a validated energy surrogate rather than by measured energy. It substantially improved the memory–computation–accuracy trade-off (parameter memory −63.4%, MACs −36.0%, a 1.12 pp selection-split loss against 4.00 pp for the old policy on the same split), supporting the design lesson that **an allocation should be learned in the space where it will be deployed.**

Phase 7B: what it did not solve. It lost more than the 0.5 pp the pre-registration allowed, and 0 of 20 seeds met the joint deployment-efficiency criterion. Its latency was not measured and its energy is unresolved. Against the MAC-matched uniform and parameter-matched LAMP structured baselines it showed no clear overall advantage (Section 6.12). Phase 7B improved part of the deployment objective; it did not solve the full objective.

The energy surrogate. The surrogate ranked calibration policies very well and made the search affordable, but surrogate validity does not turn predicted energy into measured energy; final deployment claims require direct validation on the final model.

The measurement failure (Section 6.13). Phase 7B's energy remained unresolved because the measurement setup repeatedly failed its pre-registered stability check (3% limit), while latency stayed stable within about one percent; the thesis does not claim to know the cause and does not generalise this beyond the tested system. Because the same counter on the same machine had passed its check in Phase 7A, qualification must be repeated for every measurement campaign. **It is evidence neither for nor against the Phase-7B model's energy use**: no model comparison was made, and only the dense model was measured. Studies of software energy meters likewise report that readings need careful protocol design and validation (Khan et al., 2018; Jay et al., 2023; Raffin and Trystram, 2025).

On sustainability wording. The thesis does not establish that any of its models is environmentally beneficial, lower in carbon footprint or suitable for "green" deployment. It establishes measured operational CPU energy savings for the Phase-7A structured

models, with their accuracy costs, and, for Phase 7B, memory and computation reductions with energy unresolved. Lifecycle, training-energy and carbon accounting were outside its scope.

7.11 What the Negative Results Establish

The negative and partial results, which carry much of the thesis's methodological weight, are summarised in Table 7.1.

Table 7.1: Negative and partial results and what each establishes.

Negative or partial result	What it establishes
PPO did not beat LAMP before recovery (5 of 6, then 6 of 6 settings)	RL must justify its cost through flexibility or richer objectives
Sensitivity did not improve final accuracy in any phase	Search improvement and outcome improvement must be evaluated separately
Archive selection had mixed test effects	Retention is not generalisation; it amplifies whatever the reward specifies
Validation feasibility transferred to test in 67.5% of runs	An optimisation constraint is not a generalisation guarantee
Hardware-aware RL was NO-GO	Accuracy-feasible latency room can be too small to justify a latency-driven search
Uniform beat PPO on SimpleCNN in Phase 7A	A transferred masked allocation is a poor structured allocation; deployment metrics can reverse rankings
Phase 7B missed the 0.5 pp criterion (0 of 20 seeds)	Learning in the deployment space narrows, but does not close, the accuracy gap at these compression levels
Phase-7B energy remained unresolved	Energy claims require a qualified instrument; MAC reductions and surrogates do not substitute for measurement

Each of these results came from an experiment whose decision criteria were fixed in advance, and each was kept as observed. Several changed the design of the next phase: retention led to the archive, over-pruning to the constraint, and the lossy transfer to Phase 7B. The positive findings are credible partly *because* the negative ones are visible; that is a logical point about evidential weight, not an empirical causal claim.

7.12 Generalisability

The evidence base. The core framework used CIFAR-10 and CIFAR-100, three small CNNs, four pruning units per network and an action space of 1,296 policies; the deployment extension used a subset of these settings on one laptop CPU (batch 1, one

thread). Within that scope the main qualitative results recur across both datasets and all three architectures; the deployment findings are specific to one platform. **Outside that scope** (ImageNet-scale data, larger CNNs, transformers, other hardware, batch sizes or thread counts) the thesis provides no evidence. At larger scale the policy space cannot be enumerated, so retention and search quality cannot be measured against ground truth and the case for a learned search may become stronger, while recovery may compensate less at high sparsity.

7.13 Limitations

The limitations below bound every claim in Chapters 5–7.

A. Model and dataset scale. CIFAR-scale data and small CNNs only; with four units and six ratios the space was small enough to enumerate, which made exhaustive ground truth possible but makes the RL problem easier than at realistic scale. The sustainability phases used CIFAR-10 only; Phase 7B, SimpleCNN only.

B. Optimisation. PPO's search cost is substantial. Sensitivity guidance improved exploration but not final accuracy, and one Phase-4 exploration measure reversed. LAMP remained stronger before recovery and slightly ahead in three settings after it. The results depend on reward design: no Phase-4 redesigned reward met its criterion, and the constrained reward made SimpleCNN slightly worse. Elite behavioural cloning was tested at one configuration only.

C. Validation methodology. Validation-to-test transfer of the accuracy constraint was imperfect (67.5% test feasibility, 0% in CIFAR-100 ResNet-8), and the fixed relative threshold ($\tau = 0.98$) and small selection splits make feasibility sensitive to sampling noise. The deployment phases used a single recovery seed per model; one dense reference (CIFAR-10 SimpleCNN) used an older training recipe; and Phase 6 was not a fully independent test confirmation, since the test sets had been read for earlier frozen models. The ±0.5 pp rule is a confidence-interval decision rule, not a formal equivalence test, and recovery used one fixed five-epoch protocol; longer or tuned recovery might change the rankings.

D. Hardware. One CPU platform, one batch size and one thread configuration; no GPU, mobile, embedded or accelerator evaluation. The SimpleCNN accuracy proxy failed

validation, so its accuracy-feasible latency headroom is unknown, and the exhaustive 5-epoch recovered landscape was stopped for compute cost.

E. Energy. CPU package energy only, with no DRAM, whole-system or external power measurement. The Phase-7B measurement setup failed its stability qualification repeatedly, so no Phase-7B energy or latency claim is made. The energy surrogate and the accuracy proxy are model-selection tools, not measurements.

F. Sustainability. No carbon accounting, no lifecycle or embodied-energy assessment, and no analysis of training or search energy. Phase-7A deployment-scale energy figures are projections, not results.

7.14 Overall Scientific Contribution

The central contribution. The novelty of this thesis does not lie in a claim that PPO beats existing pruning methods; it does not. It lies in **an integrated, systematically audited diagnostic framework, with pre-specified and pre-registered decision criteria across its later experimental stages**, which follows one pruning approach from allocation to measured hardware energy and isolates a separate question at each stage: allocation (RQ1), sensitivity-guided exploration (RQ2), policy retention (RQ3), reward alignment (RQ4), equal recovery (RQ5), and deployment translation and direct energy validation (RQ6). Tables 1.1 and 8.1 map these stages to the research questions and contributions.

The research questions thus form a chain: each answer changed what the next had to control for. Effective pruning is **not a single optimisation problem** but a chain of coupled problems, and a weakness at any link can hide or reverse improvements elsewhere: a good search under a misaligned reward produced worse models; a retained policy did not generalise; a good mask was a poor structure; and a strong MAC reduction remained unconfirmed as an energy saving. **This multi-stage view, supported at each stage by experiments whose decision criteria were fixed in advance, and reported including their negative results, is the thesis's central scientific contribution.** Its practical implications are drawn together in Section 8.4.

7.15 Chapter Summary

PPO learns a flexible layer-wise allocation that clearly beats uniform and random allocation but not LAMP; sensitivity improves exploration but not the final model; an archive restores validation-rank retention without improving generalisation; an accuracy constraint controls over-pruning without guaranteeing test feasibility; and equal recovery narrows method differences to small values. In deployment, sparsity is not speed, accuracy-feasible headroom is small, Phase 7A's measured energy savings carried a material accuracy cost, and Phase 7B improved the memory–computation–accuracy trade-off but missed the accuracy criterion, with its energy unresolved.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 General

This chapter answers the six research questions, states the contributions and the conclusion, and sets out the limitations and future work. Every number below was reported and audited in Chapters 5 and 6.

The thesis set out to learn layer-wise pruning allocations with Proximal Policy Optimisation (PPO), guided by layer sensitivity, and to find out what such a framework needs in order to produce a better and cheaper model. Each experiment was designed in response to what the previous one revealed: the masked-pruning core framework (RQ1–RQ5) added a centred sensitivity prior, archive selection, an accuracy-constrained reward and equal fine-tuning; the deployment extension (RQ6) added exhaustive CPU latency analysis, direct energy measurement, a deployment-oriented structured PPO and a pre-registered energy-confirmation attempt.

8.2 Conclusions by Research Question

RQ1. Does PPO learn non-uniform layer-wise pruning allocations that outperform simple allocation strategies, and how does it compare with a strong deterministic method such as LAMP?

Yes against simple strategies; no against LAMP. PPO learned meaningful non-uniform allocations that were much more accurate than uniform pruning in five of six settings, by up to approximately +31.4 percentage points of test accuracy, and than random pruning in all six; against global magnitude and ERK the comparison was mixed. Before equal recovery, LAMP was significantly more accurate than PPO in five of six settings in the first comparison and in all six for the later pipelines. The learned allocator is effective, but not universally better than LAMP.

RQ2. Does layer-sensitivity information improve the pruning search process, and does that improvement translate into higher final predictive accuracy?

It improves search measures, not the final model. Across Phases 3–5, sensitivity guidance improved one or more search or exploration measures (better-ranked policies

earlier, destructive policies less often), but these improvements did not translate into reliable gains in final predictive accuracy; under the corrected reward the prior-guided pipeline was marginally less accurate on test than the same pipeline without the prior. Its role is narrower than often assumed, not absent.

RQ3. Does PPO retain the strongest pruning policies encountered during search, and can archive-based policy selection improve retention?

PPO does not retain them; an archive restores retention on validation ranking. The terminal policy reached the top 10 of 1,296 policies in 0% of runs in five settings and 5% in the sixth, although the best visited policy was there in 95–100% of runs. Archive-based selection raised top-10 reach to 80–100%, mainly through the archive rule itself rather than through imitation during training. Its effect on test accuracy was mixed, with large losses on ResNet-8, where the archive retained exactly the over-pruned policies the reward preferred. Archive selection solves a retention problem, not a generalisation problem.

RQ4. How does reward design influence pruning aggressiveness and the accuracy–compression trade-off?

Reward design materially controls how aggressively PPO prunes. The original fixed-rate reward has an optimum that over-prunes fragile networks, visible only once search and retention improved. Under an otherwise identical pipeline, the accuracy-constrained reward cut the mean loss on CIFAR-10 ResNet-8 from 7.32 to 1.18 percentage points, but it was slightly worse on SimpleCNN, LAMP remained more accurate, and validation-certified feasibility held on the test set in only 67.5% of runs. A validation constraint controls the search; it does not guarantee behaviour on unseen data.

RQ5. How does equal post-pruning fine-tuning affect comparisons between PPO and deterministic pruning methods?

Equal five-epoch, fixed-mask recovery substantially narrowed the differences between methods. The PPO–LAMP difference fell from 0.26–0.53 to 0.00–0.22 percentage points, within the pre-registered ±0.5 percentage-point confidence-interval decision margin in all six settings. This margin is a decision rule, not a formal statistical equivalence test, and LAMP remained significantly, though slightly, ahead in three

settings. PPO's advantage over global magnitude, large before recovery, fell to at most 0.12 percentage points. Methods should therefore be compared under equal recovery budgets.

RQ6. Do compression gains translate into real deployment benefits in memory, computation, latency and measured operational energy?

Not automatically.

(a) **Latency proxies.** Across every structured policy measured, parameter reduction was a weak proxy for CPU latency (Spearman 0.31–0.55), and MAC reduction a much stronger one (0.89–0.98). Measured latency remained necessary.

(b) **ResNet-8 headroom.** Under the accuracy constraint, the best proxy-feasible candidate reached a 1.098× speed-up and the best confirmed five-epoch-feasible calibration policy reached 1.089×. Neither met the pre-registered 1.10× GO rule, so no hardware-aware search was started.

(c) **Phase 7A, measured energy with an accuracy cost.** The structured realisation of the learned allocation reduced directly measured CPU package energy by 22.8% (SimpleCNN) and 13.4% (ResNet-8), consistently across sessions, but lost 5.08 and 3.24 percentage points of VAL-RL accuracy; on SimpleCNN, MAC-matched uniform pruning was both more accurate and lower in energy.

(d) **Phase 7B, a better trade-off without an energy result.** The deployment-oriented structured PPO reduced parameter memory by 63.4% and MACs by 36.0%, at a validation-accuracy cost of 1.12 percentage points. No seed (0 of 20) met the joint deployment-efficiency criterion, its latency was not measured, and its energy remained unresolved because the measurement setup failed its stability qualification.

Compression can produce real deployment savings, but each dimension must be measured with its accuracy cost.

8.3 Main Contributions

The six thesis contributions (TC1–TC6) are stated in Table 8.1 in final form, each with its boundary.

Table 8.1: Thesis contributions and their boundaries.

#	Contribution	Boundary
TC1	**Adaptive PPO-based layer-wise pruning**, evaluated against Uniform, Random, global magnitude, ERK and LAMP at identical sparsity on two datasets and three architectures	Strongly better than uniform and random allocation; not superior to LAMP
TC2	**An analysis of sensitivity-guided exploration**, with absolute and centred priors against shuffled, constant, magnitude-matched and sign-reversed controls	Improves the search; no consistent final-accuracy gain
TC3	**A diagnosis of policy retention in RL pruning**, against exhaustive ground truth, and evaluation of archive-based selection	Validation-retention benefit; mixed effect on test accuracy
TC4	**A diagnosis of reward misalignment**, and evaluation of an accuracy-constrained, validation-aware reward	Better control of over-pruning; test feasibility not guaranteed
TC5	**An equal-recovery methodology:** identical fixed-mask fine-tuning, shared seeds and data, at identical sparsity	Small post-recovery differences do not constitute formal equivalence
TC6	**A deployment-efficiency analysis** that separates parameter compression, MAC reduction, measured latency and directly measured operational energy, including a qualified energy-measurement protocol and its reported failure	Phase-7A energy savings carry a material accuracy cost; Phase-7B energy remains unresolved

Together, they examine adaptive pruning at each stage from allocation to hardware validation.

8.4 Overall Thesis Conclusion

Adaptive neural-network pruning is not only a problem of selecting which weights or layers to remove. It is a chain of connected decisions: how pruning is allocated across layers, how the search is guided, whether good policies are retained, whether the reward matches the true objective, how post-pruning recovery is controlled, and whether compression translates into hardware benefit.

Improvement at one stage does not guarantee success at the next. Better search exposed a poor reward; better validation selection transferred to the test set only partly; lower parameter count did not mean lower latency; **lower energy came with accuracy loss exceeding the pre-registered limit**, and a simpler allocation did better; and the structured model with the best trade-off could not have its energy measured reliably.

Implications for practice. Compare methods only after an equal recovery budget; report the best visited policy separately from the returned one; write the acceptable accuracy loss into the objective; confirm any constraint on held-out data; and treat memory, computation, latency and energy as separate claims, each measured and reported with its accuracy cost.

This multi-stage view is the central conclusion of the thesis. A pruning result becomes a deployment-efficiency result only when every link has its own evidence and accuracy is reported alongside each efficiency gain; the negative results are part of that conclusion, each identifying the next link to control.

8.5 Limitations

The limitations (Section 7.13) are summarised by group.

(a) **A. Scale.** CIFAR-10 and CIFAR-100 only; each pruning space was small enough to enumerate, which enabled exact ground truth but makes the search easier than at realistic scale.

(b) **B. Models.** Three small CNNs; no larger CNN, transformer or language model. The deployment extension used a subset of these settings, and its final agent SimpleCNN only.

(c) **C. Optimisation.** PPO's search is far more expensive than a deterministic method; its results depend on reward design; LAMP remained stronger before recovery; and sensitivity guidance did not improve final accuracy.

(d) **D. Evaluation.** Validation-certified constraints transferred imperfectly to the test set and were sensitive to the fixed threshold and split size; deployment experiments used one recovery seed per model; and recovery used one fixed five-epoch protocol.

(e) **E. Hardware.** All latency and energy results come from one laptop CPU, at batch 1 and one thread, with no GPU, mobile, embedded or accelerator evaluation.

(f) **F. Energy.** Only CPU package energy was measured, with no memory or whole-system energy. The Phase-7B measurements failed their stability qualification, so there is no direct energy result for the final structured model.

(g) **G. Sustainability.** No carbon accounting, lifecycle analysis or assessment of training and search energy was carried out. The thesis is a sustainability-oriented evaluation of operational inference energy, not an environmental assessment.

8.6 Future Work

Each direction below follows from a finding or limitation of the thesis.

Short-term

(i) **Native structured RL pruning from the beginning.** Translating a masked policy into a structure was costly in accuracy (Phase 7A) and learning in the structured space narrowed that cost (Phase 7B), so future frameworks should search in the deployed structure from the start, including residual-coupled channel groups.

(ii) **Hybrid LAMP and RL.** Deterministic layer-adaptive scores could initialise, constrain or bias the RL search, combining LAMP's cheap reliability with the agent's ability to pursue objectives that one score cannot express.

(iii) **Better feasibility proxies.** The validation-to-test gap and the modest precision of the Phase-7B accuracy proxy call for calibrated predictors of post-recovery accuracy and for selection margins rather than exact thresholds.

(iv) **Stable external energy instrumentation.** The laptop's software energy counter failed its stability qualification. Future energy studies should use a logging wall-power meter, controlled thermal conditions and fixed processor clocks where possible, and qualify the instrument before any model comparison.

Medium-term

(v) **Multi-objective optimisation.** Explicit Pareto-front search over accuracy, memory, MACs, latency and measured energy, compared with deterministic baselines on the same fronts, would show trade-offs that a scalar reward hides.

(vi) **Improved generalisation constraints.** Multiple validation subsets, uncertainty-aware thresholds and more robust constraint formulations, because validation feasibility did not guarantee test feasibility.

(vii) **Larger architectures.** ResNet-18/34/50, MobileNet- and EfficientNet-class networks, or other CNNs whose pruning spaces cannot be enumerated, where a learned search may have more to offer.

(viii) **Larger datasets.** ImageNet-scale data, to test whether the retention, reward and recovery findings hold when each evaluation is much more costly.

(ix) **Edge and mobile deployment.** Mobile CPUs, ARM processors, embedded GPUs and accelerators.

(x) **GPU evaluation.** Whether the relationship between parameter reduction, MAC reduction and latency found on one CPU holds, weakens or reverses on GPUs.

Long-term

(xi) **Hardware-in-the-loop multi-objective RL.** Rewards using directly measured latency and energy rather than surrogates, attempted only after stable energy instrumentation (item (iv)) is available and with explicit accuracy constraints.

(xii) **Reliable lifecycle and carbon analysis.** Carbon-intensity conversion and lifecycle accounting become meaningful only once trustworthy direct energy measurements exist. Until then, no carbon impact should be claimed; this thesis makes no such claim.

8.7 Final Remarks

The thesis does not establish PPO as a universally superior pruning method. It shows that adaptive pruning must be evaluated as a complete pipeline: RL pruning is useful as a flexible allocator when its objective is explicitly aligned with the desired accuracy boundary, simpler deterministic methods remain stronger for ordinary compression objectives, and hardware-level validation must precede efficiency claims. The value of the work lies as much in the links it found to be weak as in those it showed to be sound.

REFERENCES

Abdelfattah, M. S., Mehrotra, A., Dudziak, Ł., and Lane, N. D. (2021). Zero-cost proxies for lightweight NAS, Proc. International Conference on Learning Representations (ICLR).

Achiam, J., Held, D., Tamar, A., and Abbeel, P. (2017). Constrained policy optimization, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 70, pp. 22–31.

Alwani, M., Wang, Y., and Madhavan, V. (2022). DECORE: Deep compression with reinforcement learning, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12339–12349. DOI: 10.1109/cvpr52688.2022.01203

Andrychowicz, M., Raichuk, A., Stańczyk, P., Orsini, M., Girgin, S., Marinier, R. et al. (2021). What matters for on-policy deep actor-critic methods? A large-scale study, Proc. International Conference on Learning Representations (ICLR).

Blalock, D., Gonzalez Ortiz, J. J., Frankle, J., and Guttag, J. (2020). What is the state of neural network pruning? Proc. Machine Learning and Systems (MLSys), Vol. 2, pp. 129–146.

Cai, H., Gan, C., Wang, T., Zhang, Z., and Han, S. (2020). Once-for-all: Train one network and specialize it for efficient deployment, Proc. International Conference on Learning Representations (ICLR).

Camci, E., Gupta, M., Wu, M., and Lin, J. (2022). QLP: Deep Q-learning for pruning deep neural networks, IEEE Transactions on Circuits and Systems for Video Technology, Vol. 32, No. 10, pp. 6488–6501. DOI: 10.1109/tcsvt.2022.3167951

Chen, T., Liang, L., Ding, T., Zhu, Z., and Zharkov, I. (2023). OTOv2: Automatic, generic, user-friendly, Proc. International Conference on Learning Representations (ICLR).

Cheng, H., Zhang, M., and Shi, J. Q. (2024). A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations, IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 46, No. 12, pp. 10558–10578. DOI: 10.1109/tpami.2024.3447085

Chin, T.-W., Ding, R., Zhang, C., and Marculescu, D. (2020). Towards efficient model compression via learned global ranking, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1515–1525. DOI: 10.1109/cvpr42600.2020.00159

Dehghani, M., Tay, Y., Arnab, A., Beyer, L., and Vaswani, A. (2022). The efficiency misnomer, Proc. International Conference on Learning Representations (ICLR).

Desislavov, R., Martínez-Plumed, F., and Hernández-Orallo, J. (2023). Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning, Sustainable Computing: Informatics and Systems, Vol. 38, Article No. 100857. DOI: 10.1016/j.suscom.2023.100857

Diao, E., Wang, G., Zhang, J., Yang, Y., Ding, J., and Tarokh, V. (2023). Pruning deep neural networks from a sparsity perspective, Proc. International Conference on Learning Representations (ICLR).

Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., and Clune, J. (2021). First return, then explore, Nature, Vol. 590, No. 7847, pp. 580–586. DOI: 10.1038/s41586-020-03157-9

El Halabi, M., Srinivas, S., and Lacoste-Julien, S. (2022). Data-efficient structured pruning via submodular optimization, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L. et al. (2020). Implementation matters in deep RL: A case study on PPO and TRPO, Proc. International Conference on Learning Representations (ICLR).

Evci, U., Gale, T., Menick, J., Castro, P. S., and Elsen, E. (2020). Rigging the lottery: Making all tickets winners, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 119, pp. 2943–2952.

Fang, G., Ma, X., Song, M., Mi, M. B., and Wang, X. (2023). DepGraph: Towards any structural pruning, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16091–16101. DOI: 10.1109/cvpr52729.2023.01544

Fang, G., Ma, X., Mi, M. B., and Wang, X. (2024). Isomorphic pruning for vision models, Computer Vision – ECCV 2024, Lecture Notes in Computer Science, Springer, pp. 232–250. DOI: 10.1007/978-3-031-73404-5_14

Frankle, J. and Carbin, M. (2019). The lottery ticket hypothesis: Finding sparse, trainable neural networks, Proc. International Conference on Learning Representations (ICLR).

Frantar, E. and Alistarh, D. (2022). Optimal brain compression: A framework for accurate post-training quantization and pruning, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Frantar, E. and Alistarh, D. (2023). SparseGPT: Massive language models can be accurately pruned in one-shot, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 202, pp. 10323–10337.

Gadhikar, A. H., Mukherjee, S., and Burkholz, R. (2023). Why random pruning is all we need to start sparse, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 202, pp. 10542–10570.

Ganjdanesh, A., Gao, S., and Huang, H. (2024). Jointly training and pruning CNNs via learnable agent guidance and alignment, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16058–16069. DOI: 10.1109/cvpr52733.2024.01520

Gu, S., Yang, L., Du, Y., Chen, G., Walter, F., Wang, J. et al. (2024). A review of safe reinforcement learning: Methods, theories and applications, IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 46, No. 12, pp. 11216–11235. DOI: 10.1109/tpami.2024.3457538

Han, S., Pool, J., Tran, J., and Dally, W. (2015). Learning both weights and connections for efficient neural network, Advances in Neural Information Processing Systems (NeurIPS), Vol. 28.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition, Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778. DOI: 10.1109/cvpr.2016.90

He, Y. and Xiao, L. (2024). Structured pruning for deep convolutional neural networks: A survey, IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 46, No. 5, pp. 2900–2919. DOI: 10.1109/tpami.2023.3334614

He, Y., Zhang, X., and Sun, J. (2017). Channel pruning for accelerating very deep neural networks, Proc. IEEE International Conference on Computer Vision (ICCV), pp. 1398–1406. DOI: 10.1109/iccv.2017.155

He, Y., Lin, J., Liu, Z., Wang, H., Li, L.-J., and Han, S. (2018). AMC: AutoML for model compression and acceleration on mobile devices, Computer Vision – ECCV 2018, Lecture Notes in Computer Science, Springer, pp. 815–832. DOI: 10.1007/978-3-030-01234-2_48

Henderson, P., Hu, J., Romoff, J., Brunskill, E., Jurafsky, D., and Pineau, J. (2020). Towards the systematic reporting of the energy and carbon footprints of machine learning, Journal of Machine Learning Research, Vol. 21, No. 248, pp. 1–43.

Hoefler, T., Alistarh, D., Ben-Nun, T., Dryden, N., and Peste, A. (2021). Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks, Journal of Machine Learning Research, Vol. 22, No. 241, pp. 1–124.

Holm, S. (1979). A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics, Vol. 6, pp. 65–70.

Hou, Z., Qin, M., Sun, F., Ma, X., Yuan, K., Xu, Y. et al. (2022). CHEX: CHannel EXploration for CNN model compression, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12277–12288. DOI: 10.1109/cvpr52688.2022.01197

Humble, R., Shen, M., Latorre, J. A., Darve, E., and Alvarez, J. (2022). Soft masking for cost-constrained channel pruning, Computer Vision – ECCV 2022, Lecture Notes in Computer Science, Springer, pp. 641–657. DOI: 10.1007/978-3-031-20083-0_38

Iofinova, E., Peste, A., Kurtz, M., and Alistarh, D. (2022). How well do sparse ImageNet models transfer? Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12256–12266. DOI: 10.1109/cvpr52688.2022.01195

Jay, M., Ostapenco, V., Lefevre, L., Trystram, D., Orgerie, A.-C., and Fichel, B. (2023). An experimental comparison of software-based power meters: Focus on CPU and GPU, Proc. IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid), pp. 106–118. DOI: 10.1109/ccgrid57682.2023.00020

Khan, K. N., Hirki, M., Niemi, T., Nurminen, J. K., and Ou, Z. (2018). RAPL in action: Experiences in using RAPL for power measurements, ACM Transactions on

Modeling and Performance Evaluation of Computing Systems, Vol. 3, No. 2, pp. 1–26. DOI: 10.1145/3177754

Krizhevsky, A. (2009). Learning multiple layers of features from tiny images, Technical Report, University of Toronto, Toronto, Canada.

Kuzmin, A., Nagel, M., van Baalen, M., Behboodi, A., and Blankevoort, T. (2023). Pruning vs quantization: Which is better? Advances in Neural Information Processing Systems (NeurIPS), Vol. 36.

Kwon, W., Kim, S., Mahoney, M. W., Hassoun, J., Keutzer, K., and Gholami, A. (2022). A fast post-training pruning framework for transformers, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Le, D. H. and Hua, B.-S. (2021). Network pruning that matters: A case study on retraining variants, Proc. International Conference on Learning Representations (ICLR).

LeCun, Y., Denker, J. S., and Solla, S. A. (1989). Optimal brain damage, Advances in Neural Information Processing Systems (NeurIPS), Vol. 2.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition, Proceedings of the IEEE, Vol. 86, No. 11, pp. 2278–2324. DOI: 10.1109/5.726791

Lee, J., Park, S., Mo, S., Ahn, S., and Shin, J. (2021). Layer-adaptive sparsity for the magnitude-based pruning, Proc. International Conference on Learning Representations (ICLR).

Lee, N., Ajanthan, T., and Torr, P. (2019). SNIP: Single-shot network pruning based on connection sensitivity, Proc. International Conference on Learning Representations (ICLR).

Li, B., Wu, B., Su, J., and Wang, G. (2020). EagleEye: Fast sub-net evaluation for efficient neural network pruning, Computer Vision – ECCV 2020, Lecture Notes in Computer Science, Springer, pp. 639–654. DOI: 10.1007/978-3-030-58536-5_38

Li, H., Kadav, A., Durdanovic, I., Samet, H., and Graf, H. P. (2017). Pruning filters for efficient ConvNets, Proc. International Conference on Learning Representations (ICLR).

Li, Y., Adamczewski, K., Li, W., Gu, S., Timofte, R., and Van Gool, L. (2022). Revisiting random channel pruning for neural network compression, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 191–201. DOI: 10.1109/cvpr52688.2022.00029

Liebenwein, L., Baykal, C., Carter, B., Gifford, D., and Rus, D. (2021). Lost in pruning: The effects of pruning neural networks beyond test accuracy, Proc. Machine Learning and Systems (MLSys), Vol. 3, pp. 93–138.

Liu, S., Chen, T., Chen, X., Shen, L., Mocanu, D. C., Wang, Z. et al. (2022). The unreasonable effectiveness of random pruning: Return of the most naive baseline

for sparse training, Proc. International Conference on Learning Representations (ICLR).

Liu, S., Chen, T., Zhang, Z., Chen, X., Huang, T., Jaiswal, A. K. et al. (2023). Sparsity may cry: Let us fail (current) sparse neural networks together! Proc. International Conference on Learning Representations (ICLR).

Liu, Z., Sun, M., Zhou, T., Huang, G., and Darrell, T. (2019). Rethinking the value of network pruning, Proc. International Conference on Learning Representations (ICLR).

Luccioni, A. S., Viguier, S., and Ligozat, A.-L. (2023). Estimating the carbon footprint of BLOOM, a 176B parameter language model, Journal of Machine Learning Research, Vol. 24, No. 253, pp. 1–15.

Luccioni, S., Jernite, Y., and Strubell, E. (2024). Power hungry processing: Watts driving the cost of AI deployment? Proc. ACM Conference on Fairness, Accountability, and Transparency (FAccT), pp. 85–99. DOI: 10.1145/3630106.3658542

Ma, N., Zhang, X., Zheng, H.-T., and Sun, J. (2018). ShuffleNet V2: Practical guidelines for efficient CNN architecture design, Computer Vision – ECCV 2018, Lecture Notes in Computer Science, Springer, pp. 122–138. DOI: 10.1007/978-3-030-01264-9_8

Ma, X., Fang, G., and Wang, X. (2023). LLM-Pruner: On the structural pruning of large language models, Advances in Neural Information Processing Systems (NeurIPS), Vol. 36.

Menghani, G. (2023). Efficient deep learning: A survey on making deep learning models smaller, faster, and better, ACM Computing Surveys, Vol. 55, No. 12, pp. 1–37. DOI: 10.1145/3578938

Molchanov, P., Tyree, S., Karras, T., Aila, T., and Kautz, J. (2017). Pruning convolutional neural networks for resource efficient inference, Proc. International Conference on Learning Representations (ICLR).

Molchanov, P., Mallya, A., Tyree, S., Frosio, I., and Kautz, J. (2019). Importance estimation for neural network pruning, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 11256–11264. DOI: 10.1109/cvpr.2019.01152

Nonnenmacher, M., Pfeil, T., Steinwart, I., and Reeb, D. (2022). SOSP: Efficiently capturing global correlations by second-order structured pruning, Proc. International Conference on Learning Representations (ICLR).

Oh, J., Guo, Y., Singh, S., and Lee, H. (2018). Self-imitation learning, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 80, pp. 3878–3887.

Pan, A., Bhatia, K., and Steinhardt, J. (2022). The effects of reward misspecification: Mapping and mitigating misaligned models, Proc. International Conference on Learning Representations (ICLR).

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G. et al. (2019). PyTorch: An imperative style, high-performance deep learning library, Advances in Neural Information Processing Systems (NeurIPS), Vol. 32.

Patterson, D., Gonzalez, J., Holzle, U., Le, Q., Liang, C., Munguia, L.-M. et al. (2022). The carbon footprint of machine learning training will plateau, then shrink, Computer, Vol. 55, No. 7, pp. 18–28. DOI: 10.1109/mc.2022.3148714

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations, Journal of Machine Learning Research, Vol. 22, No. 268, pp. 1–8.

Raffin, G. and Trystram, D. (2025). Dissecting the software-based measurement of CPU energy consumption: A comparative analysis, IEEE Transactions on Parallel and Distributed Systems, Vol. 36, No. 1, pp. 96–107. DOI: 10.1109/tpds.2024.3492336

Renda, A., Frankle, J., and Carbin, M. (2020). Comparing rewinding and fine-tuning in neural network pruning, Proc. International Conference on Learning Representations (ICLR).

Samsi, S., Zhao, D., McDonald, J., Li, B., Michaleas, A., Jones, M. et al. (2023). From words to watts: Benchmarking the energy costs of large language model inference, Proc. IEEE High Performance Extreme Computing Conference (HPEC), pp. 1–9. DOI: 10.1109/hpec58863.2023.10363447

Schulman, J., Moritz, P., Levine, S., Jordan, M. I., and Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation, Proc. International Conference on Learning Representations (ICLR).

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347.

Schwartz, R., Dodge, J., Smith, N. A., and Etzioni, O. (2020). Green AI, Communications of the ACM, Vol. 63, No. 12, pp. 54–63. DOI: 10.1145/3381831

Shen, M., Yin, H., Molchanov, P., Mao, L., Liu, J., and Alvarez, J. M. (2022). Structural pruning via latency-saliency knapsack, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Skalse, J., Howe, N., Krasheninnikov, D., and Krueger, D. (2022). Defining and characterizing reward gaming, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Strubell, E., Ganesh, A., and McCallum, A. (2019). Energy and policy considerations for deep learning in NLP, Proc. 57th Annual Meeting of the Association for Computational Linguistics, pp. 3645–3650. DOI: 10.18653/v1/p19-1355

Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. (2024). A simple and effective pruning approach for large language models, Proc. International Conference on Learning Representations (ICLR).

Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A. et al. (2019). MnasNet: Platform-aware neural architecture search for mobile, Proc. IEEE/CVF

Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2815–2823. DOI: 10.1109/cvpr.2019.00293

Tran, C., Fioretto, F., Kim, J.-E., and Naidu, R. (2022). Pruning has a disparate impact on model accuracy, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Tschand, A., Rajan, A. T. R., Idgunji, S., Ghosh, A., Holleman, J., Kiraly, C. et al. (2025). MLPerf Power: Benchmarking the energy efficiency of machine learning systems from μWatts to MWatts for sustainable AI, Proc. IEEE International Symposium on High Performance Computer Architecture (HPCA), pp. 1201–1216. DOI: 10.1109/hpca61900.2025.00092

Wang, K., Liu, Z., Lin, Y., Lin, J., and Han, S. (2019). HAQ: Hardware-aware automated quantization with mixed precision, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 8604–8612. DOI: 10.1109/cvpr.2019.00881

Wang, X., Na, C., Strubell, E., Friedler, S., and Luccioni, S. (2023). Energy and carbon considerations of fine-tuning BERT, Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 9058–9069. DOI: 10.18653/v1/2023.findings-emnlp.607

Wang, Z. and Li, C. (2022). Channel pruning via lookahead search guided reinforcement learning, Proc. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 3513–3524. DOI: 10.1109/wacv51458.2022.00357

Wen, W., Wu, C., Wang, Y., Chen, Y., and Li, H. (2016). Learning structured sparsity in deep neural networks, Advances in Neural Information Processing Systems (NeurIPS), Vol. 29.

Wilcoxon, F. (1945). Individual comparisons by ranking methods, Biometrics Bulletin, Vol. 1, No. 6, p. 80. DOI: 10.2307/3001968

Wu, C.-J., Raghavendra, R., Gupta, U., Acun, B., Ardalani, N., Maeng, K. et al. (2022). Sustainable AI: Environmental implications, challenges and opportunities, Proc. Machine Learning and Systems (MLSys), Vol. 4, pp. 795–813.

Xia, M., Gao, T., Zeng, Z., and Chen, D. (2024). Sheared LLaMA: Accelerating language model pre-training via structured pruning, Proc. International Conference on Learning Representations (ICLR).

Xu, P., Shao, W., Chen, M., Tang, S., Zhang, K., Gao, P. et al. (2024). BESA: Pruning large language models with blockwise parameter-efficient sparsity allocation, Proc. International Conference on Learning Representations (ICLR).

Yang, H., Yin, H., Shen, M., Molchanov, P., Li, H., and Kautz, J. (2023). Global vision transformer pruning with Hessian-aware saliency, Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 18547–18557. DOI: 10.1109/cvpr52729.2023.01779

Yang, T.-J., Chen, Y.-H., and Sze, V. (2017). Designing energy-efficient convolutional neural networks using energy-aware pruning, Proc. IEEE Conference on

Computer Vision and Pattern Recognition (CVPR), pp. 6071–6079. DOI: 10.1109/cvpr.2017.643

Yang, T.-J., Howard, A., Chen, B., Zhang, X., Go, A., Sandler, M. et al. (2018). NetAdapt: Platform-aware neural network adaptation for mobile applications, Computer Vision – ECCV 2018, Lecture Notes in Computer Science, Springer, pp. 289–304. DOI: 10.1007/978-3-030-01249-6_18

Yin, L., Wu, Y., Zhang, Z., Hsieh, C.-Y., Wang, Y., Jia, Y. et al. (2024). Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMs to high sparsity, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 235, pp. 57101–57115.

Youden, W. J. (1950). Index for rating diagnostic tests, Cancer, Vol. 3, No. 1, pp. 32–35. DOI: 10.1002/1097-0142(1950)3:1<32::aid-cncr2820030106>3.0.co;2-3

Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A. et al. (2022). The surprising effectiveness of PPO in cooperative multi-agent games, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Yu, K., Sciuto, C., Jaggi, M., Musat, C., and Salzmann, M. (2020). Evaluating the search phase of neural architecture search, Proc. International Conference on Learning Representations (ICLR).

Yu, S., Mazaheri, A., and Jannesari, A. (2022). Topology-aware network pruning using multi-stage graph embedding and reinforcement learning, Proc. International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, Vol. 162, pp. 25656–25667.

Yu, S., Yao, Z., Gholami, A., Dong, Z., Kim, S., Mahoney, M. W. et al. (2022). Hessian-aware pruning and optimal neural implant, Proc. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 3665–3676. DOI: 10.1109/wacv51458.2022.00372

Zhang, Y., Yao, Y., Ram, P., Zhao, P., Chen, T., Hong, M. et al. (2022). Advancing model pruning via bi-level optimization, Advances in Neural Information Processing Systems (NeurIPS), Vol. 35.

Zoph, B. and Le, Q. V. (2017). Neural architecture search with reinforcement learning, Proc. International Conference on Learning Representations (ICLR).

APPENDIX A
ARCHITECTURES, HYPERPARAMETERS AND REPRODUCIBILITY

The appendices give only the supporting values an examiner may need to verify statements in Chapters 3–6 without opening the code repository. Each table is generated from the frozen phase records by `tools/build_appendices.py` and names its source record.

A.1 Pruning units

Table A.1: Pruning units of the three networks: weights, share of prunable weights and MACs per image (CIFAR-10 output layer).

Network	Unit	Role	Weights	Share of prunable	MACs per image
SimpleCNN	features.0	pruned (PPO decision)	864	0.14%	884,736
SimpleCNN	features.3	pruned (PPO decision)	18,432	2.99%	4,718,592
SimpleCNN	features.6	pruned (PPO decision)	73,728	11.94%	4,718,592
SimpleCNN	classifier.1	pruned (PPO decision)	524,288	84.93%	524,288
SimpleCNN	classifier.3	never pruned	2,560	—	2,560
LeNet-5	conv1	pruned (PPO decision)	450	0.74%	352,800
LeNet-5	conv2	pruned (PPO decision)	2,400	3.94%	240,000
LeNet-5	fc1	pruned (PPO decision)	48,000	78.78%	48,000
LeNet-5	fc2	pruned (PPO decision)	10,080	16.54%	10,080
LeNet-5	classifier.5	never pruned	840	—	840
ResNet-8	stem	pruned (PPO decision)	432	0.56%	442,368
ResNet-8	stage1	pruned (PPO decision)	4,608	6.01%	4,718,592
ResNet-8	stage2	pruned (PPO decision)	14,336	18.69%	3,670,016
ResNet-8	stage3	pruned (PPO decision)	57,344	74.74%	3,670,016
ResNet-8	fc	never pruned	640	—	640

Source: `cpu_prunable_units.csv`. The PPO hyper-parameters are in Table 4.6 and the equal recovery protocol in Section 4.13.

A.2 Reproducibility

Every experiment was pre-registered and frozen before its confirmatory runs; Table 4.10 lists the governing records and the commits that first added them. The complete per-seed experimental records, per-session measurement records, machine-readable result files (CSV and JSON), checkpoint hashes and audit scripts are retained in the associated research repository, organised by phase: `architecture_generalization` (Phases 2–3), `phase4`, `phase5`, `phase6`, `paper3` (latency), `phase7a_sustainability`, the

Phase-7B folder and `phase7b_energy_confirmation`; the thesis sources and audits are in `thesis_final_writing`.

APPENDIX B
SUPPLEMENTARY CORE-PRUNING RESULTS

B.1 Phase 3 absolute accuracies

Table B.1: Phase 3 mean test accuracy (%) of PPO (P0) and the five baselines built at each PPO run's zero count (20 runs).

Setting (dense test)	P0 sparsity (%)	PPO (P0)	Uniform	Random	Global	ERK	LAMP
C10 SimpleCNN (77.03)	55.06	76.44	45.08	16.84	76.51	75.70	76.99
C10 LeNet-5 (64.96)	51.54	61.67	45.38	12.99	58.94	59.74	63.31
C10 ResNet-8 (78.69)	28.35	76.95	62.79	12.59	69.57	77.01	76.80
C100 SimpleCNN (50.50)	53.28	49.66	19.96	2.37	49.67	45.31	50.30
C100 LeNet-5 (32.28)	34.59	31.19	27.51	3.29	31.32	30.88	31.86
C100 ResNet-8 (43.99)	9.46	43.01	42.78	4.25	42.74	43.81	43.76

Source: `phase3_summary.csv`. *Paired differences and their statistics are in Table 5.2.*

B.2 Sensitivity vectors

Table B.2: Normalised loss sensitivity S_l of the four units by setting and the Phase-5 centred prior coefficients.

Setting	Configuration I (Phases 3–4)	Configuration II (Phase 5)	Phase-5 coefficients $c_l = S_l - \bar{S}$
C10 SimpleCNN	1.000, 0.032, 0.009, 0.000	1.000, 0.030, 0.012, 0.000	0.740, −0.230, −0.249, −0.260
C10 LeNet-5	1.000, 0.422, 0.046, 0.000	1.000, 0.440, 0.042, 0.000	0.629, 0.070, −0.329, −0.371
C10 ResNet-8	1.000, 0.138, 0.331, 0.000	1.000, 0.145, 0.320, 0.000	0.634, −0.221, −0.046, −0.366
C100 SimpleCNN	1.000, 0.103, 0.014, 0.000	1.000, 0.119, 0.010, 0.000	0.718, −0.164, −0.272, −0.282
C100 LeNet-5	1.000, 0.587, 0.000, 0.086	1.000, 0.564, 0.000, 0.059	0.594, 0.159, −0.406, −0.347
C100 ResNet-8	0.987, 0.310, 1.000, 0.000	1.000, 0.309, 0.997, 0.000	0.423, −0.268, 0.421, −0.577

Sources: `phase3_frozen_inputs.json`; `phase5_config.json`.

APPENDIX C
ARCHIVE, REWARD AND RECOVERY DIAGNOSTICS

C.1 Recovery gains under equal fine-tuning

Table C.1: Phase 6 mean recovery (test accuracy after minus before equal fine-tuning, pp; 20 runs) by setting and method.

Setting (sparsity)	PPO	LAMP	Global magnitude	Dense control
C10 SimpleCNN (56.9%)	+3.28	+3.01	+3.35	+3.05
C10 LeNet-5 (38.6%)	+1.04	+0.91	+2.42	+0.14
C10 ResNet-8 (24.5%)	+1.23	+0.70	+7.67	+0.14
C100 SimpleCNN (53.1%)	+0.46	+0.22	+0.76	+0.10
C100 LeNet-5 (30.6%)	+0.57	+0.44	+0.81	+0.11
C100 ResNet-8 (15.6%)	+1.20	+0.72	+4.69	+0.09

Source: `phase6_pre_post_accuracy.csv`. *Accuracies before and after recovery and the PPO–LAMP and PPO–Global gaps are in Table 5.7. Phase-4 search quality and Phase-5 per-run selections are summarised in Tables 5.3, 5.4 and 5.5; their run-level records are in the repository.*

APPENDIX D
HARDWARE AND STRUCTURED-PRUNING PROTOCOL

D.1 Latency-measurement calibration

Table D.1: H0 calibration: decision quantities of the three candidate measurement configurations.

Configuration	Signal (ms)	Noise (ms)	SNR	Relative reduction	Positive in all sessions	Usable
A	0.4127	0.0335	12.32	61.6%	True	True
B	0.1222	0.0287	4.26	31.8%	True	False
C	15.4531	1.6070	9.62	63.2%	True	True

Source: `h0_decision.json`. Rule: SNR ≥ 5, reduction ≥ 10%, positive in every session; preference order A, C, B.

D.2 Latency-table sessions

Table D.2: Exhaustive latency tables: accepted sessions, repeated attempts and anchor drift (limit 10%).

Latency table	Accepted sessions	Repeated attempts	Largest accepted drift	Largest rejected drift
ResNet-8, CIFAR-10	5	2	9.7%	27.1%
ResNet-8, CIFAR-100	5	0	9.6%	—
SimpleCNN, CIFAR-10	5	0	9.4%	—
SimpleCNN, CIFAR-100	5	2	9.1%	40.6%

Source: `h2_latency_freeze.json(session_log)`; the per-session log is in the repository.

APPENDIX E
ENERGY MEASUREMENT AND QUALIFICATION

E.1 Phase 7A energy sessions

Table E.1: Phase 7A energy: session summaries (five sessions per architecture).

Network	Method	Median J / 1,000	CV over sessions	Median saving	Min–max saving	Sessions below dense	One-sided Wilcoxon p
SimpleCNN	dense	11.054	2.14%	0.0%	0.0–0.0%	0	—
SimpleCNN	ppo	8.541	1.83%	22.8%	19.5–25.7%	5	0.031
SimpleCNN	uniform	7.906	1.54%	27.9%	27.3–31.2%	5	0.031
SimpleCNN	lamp	9.760	3.59%	12.1%	6.8–15.1%	5	0.031
ResNet-8	dense	10.882	1.84%	0.0%	0.0–0.0%	0	—
ResNet-8	ppo	9.592	2.84%	13.4%	5.6–14.5%	5	0.031
ResNet-8	uniform	9.645	3.63%	11.8%	4.7–13.9%	5	0.031
ResNet-8	lamp	9.819	1.46%	8.9%	7.4–12.3%	5	0.031

Source: `energy_summary.csv`; per-block session records are in the repository.

E.2 Energy-measurement qualification

Table E.2: Dense-only stability pilots and qualification (energy per 1,000 inferences, J; six dense blocks each).

Measurement	N per block	Dense blocks	CV	Limit	Decision
7A pilot, SimpleCNN	100,000	11.518, 11.150, 11.081, 11.253, 10.963, 10.768	2.30%	3%	passed
7A pilot, ResNet-8	100,000	10.793, 10.489, 10.555, 10.667, 10.358, 10.312	1.74%	3%	passed
7B surrogate-calibration pilot	20,000	12.200, 11.198, 11.107, 11.729, 11.433, 11.420	3.47%	4%	stable
7B confirmatory pilot (N = 100,000)	100,000	13.088, 11.414, 11.204, 15.125, 12.265, 12.273	11.36%	3%	unstable
7B confirmatory pilot (N = 200,000)	200,000	11.814, 12.074, 11.039, 11.539, 11.025, 11.033	4.01%	3%	unstable
7B energy-confirmation qualification	200,000	11.417, 14.278, 12.962, 15.221, 9.514, 9.473	19.92%	3%	not qualified

Sources: `pilot_n100000.json` (Phase 7A); `pilot_n20000.json`, `statistics.json` (Phase 7B); `measurement_qualification.json` (energy confirmation). A doubled-N Phase-7B attempt stopped by the load gate was declared void by a recorded decision of the author and is not listed. The per-block diagnostics and the eleven recovered Phase-7B models (summarised in Table 6.4) are in the repository.